



Politechnika Wrocławska

Wydział Zarządzania

kierunek studiów: Inżynieria zarządzania

specjalność: Zastosowanie IT w biznesie

Praca dyplomowa – inżynierska

**Projekt i opracowanie narzędzia do identyfikacji
kluczowych czynników sukcesu gier wideo studia
FromSoftware z wykorzystaniem technik
eksploracji tekstu**

Wiktor Zajda

słowa kluczowe:
techniki eksploracji
tekstu, python,
analiza wydźwięku

krótkie streszczenie:

W ramach niniejszej pracy opracowano
narzędzie w Python i Tableau Desktop,
które wspomaga proces identyfikacji
kluczowych czynników sukcesu gier
wideo studia FromSoftware

Opiekun/ka pracy dyplomowej	Dr inż. Marek Lubicz		
	Tytuł/stopień naukowy/imię i nazwisko		
Ostateczna ocena za pracę dyplomową			
Przewodniczący/a Komisji egzaminu dyplomowego			
	Tytuł/stopień naukowy/imię i nazwisko	ocena	podpis

Do celów archiwalnych pracę dyplomową zakwalifikowano do:*

a) kategorii A (akta wieczyste)

b) kategorii BE 50 (po 50 latach podlegające ekspertyzie)

*niepotrzebne skreślić

pieczęć wydziałowa

Wrocław, 2025

Streszczenie

Celem pracy było zaprojektowanie i opracowanie narzędzia analitycznego do identyfikacji kluczowych czynników sukcesu gier wideo studia FromSoftware przy użyciu technik eksploracji tekstu. Podjęcie tematu wynikało z problemu błędnej interpretacji specyficznego żargonu graczy przez standardowe rozwiązania analityczne. W ramach badań przeanalizowano recenzje z platformy Steam dla tytułów z gatunku soulslike. Projekt zrealizowano zgodnie z metodyką CRISP-DM, obejmując automatyczną akwizycję danych, ich preprocessing oraz modelowanie z wykorzystaniem metody TF-IDF i regresji logistycznej. Wynikiem prac jest narzędzie przekształcające dane nieustrukturyzowane w wiedzę biznesową, zaprezentowaną w formie interaktywnych pulpitów menedżerskich w środowisku Tableau. Rozwiązanie umożliwia ilościową ocenę wpływu czynników i ich składowych gry na jej odbiór. Czynniki o statystycznie istotnym, pozytywnym wpływie zdefiniowano jako kluczowe czynniki sukcesu. Wkład własny autora obejmuje zaprojektowanie pełnego potoku przetwarzania danych, budowę modelu klasyfikacyjnego oraz opracowanie wizualizacji wspierających procesy decyzyjne w branży gier.

Abstract

The aim of this thesis was to design and develop an analytical tool to identify key success factors for video games developed by FromSoftware, utilizing text mining techniques. The research was prompted by the business challenge of standard analytical solutions misinterpreting the specific jargon used by players. As part of the solution, reviews from the Steam platform for titles in the soulslike genre were analyzed. The project was executed in accordance with the CRISP-DM methodology, covering the process of automated data acquisition, preprocessing, and modeling using TF-IDF and logistic regression. The result is a tool capable of transforming unstructured data into valuable business intelligence, presented through interactive managerial dashboards in Tableau. This solution allows for a quantitative assessment of the impact of various game components and their constituents on player reception. Factors characterized by a statistically significant positive influence were defined as the key success factors. The author's original contribution includes the design of a comprehensive data pipeline, the construction of the classification model, and the development of visualizations supporting decision-making processes in the video game industry.

Spis treści

Wprowadzenie	2
1. Wstęp	3
1.1 Opis problemu biznesowego	3
1.2 Przedstawienie przedsiębiorstwa FromSoftware.....	3
1.3 Dostępne rozwiązania na rynku analizy opinii klientów.....	4
2. Przegląd literatury	6
2.1 Wprowadzenie do analizy tekstu w badaniach nad opiniami użytkowników.....	6
2.2 Etapy analizy tekstu w badaniach naukowych	7
2.2.1 Źródła danych tekstowych.....	7
2.2.2 Sposoby pozyskiwania danych (web scraping).....	7
2.2.3 Wstępne przetwarzanie danych (preprocessing)	9
2.2.4 Analiza wydźwięku	10
2.3 Metodyka CRISP-DM w projektowaniu systemów analitycznych.....	13
2.4 Przegląd technologii	15
2.4.1 Technologia do akwizycji i eksploracji danych	15
2.4.2 Technologia do modelowania predykcyjnego.....	18
2.4.3 Technologia do wizualizacji danych	19
3. Projekt narzędzia.....	22
3.1 Uwagi wstępne	22
3.2 Akwizycja i zrozumienie danych	23
3.3 Przygotowanie danych	24
3.4 Ekstrakcja czynników sukcesu	29
3.5 Projekt pulpitów	33
4. Implementacja.....	36
4.1 Budowa skryptu pobierającego dane z platformy Steam	36
4.2 Wstępne oczyszczenie danych	39
4.3 Eksploracyjna analiza danych	46
4.4 Analiza sentymentu	50
4.4.1 Ekstrakcja wydźwięku czynników	50
4.4.2 Segregacja tematyczna czynników	55
4.5 Budowa pulpitów menedżerskich	60
5. Ewaluacja.....	65
5.1 Analiza wyników modelu.....	65
5.2 Wyniki analizy danych przykładowych	67
5.3 Ocena w kontekście rozwiązania problemu biznesowego	74
5.4 Możliwość wdrożenia.....	75
5.5 Perspektywa rozszerzenia narzędzia	75
Zakończenie.....	77
Bibliografia.....	78
Spis rysunków	80
Spis tabel	82

Wprowadzenie

Gry wideo, niegdyś postrzegane wyłącznie jako forma rozrywki, dziś są doceniane jako istotny element współczesnej kultury i jeden z najważniejszych produktów przemysłu kreatywnego. Dynamiczny rozwój tego rynku charakteryzuje się ogromną konkurencyjnością, a sukces komercyjny produkcji zależy nie tylko od jej jakości, ale także od trafnego zaspokojenia oczekiwań graczy. W tym kontekście kluczowego znaczenia nabiera zdolność do analizy opinii użytkowników, które, dostępne masowo w serwisach recenzenckich, stanowią cenne źródło wiedzy o czynnikach decydujących o powodzeniu gry. Mimo to brakuje zautomatyzowanych rozwiązań pozwalających w sposób systematyczny i obiektywny identyfikować te czynniki, co stanowi istotny problem biznesowy dla deweloperów.

Wyzwanie to jest szczególnie widoczne w przypadku gier niszowych, które polaryzują społeczność graczy, takich jak produkcje z gatunku „soulslike”, zapoczątkowanego przez studio FromSoftware. Charakteryzują się one unikalnymi cechami, takimi jak wysoki poziom trudności i niebezpośrednia narracja, co sprawia, że standardowe narzędzia do analizy wydźwięku, często błędnie, interpretują specyficzny język i oczekiwania ich fanów. Słowa takie jak „trudne”, odbierane negatywnie przez typowe algorytmy, dla miłośników tego gatunku są synonimem pozytywnego wyzwania.

W odpowiedzi na te przesłanki, celem niniejszej pracy jest zaprojektowanie i stworzenie narzędzia do identyfikacji kluczowych czynników sukcesu gier wideo studia FromSoftware z wykorzystaniem technik eksploracji tekstu. Zakres pracy obejmuje analizę opinii graczy dotyczących tytułów FromSoftware oraz produkcji konkurencyjnych, aby wyodrębnić pozytywne i negatywne aspekty najczęściej poruszane w recenzjach. Do realizacji celu wykorzystano metody badawcze obejmujące przegląd literatury z zakresu przetwarzania języka naturalnego (NLP) i analizy sentymentu, a także analizę danych tekstowych pochodzących z platform dystrybucji cyfrowej, takich jak Steam. Projekt narzędzia oparto na metodyce CRISP-DM, która strukturyzuje proces od zrozumienia problemu biznesowego, przez przygotowanie i modelowanie danych, aż po ewaluację wyników. Praca przedstawia kolejne etapy tego procesu, od pozyskania i wstępnego przetworzenia danych, przez budowę modeli analitycznych, aż po wizualizację i interpretację końcowych

1. Wstęp

1.1 Opis problemu biznesowego

Współczesny rynek gier wideo charakteryzuje się ogromną konkurencyjnością, a sukces komercyjny danej produkcji coraz częściej zależy nie tylko od jakości wykonania, lecz także od trafnego zaspokojenia oczekiwań graczy. Wydawcy i producenci gier potrzebują narzędzi umożliwiających skuteczną analizę opinii użytkowników, by móc lepiej dopasowywać swoje produkty do preferencji rynku. Brakuje jednak zautomatyzowanych rozwiązań, które pozwalałyby w sposób systematyczny i obiektywny identyfikować czynniki wpływające na pozytywny lub negatywny odbiór gier.

Warto zaznaczyć, że gatunków, jak i podgatunków gier jest nieskończenie wiele. Od najprostszych gier mobilnych po złożone produkcje fabularne, strategie czasu rzeczywistego czy symulatory. Każdy z tych segmentów rządzi się innymi prawami i pokrywa różną część społeczności graczy. Użytkownicy mający silne powiązanie z konkretnym typem gry kreują personalne preferencje i oczekiwania w stosunku do podobnych w gatunku produktów (Peever, Johnson, Gardner, 2012). Tworzy to barierę do stworzenia uniwersalnego narzędzia analitycznego mającego na celu badanie opinii graczy. Jest to szczególnie zauważalne w przypadku tytułów, które mają niekonwencjonalne założenia i w znacznym stopniu polaryzują społeczność graczy. Takim typem gry jest „soulslike”, który zapoczątkowany został przez FromSoftware wraz z premierą Demon’s Souls w 2009 roku. Gatunek ten charakteryzuje się najwyższym poziomem trudności, wymagającym od użytkownika pełnego skupienia i zrozumienia mechaniki walki, jak nigdzie indziej. Dodatkowo charakteryzuje się nietypową narracją, która przekazuje fabułę w sposób niebezpośredni, aby potęgować poczucie zagubienia gracza w świecie gry. Tego rodzaju tytuły budzą skrajne emocje i polaryzują społeczność na wielbicieli produkcji, którzy doceniają głębię i stawiane przed nimi wyzwania, jak i krytyków podkreślających brak przystępności dla mniej doświadczonych graczy i nieklarowną historię przedstawioną.

Ten niszowy charakter gatunku i jego całkowicie odmienne podejście do doświadczenia gracza powoduje, że opinie użytkowników są niezrozumiałe dla aktualnych systemów analizy tekstu takich jak VADER i SentiWordNet co prowadzi do zdefiniowania problemu biznesowego przedsiębiorstwa FromSoftware:

- Brak zautomatyzowanych narzędzi analitycznych dostosowanych do specyfiki niszowych i polaryzujących gatunków gier (takich jak „soulslike”), co uniemożliwia producentom skuteczną interpretację opinii graczy oraz precyzyjne zidentyfikowanie kluczowych czynników decydujących o sukcesie i porażce ich produkcji.

Celem niniejszej pracy jest zaprojektowanie i stworzenie narzędzia do analizy opinii klientów, które pozwoli na identyfikację kluczowych czynników na sukcesy i porażki gier studia FromSoftware, poprzez wyodrębnienie pozytywnych i negatywnych aspektów z recenzji klientów. Przeanalizowane zostaną opinie klientów studia FromSoftware, jak i studiów konkurencyjnych, które tworzą gry w podobnym nurcie.

1.2 Przedstawienie przedsiębiorstwa FromSoftware

FromSoftware, Inc. to japońskie przedsiębiorstwo deweloperskie, które od momentu założenia w 1986 roku nieprzerwanie wyznacza nowe kierunki w światowej branży gier wideo. Początkowo firma koncentrowała się na oprogramowaniu biznesowym, jednak przełomem okazała się premiera gry King’s Field w 1994 roku, która zapoczątkowała zwrot ku produkcji gier komputerowych i konsolowych. Od tego czasu FromSoftware konsekwentnie budowało swoją pozycję, tworząc m.in. serie Armored Core oraz Echo

Night, jednak prawdziwy rozgłos i status jednej z najważniejszych firm w branży przyniosły jej tytuły: *Demon's Souls*, *Dark Souls*, *Bloodborne*, *Sekiro* *Shadows Die Twice* oraz *Elden Ring*

Kluczowym wyróżnikiem produkcji FromSoftware jest unikalna filozofia projektowania, która opiera się na wysokim poziomie trudności, immersyjnej eksploracji oraz narracji środowiskowej. Gry tego studia nie prowadzą gracza za rękę, lecz wymagają od niego zaangażowania, wytrwałości i samodzielnego odkrywania świata przedstawionego. To wymagające podejście jest fundamentem głębszej strategii, która stawia na pierwszym miejscu dbałość o gracza i dostarczenie mu jak najlepszego produktu, a nie na maksymalizację krótkoterminowego zysku. Jak wielokrotnie potwierdzał prezes studia, Hidetaka Miyazaki, obniżenie poziomu trudności tylko po to, by dotrzeć do szerszej publiczności i zwiększyć sprzedaż, "zniszczyłoby samą grę", ponieważ satysfakcja płynąca z pokonania wyzwania jest jej fundamentalną wartością (Miyazaki, 2024). FromSoftware traktuje gigantyczny sukces finansowy, jak w przypadku *Elden Ring* jako bezpośredni rezultat bezkompromisowego przywiązania do jakości i spójnej wizji artystycznej, a nie jako cel sam w sobie. Zyski są reinwestowane w tworzenie kolejnych, często niszowych projektów, co gwarantuje studiu wolność twórczą (Miyazaki & Yamamura, 2022). Takie podejście, oparte na głębokim szacunku dla inteligencji i determinacji odbiorcy, nie tylko zrewolucjonizowało gatunek fabularnych gier akcji, ale również zainspirowało powstanie całego podgatunku określanego mianem „soulslike”. Elementy tej filozofii, jak wskazują liczne analizy branżowe, są dziś obecne w wielu produkcjach konkurencyjnych studiów na całym świecie, które dostrzegły, że autentyczne wyzwanie buduje znacznie silniejszą i bardziej lojalną społeczność niż produkty nastawione wyłącznie na szybką konsumpcję.

Na szczególną uwagę zasługuje także kulturowa hybryda obecna w grach FromSoftware. Mimo że firma działa w Japonii, jej twórczość, zwłaszcza pod kierownictwem Hidetaki Miyazakiego, czerpie inspiracje zarówno z tradycji wschodnioazjatyckiej, jak i z mitologii, estetyki oraz struktur narracyjnych zachodu. Efektem jest wyjątkowa głębia światów przedstawionych, które są jednocześnie bliskie i egzotyczne dla odbiorcy zachodniego przez co pozwalają graczom z różnych kręgów kulturowych odnaleźć w nich elementy znane, a zarazem odkrywać nowe sensy i estetyki.

1.3 Dostępne rozwiązania na rynku analizy opinii klientów

Najpopularniejszym i najbardziej rozpoznawalnym rozwiązaniem na rodzimym oraz globalnym rynku monitoringu mediów jest Brand24. Jest to platforma skoncentrowana przede wszystkim na "social listeningu", której główną funkcją jest agregacja publicznie dostępnych wzmianek o marce (lub dowolnym zdefiniowanym słowie kluczowym) w czasie niemal rzeczywistym. System ten działa w oparciu o zautomatyzowane crawlersy, które nieustannie skanują szerokie spektrum źródeł internetowych od mediów społecznościowych (takich jak Facebook, Instagram, Twitter/X), przez fora dyskusyjne i blogi, aż po portale informacyjne i serwisy z recenzjami. Zebrane w ten sposób wzmianki są następnie poddawane podstawowej analizie sentymentu, która klasyfikuje wypowiedzi jako pozytywne, negatywne lub neutralne, co pozwala na szybką ocenę nastrojów wokół marki. Chociaż Brand24 jest narzędziem wysoce efektywnym w śledzeniu wolumenu dyskusji, zasięgu oraz dynamiki zmian zainteresowania danym tematem, posiada istotne ograniczenia w kontekście bardziej złożonych analiz semantycznych. Jego moduł analizy wydźwięku bazuje na ogólnych, predefiniowanych słownikach językowych, co czyni go w dużej mierze niezdolnym do poprawnej interpretacji specyficznego, niszowego żargonu. W przypadku branż posługujących się hermetycznym językiem, takich jak gry wideo czy specjalistyczne IT, algorytmy te mogą błędnie klasyfikować ironię, slang czy techniczne określenia jako nacechowane negatywnie, co zaburza rzeczywisty obraz opinii konsumentów.

Innym, znacznie potężniejszym graczem na tym rynku jest Brandwatch, który reprezentuje bardziej zaawansowaną kategorię narzędzi typu "Digital Consumer Intelligence". W odróżnieniu od prostego monitoringu, platforma ta łączy śledzenie mediów z głęboką analizą demograficzną i psychograficzną odbiorców, co pozwala na budowanie precyzyjnych profili konsumenckich. Wykorzystując zaawansowane algorytmy sztucznej inteligencji i uczenia maszynowego, Brandwatch nie ogranicza się jedynie do agregacji wzmianek. System ten stara się zrozumieć szersze trendy kulturowe, automatycznie segmentować konsumentów na podstawie ich zachowań oraz wizualizować niezwykle złożone zbiory danych w formie przejrzystych raportów. Mimo swojej imponującej mocy obliczeniowej i elastyczności, Brandwatch pozostaje jednak systemem horyzontalnym. Został on zaprojektowany do analizy szerokich rynków, co sprawia, że jego uniwersalne modele mogą wymagać znacznej kalibracji przy próbie zastosowania w bardzo specyficznych niszach, gdzie kontekst wypowiedzi jest kluczowy dla zrozumienia intencji użytkownika.

Z kolei najpełniejszym pod względem funkcjonalności i zakresu działania jest system Medallia. Jest to wiodące rozwiązanie klasy "Customer Experience Management" (CEM), które fundamentalnie różni się podejściem od typowego "social listeningu" nastawionego na media zewnętrzne. Medallia integruje dane zwrotne z niemal wszystkich możliwych punktów styku klienta z marką. Platforma ta agreguje nie tylko wyniki ankiet satysfakcji i opinie ze stron www, ale również analizuje transkrypcje rozmów telefonicznych z biurem obsługi klienta, zgłoszenia serwisowe, a także monitoruje recenzje online. Dzięki tak szerokiemu spektrum danych, Medallia umożliwia analizę zarówno ustrukturyzowanych, jak i nieustrukturyzowanych informacji zwrotnych. Nadrzędnym celem platformy jest stworzenie holistycznego obrazu "podróży klienta" i precyzyjna identyfikacja punktów problematycznych (pain points), które mogą prowadzić do odejścia klienta. Jest to zatem narzędzie służące nie tylko do obserwacji rynku, ale przede wszystkim do operacyjnego zarządzania doświadczeniem klienta wewnątrz organizacji.

W niniejszej pracy projektowane narzędzie wypełnia zidentyfikowaną lukę technologiczną, oferując rozwiązanie wertykalne, ściśle dopasowane do specyfiki branży gier wideo, w przeciwieństwie do horyzontalnych systemów rynkowych bazujących na uniwersalnych algorytmach. Kluczową przewagą jest zdolność do kontekstowej interpretacji hermetycznego żargonu graczy gatunku soulslike, gdzie standardowe słowniki sentymentu błędnie klasyfikują cechy takie jak „wysoki poziom trudności” jako negatywne. Zamiast ogólnego monitoringu wizerunku marki czy obsługi klienta, narzędzie koncentruje się na analizie produktowej, ekstrahując i kwantyfikując konkretne czynniki sukcesu, co dostarcza deweloperom precyzyjnych danych niezbędnych w procesie rozwoju gry.

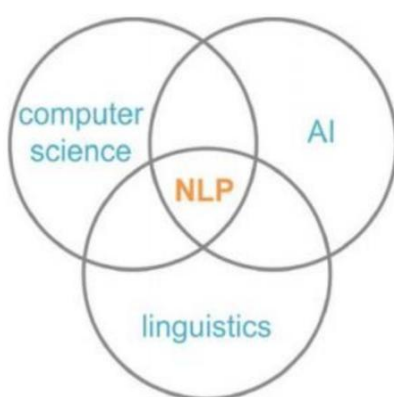
2. Przegląd literatury

2.1 Wprowadzenie do analizy tekstu w badaniach nad opiniami użytkowników

W erze cyfrowej, w której konsumenci mają niemal nieograniczoną możliwość dzielenia się swoimi opiniami online, analiza tekstu (ang. text mining) stała się jednym z najważniejszych narzędzi badawczych w zakresie rozpoznawania potrzeb i oczekiwań użytkowników. Opinie publikowane w Internecie, takie jak recenzje, komentarze, wpisy na forach, czy posty w mediach społecznościowych zawierają bogaty zbiór informacji, który może służyć jako podstawa do podejmowania decyzji biznesowych, projektowych i marketingowych (Feldman & Sanger, 2009).

W przeciwieństwie do tradycyjnych metod badań jakościowych, takich jak wywiady czy ankiety, dane tekstowe dostępne w sieci powstają spontanicznie, są naturalne i często bardziej autentyczne. Użytkownicy nie odpowiadają na narzucone pytania tylko sami decydują, co i w jaki sposób chcą przekazać. Z jednej strony czyni to analizę bardziej wartościową, z drugiej znacznie trudniejszą, ponieważ dane są nieustrukturyzowane, niejednorodne i często nacechowane emocjonalnie.

Z potrzeby automatyzacji przetwarzania dużych zbiorów nieustrukturyzowanych danych powstała analiza tekstu i techniki eksploracji tekstu. W przypadku badań nad grami wideo, opinie użytkowników są szczególnie zróżnicowane i pełne subiektywnych odczuć. Gracze opisują w nich nie tylko ogólne wrażenia, ale też oceniają konkretne elementy jakimi są mechaniki rozgrywki, grafikę, fabułę, optymalizację, poziom trudności, długość gry, muzykę, czy też wsparcie techniczne po premierze. Co więcej, często używają języka potocznego, slangu branżowego, a także ironii, sarkazmu i memów, co dodatkowo komplikuje analizę. Tradycyjne narzędzia statystyczne, czy prosty podział na pozytywne i negatywne komentarze są tu niewystarczające. Współczesna analiza tekstu wymaga zastosowania technik przetwarzania języka naturalnego (ang. Natural Language Processing, NLP), które umożliwiają głębsze zrozumienie treści zarówno na poziomie emocji, jak i tematyki wypowiedzi (Schuller, et al., 2013). Relację tych dziedzin oraz umiejscowienie NLP w nauce obrazuje Rysunek 1.



Rysunek 1 Przedstawienie istoty NLP
Źródło: (Obinnaya, 2023)

Natural Language Processing (NLP), czyli przetwarzanie języka naturalnego, stanowi interdyscyplinarną dziedzinę nauki, która umożliwia komputerom rozumienie, interpretowanie i generowanie ludzkiego języka w sposób zbliżony do naturalnej

komunikacji między ludźmi. Jak na Rysunku 1. NLP łączy w sobie osiągnięcia z zakresu informatyki, sztucznej inteligencji oraz lingwistyki. Dzięki integracji tych trzech dziedzin, NLP umożliwia automatyczną analizę i interpretację nieustrukturyzowanych danych tekstowych. W praktyce oznacza to możliwość wykrywania sentymentu, rozpoznawania tematów, identyfikowania ironii czy sarkazmu, a także ekstrakcji kluczowych informacji z dużych zbiorów opinii użytkowników. W kontekście badań nad grami wideo, zastosowanie NLP pozwala na znacznie bardziej precyzyjne i wielowymiarowe zrozumienie tego, jak gracze postrzegają poszczególne elementy gry oraz jakie aspekty mają kluczowe znaczenie dla ich satysfakcji i zaangażowania.

2.2 Etapy analizy tekstu w badaniach naukowych

2.2.1 Źródła danych tekstowych

Analiza tekstu, niezależnie od dziedziny zastosowania, wymaga dostępu do odpowiednio dużej i reprezentatywnej próbki danych tekstowych. Etap pozyskiwania danych jest więc fundamentem każdego procesu badawczego w tej dziedzinie. Jakość, różnorodność i adekwatność danych wpływają bezpośrednio na trafność wniosków oraz efektywność modeli analitycznych (Han, et al., 2011). W przypadku analizy opinii użytkowników o grach wideo, a zwłaszcza gier o wysokim poziomie emocjonalnego zaangażowania, jak produkcje typu soulslike, pozyskanie wiarygodnych i obszernych danych jest szczególnie istotne. Literatura i praktyka badawcza wskazują na kilka głównych źródeł danych tekstowych, które są przydatne w analizie opinii o grach.

W świecie gier wideo sztandarowym źródłem opinii są platformy dystrybucji cyfrowej takie jak Steam, GOG czy Epic Games Store. Zawierają one dziesiątki tysięcy recenzji użytkowników, które zawierają ocenę produktu, dodatkowy komentarz i kluczowe informacje o twórcy wpisu, jak czas dodania opinii, czy czas spędzony w grze. Wspomniane platformy stanowiąc będą główne źródło pozyskiwania danych z naciskiem na najpopularniejszą z platform, czyli Steam.

Kolejną kopalnią opinii są zewnętrzne strony agregujące oceny i recenzje użytkowników i krytyków, takie jak Metacritic. Dużym problemem takiego rozwiązania jest brak weryfikacji zakupu i gry przez wystawiającego opinię, co może prowadzić do nierzetelnych informacji. Są to też często kanały tak zwanego „review bombing”, czyli zjawiska polegającego na masowym, sztucznym wystawianiu negatywnych recenzji danemu produktowi, najczęściej grze wideo, filmowi, aplikacji lub innemu dziełu kultury cyfrowej, przez użytkowników serwisów recenzenckich. Celem takich działań jest zwykle wyrażenie sprzeciwu wobec określonych decyzji twórców, wydawców lub polityki firmy, a niekoniecznie rzetelna ocena jakości samego produktu. (Cantone, et al., 2024)

Ostatnim, lecz nie mniej ważnym źródłem opinii, są portale branżowe, które charakteryzują się wysokim poziomem formalizacji i ekspertyzy w recenzowaniu gier. Często opinie z tych portali są punktem odniesienia do opinii graczy.

2.2.2 Sposoby pozyskiwania danych (web scraping)

Identyfikacja źródła danych stanowi wstępny warunek analizy, jednak kluczowym wyzwaniem technicznym pozostaje dobór odpowiedniej metody ich pozyskania. Z uwagi na dużą skalę analizowanych zbiorów, obejmujących tysiące opinii, automatyzacja procesu akwizycji jest warunkiem koniecznym. Metodyka pozyskiwania danych jest ściśle uzależniona od architektury technicznej źródła, a przede wszystkim od tego, czy jest to strona statyczna, dynamiczna, czy też system udostępniający formalny Interfejs Programowania Aplikacji (API).

W przypadku stron statycznych i dynamicznych wykorzystujemy parsowanie, czyli przekształcanie surowego kodu HTML w zorganizowaną, hierarchiczną strukturę danych,

która łatwa jest do dalszego przetwarzania, nawigacji i manipulacji przez inne oprogramowanie analityczne. W literaturze wyróżniamy dwa podejścia do automatycznego gromadzenia danych ze stron internetowych (Mitchell, 2018). Pierwszym jest parsowanie stron statycznych. W takim przypadku proces scrapingu opiera się zazwyczaj na dwuetapowej procedurze. W pierwszym kroku skrypt wysyła żądanie do serwera i pobiera surową zawartość kodu HTML. W drugim kroku ten surowy tekst jest przekazywany do wyspecjalizowanego parsera, takiego jak powszechnie wykorzystywana w tym celu biblioteka BeautifulSoup. Narzędzie to przekształca pobrany tekst HTML w ustrukturyzowany, obiektowy model danych nazywany drzewem DOM (Document Object Model). Ta drzewiasta reprezentacja odzwierciedla pełną hierarchię i zależności między elementami na stronie, dzięki temu analityk może programistycznie nawigować po tej strukturze i precyzyjnie wyodrębnić pożądane treści, odwołując się do ich znaczników (np. `<div>`), atrybutów, klas CSS (np. `class="review-text"`) lub identyfikatorów. Metoda ta jest wysoce efektywna i szybka, jednak jej zastosowanie ogranicza się wyłącznie do stron statycznych.

W przypadku dynamicznych aplikacji webowych tradycyjne metody scrapingu, oparte na parserze HTML, okazują się niewystarczające. Dzieje się tak, ponieważ serwer najczęściej dostarcza jedynie początkowy "szkielet" strony, natomiast kluczowa treść, taka jak kolejne recenzje na liście, czy komentarze, są pobierane asynchronicznie. Proces ten inicjowany jest po stronie klienta, zazwyczaj przez kod JavaScript, który wykonuje dodatkowe zapytania do serwera w odpowiedzi na działania użytkownika, takie jak kliknięcie przycisku "Więcej" lub przewijanie strony. Parser pobierający jedynie pierwotną odpowiedź HTTP, nie wykonuje kodu JavaScript, przez co "widzi" wyłącznie niekompletny szkielet strony, pozbawiony dynamicznie ładowanych danych. Rozwiązaniem tego problemu jest zastosowanie narzędzi do automatyzacji przeglądarki, określanych mianem wirtualnych agentów. Wiodącą technologią w tej dziedzinie jest biblioteka Selenium języka Python. Fundamentem jej działania jest WebDriver, interfejs programistyczny, który pozwala skryptowi analitycznemu na programistyczne sterowanie pełnoprawną instancją przeglądarki. Skrypt sterujący może wówczas precyzyjnie symulować rzeczywiste działania użytkownika: odnajdywać elementy na podstawie ich selektorów CSS, klikać w przyciski, a przede wszystkim wykonywać operację przewijania strony, aby aktywować mechanizmy ujawniające się w trakcie przewijania strony, takie jak ładowanie dodatkowych recenzji czy komentarzy. Dopiero po pełnym wyrenderowaniu dynamicznej zawartości, skrypt pobiera z przeglądarki jej zaktualizowany kod źródłowy strony. Ten kompletny już kod HTML może być następnie przekazany do parsera, w celu właściwej ekstrakcji danych. Należy jednak zaznaczyć, że choć jest to metoda skuteczna, jest ona znacznie wolniejsza i bardziej zasobożerna niż bezpośrednie parsowanie statyczne.

Alternatywnym rozwiązaniem jest bezpośrednia komunikacja z Interfejsem Programowania Aplikacji (API) (McKinney, 2022). W przeciwieństwie do web scrapingu, który w praktyce polega na inżynierii wstecznej publicznie dostępnego kodu HTML, API stanowi formalny, udokumentowany i świadomie udostępniony przez dostawcę usługi kanał dostępu do danych. Kluczową zaletą jest format odpowiedzi. Zamiast nieustrukturyzowanego tekstu HTML, który wymaga skomplikowanego parsowania w celu ekstrakcji treści, API zwraca dane w ustandaryzowanym, maszynowo czytelnym formacie (zazwyczaj JSON). Struktura ta mapuje się bezpośrednio na obiekty i tabele wykorzystywane w dalszych etapach analizy, co drastycznie upraszcza proces i całkowicie eliminuje ryzyko błędów parsowania.

2.2.3 Wstępne przetwarzanie danych (preprocessing)

Wstępne przetwarzanie danych tekstowych, znane jako preprocessing, to jeden z najważniejszych i najbardziej fundamentalnych etapów w całym procesie eksploracji tekstu.

Choć często traktowany jest on jako techniczne przygotowanie do właściwej analizy, ma on zasadniczy wpływ na jakość i wiarygodność końcowych wyników. Surowe dane tekstowe, zwłaszcza te pochodzące ze źródeł internetowych, są z natury zanieczyszczone. Wyzwanie to jest szczególnie widoczne w przypadku recenzji graczy, które często zawierają niejednorodny język, błędy ortograficzne i gramatyczne, celowy lub niezamierzony slang, sarkazm oraz ironię. Dodatkowo, teksty te obfitują w znaki specjalne, emotikony, linki, a także treści niekonstrukttywne. Nadrzędnym celem preprocessingu jest transformacja tego nieustrukturyzowanego, zaszumionego tekstu w czysty, ujednolicony i ustandaryzowany format, który może zostać poddany dalszemu przetwarzaniu (Jurafsky & Martin, 2025). Proces ten realizowany jest poprzez następujące etapy:

- Normalizacja - Jest to pierwszy krok czyszczenia. Obejmuje on szereg operacji mających na celu ujednolicenie tekstu, takich jak konwersja wszystkich liter na małe (aby słowa „Gra” i „gra” były traktowane identycznie) oraz usunięcie elementów nie-tekstowych, które nie niosą wartości semantycznej, np. znaków interpunkcyjnych, liczb, znaczników HTML czy wspomnianych emotikony. Redukuje to zbędną złożoność słownika.
- Tokenizacja - Fundamentalny proces polegający na podziale oczyszczonego ciągu znaków (tekstu) na mniejsze jednostki analityczne, nazywane tokenami. Najczęściej tokenami są pojedyncze słowa, ale w zależności od potrzeb mogą to być również zdania. Na przykład, zdanie [„gra”, „jest”, „super”] staje się listą trzech tokenów. Tokenizacja jest warunkiem koniecznym dla wszystkich dalszych operacji.
- Usuwanie Stopwords - Po podziale tekstu na tokeny, kolejnym krokiem jest filtracja słów niewnoszących istotnego znaczenia analitycznego. Są to stopwords, czyli wyrazy powszechnie występujące w danym języku, takie jak spójniki, przyimki czy zaimki (np. „i”, „ale”, „w”, „jest”, „się”). Ich usunięcie pozwala algorytmom skupić się na tokenach niosących faktyczną treść merytoryczną i sentyment.

Tak przygotowany tekst jest następnie, albo przekazywany dalszym etapom analitycznym, jak analiza wydźwięku, lub jest poddawany procesowi generowania list słów, które zawierają częstość występowania słów w zakresie analizowanych opinii. Tak przygotowane listy są podstawą do wstępnych wniosków analitycznych. Zdarza się jednak również sytuacja, w której sama lista słów jest niewystarczająca poprzez ograniczenia występujące przez nieujednolicenie słów ze względu na rdzeń morfologiczny lub lingwistyczny. Przez zastosowanie tych metod tokeny „kolega” i „koledzy” są traktowane jako osobne, mimo że wynikają z jednego słowa i mają ten sam przekaz. Zniwelowanie tego problemu wymaga użycia jednej z dwóch poniższych technik:

- Stemming - To prostsza, algorytmiczna metoda wyszukiwania informacji o słowach poprzez mechaniczne wydobywanie „rdzenia” (ang. stem) słowa przez pozbywanie się końcówek leksykalnych i fleksyjnych. Przykładowo słowa „gracza”, „granie”, „grałem” mogą zostać sprowadzone do wspólnego rdzenia „gra” co ułatwia analizę i zapobiega rozważaniu się utworzonej listy słów przez słowa pokrewne.

- Lematyzacja - jest procesem bardziej zaawansowanym lingwistycznie, który wykorzystując słownik morfologiczny sprowadza słowa do jego formy podstawowej. Przykładowo słowa [„był”, „byli”, „będą”] zmienia się w [„być”], a [„lepszy”] w [„dobry”].

Tak przygotowane dane na tym poziomie mogą już być skutecznie analizowane. Jednak do pogłębienia analizy często wykorzystywane są całe frazy, zamiast pojedynczych słów. W tym celu wykorzystujemy generowanie n-gramów, które polega na łączeniu słów w sekwencyjne zbiory o określonej długości n (Jurafsky & Martin, 2025). N-gramy o długości $n = 2$ nazywamy bi-gramami, a przykładem takiego bi-gramu jest „projekt świata” czy „system walki”. W taki sposób możemy ponownie utworzyć listę n-gramów, występujących na przestrzeni analizowanych opinii. Tak utworzone listy stanowią bardzo ważny aspekt, jakim jest kontekst użycia. Słowo „unikalny” będzie miało inne znaczenie w przypadku sparowania ze słowem „smak”, a inne z słowem „wygląd”. N-gramy dają nam więc szerszą perspektywę na kontekst wykorzystywanych słów i przybliżają nas do prawdziwego sensu zdania.

2.2.4 Analiza wydźwięku

Analiza wydźwięku, w literaturze naukowej często określana również jako eksploracja opinii, stanowi fundamentalną poddziedzinę przetwarzania języka naturalnego (NLP). W swojej najszerzej definicji, sformułowanej przez Binga Liu, jest to dziedzina badań zajmująca się automatyczną identyfikacją, ekstrakcją, kwantyfikacją i analizą subiektywnych informacji, postaw, emocji i opinii zawartych w danych tekstowych. Celem jest przekształcenie masowo generowanych, nieustrukturyzowanych treści – takich jak recenzje produktów, komentarze w mediach społecznościowych, artykuły prasowe czy wpisy na forach w ustrukturyzowane dane, które pozwalają na zrozumienie nastrojów społecznych i postaw konsumenckich (Liu, 2015).

Analiza wydźwięku nie jest pojedynczym zadaniem, lecz obejmuje szerokie spektrum problemów badawczych. Należą do nich nie tylko podstawowa klasyfikacja polaryzacji (pozytywna, negatywna, neutralna), ale także bardziej złożone zadania, jak ekstrakcja informacji o opinii, podsumowywanie opinii, wykrywanie intencji, a nawet identyfikacja fałszywych opinii. Kluczowym wyzwaniem teoretycznym, na które wskazuje literatura jest istnienie „luki poznawczej” między rzeczywistym stanem psychologicznym autora a językiem, którego używa on do jego wyrażenia. Złożone konstrukcje językowe, takie jak sarkazm, ironia czy ukryte implikacje, sprawiają, że dosłowna interpretacja słów często prowadzi do błędnych wniosków. To fundamentalne założenie wyjaśnia, dlaczego proste metody analityczne napotykają na trudności w interpretacji bardziej złożonych lub specyficznych domenowo tekstów.

Literatura przedmiotu kategoryzuje zadania analizy sentymentu na podstawie poziomu granulacji na którym operuje analiza (Liu, 2015). Wybór odpowiedniego poziomu jest kluczowy dla celu badawczego.

- **Analiza na poziomie dokumentu (Document-Level SA)** Jest to najbardziej podstawowe podejście, w którym zakłada się, że cały dokument (np. jedna recenzja produktu, jeden artykuł) wyraża opinię na temat jednego obiektu. Zadanie polega na klasyfikacji całego tekstu do jednej, ogólnej kategorii sentymentu. Ograniczeniem tej metody jest jej niezdolność do radzenia sobie z dokumentami, które zawierają opinie na temat wielu różnych cech lub prezentują sprzeczne opinie.

- **Analiza na poziomie zdania (Sentence-Level SA):** Podejście to schodzi o poziom niżej, dążąc do określenia polaryzacji każdego pojedynczego zdania w dokumencie. Pozwala to na bardziej precyzyjne rozróżnienie, a także na kluczową w wielu zastosowaniach identyfikację zdań obiektywnych (podających fakty) i oddzielenie ich od zdań subiektywnych (wyrażających opinię).
- **Analiza na poziomie aspektu (Aspect-Based Sentiment Analysis - ABSA):** Jest to najbardziej szczegółowy, zaawansowany i najbardziej adekwatny do analizy recenzji poziom granulacji. ABSA nie koncentruje się na ogólnym wydźwięku tekstu, lecz dąży do zidentyfikowania konkretnych aspektów (cech, atrybutów, komponentów) analizowanego bytu oraz określenia sentymentu wyrażonego bezpośrednio wobec każdego z tych aspektów. Przykładowo, w recenzji restauracji („Jedzenie [aspekt 1] było pyszne [pozytywny], ale obsługa [aspekt 2] była powolna [negatywny]”) lub recenzji smartfona („Jakość głosu [aspekt 1] jest świetna [pozytywny], ale bateria [aspekt 2] jest słaba [negatywny]”), ABSA pozwala na uchwycenie tych niuansów.

W odpowiedzi na rosnącą złożoność języka opinii, w literaturze naukowej można zaobserwować wyraźną ewolucję metodologiczną, od systemów opartych na predefiniowanych słownikach, przez klasyczne uczenie maszynowe, aż po zaawansowane, kontekstowe modele głębokiego uczenia.

- **Podejście Oparte na Leksykonach i Zasadach (Lexicon-Based)**

Jest to historycznie najwcześniejsze podejście, które opiera się na założeniu, że sentyment tekstu można wiarygodnie przybliżyć poprzez agregację wartości sentymentu przypisanych do jego słów składowych. Metody te wykorzystują leksykony (słowniki sentymentu), w których każdemu słowu przypisano ocenę polaryzacji (np. „świetny” +2, „słaby” -1.5). Do kluczowych zasobów tej kategorii należą:

- **SentiWordNet:** Nie jest to prosty słownik, lecz zasób leksykalny zbudowany na bazie lingwistycznej bazy danych WordNet. Każdemu zbiorowi synonimów w WordNet przypisuje on trzy oceny: pozytywną, negatywną i obiektywną, co pozwala na uchwycenie wieloznaczności słów. Oceny te są generowane pół-automatycznie przy użyciu metod pół-nadzorowanego uczenia do analizy definicji słownikowych (Bacianella et al., 2010).
- **VADER:** Jest to model hybrydowy, oparty nie tylko na leksykonie, ale także na zestawie heurystycznych reguł. Został on specjalnie zaprojektowany i zoptymalizowany pod kątem analizy krótkich tekstów w mediach społecznościowych (np. Twitter). Jego siłą leży w zdolności do interpretowania modyfikatorów sentymentu. Uwzględnia on wpływ wielkich liter (np. „SUPER” jest bardziej pozytywne niż „super”), znaków interpunkcyjnych (np. „!!!”), słów wzmacniających (np. „bardzo”), a także slangu i emotikon (Hutto & Gilbert, 2014):.

Chociaż VADER i SentiWordNet są często wymieniane razem, reprezentują dwie różne filozofie. SentiWordNet jest zasobem szerokim, ogólnym i opartym na formalnej semantyce. VADER jest narzędziem wąskim, heurystycznym i już dostosowanym do specyficznej domeny języka mediów społecznościowych, gdzie w badaniach przewyższał skutecznością generyczne zasoby, w tym SentiWordNet.

Główną i wspólną wadą metod leksykalnych jest ich niska wrażliwość na kontekst. Traktują one słowa w izolacji, co czyni je bezradnymi wobec ironii, sarkazmu oraz, co najważniejsze, specyfiki domenowej, gdzie słowa mogą nabywać odmiennych, a nawet odwrotnych konotacji.

- **Podejście Oparte na Klasycznym Uczeniu Maszynowym (Supervised ML)**

W podejściu tym model statystyczny „uczy się” rozróżniać między tekstami pozytywnymi i negatywnymi na podstawie dostarczonego zbioru treningowego. Autorzy tego rozwiązania zastosowali klasyczne algorytmy uczenia maszynowego, takie jak Naive Bayes, Maximum Entropy oraz SVM do zadania klasyfikacji recenzji filmowych z bazy IMDb (Pang, et al., 2002). Kluczowym odkryciem tej pracy było stwierdzenie, że standardowe techniki ML, mimo że skuteczne, nie radziły sobie tak dobrze z klasyfikacją sentymentu, jak z tradycyjną klasyfikacją tematyczną. Wskazuje to na fundamentalną różnicę - klasyfikacja tematyczna często opiera się na obecności konkretnych słów kluczowych, podczas gdy sentyment jest często wyrażany w sposób subtelny, zależny od kontekstu, modyfikatorów i kolejności słów. Jak zauważono, to samo słowo (np. „nieprzewidywalny”) może mieć pozytywny wydźwięk w recenzji filmu („nieprzewidywalna fabuła”), ale negatywny w recenzji samochodu („nieprzewidywalne sterowanie”).

Wymusiło to rozwój inżynierii cech, czyli procesu przekształcania tekstu w wektory liczbowe, które oddają istotne dla sentymentu właściwości. Najczęściej stosowanymi cechami, obok prostego zliczania słów (ang. Bag of Words), stały się:

- **TF-IDF (Term Frequency-Inverse Document Frequency):** Metoda wagowa oceniająca istotność słowa w dokumencie na tle całego zbioru.
- **N-gramy:** Wykorzystanie sekwencji kolejnych słów (np. bi-gramy, tri-gramy) zamiast pojedynczych słów. Jest to kluczowe dla uchwycenia fraz, których znaczenie różni się od sumy znaczeń słów składowych (np. bi-gram „world design”).

Wśród algorytmów, SVM okazał się szczególnie skuteczny w klasyfikacji tekstu (i sentymentu), co literatura przypisuje ich zdolności do efektywnego działania w przestrzeniach o wysokiej wymiarowości (jakie generują dane tekstowe) i maksymalizacji marginesu między klasami (Pang, et al., 2002). Często jako silny model bazowy wykorzystywana jest również Regresja Logistyczna, szczególnie w połączeniu z cechami TF-IDF i n-gramami.

- **Podejście Oparte na Uczeniu Głębokim (Deep Learning)**

Ostatnia dekada przyniosła rewolucję w NLP w postaci modeli głębokiego uczenia (DL), które w dużej mierze zautomatyzowały proces inżynierii cech. Zamiast ręcznie definiować cechy (jak n-gramy czy TF-IDF), modele DL uczą się wielowymiarowej, semantycznej reprezentacji tekstu bezpośrednio z danych.

Pierwotnie dominowały tu Rekurencyjne Sieci Neuronowe (RNN) oraz ich zaawansowany wariant, LSTM (Long Short-Term Memory). Architektury te były naturalnym wyborem do przetwarzania tekstu, ponieważ przetwarzają dane sekwencyjnie, uwzględniając kolejność słów. Sieci LSTM, dzięki zastosowaniu mechanizmu bramek, rozwiązały problem „zanikającego gradientu” i stały się zdolne do przechwytywania długoterminowych zależności w tekście. Jest to kluczowe dla analizy sentymentu, gdzie słowo na początku zdania może wpływać na interpretację słowa na jego końcu.

Prawdziwy przełom nastąpił jednak wraz z publikacją architektury Transformer (Vaswani et al., 2017) oraz modeli na niej bazujących, przede wszystkim BERT (Devlin et al., 2018). Transformery zrezygnowały z kosztownej obliczeniowo rekurencji na rzecz przetwarzania równoległego. Serce tej architektury jest mechanizm uwagi.

Mechanizm uwagi pozwala modelowi, podczas przetwarzania jednego słowa, „spojrzeć” na wszystkie inne słowa w sekwencji i dynamicznie ocenić, które z nich są najważniejsze dla zrozumienia kontekstu tego słowa. W analizie sentymentu, mechanizm ten uczy się automatycznie skupiać na słowach-nośnikach emocji oraz ich powiązaniach z aspektami, o których mowa.

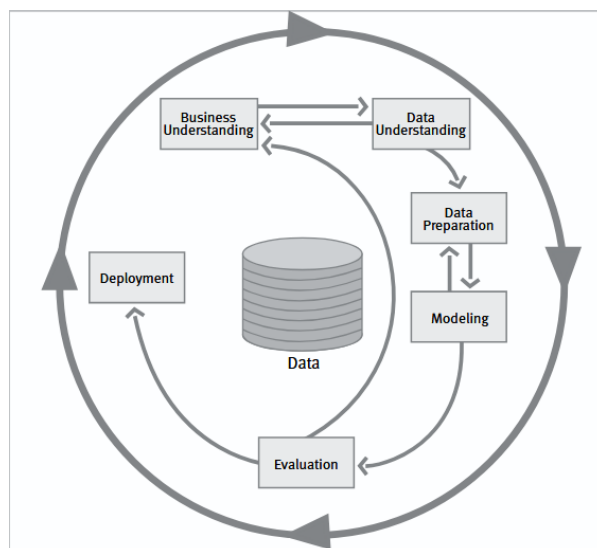
Pomimo ogromnego postępu technologicznego, analiza wydźwięku pozostaje problemem otwartym, a modele (nawet te oparte na architekturze Transformer) napotykają na fundamentalne wyzwania związane ze złożonością języka naturalnego.

- **Obsługa Negacji:** Negacje (np. „nie”, „nigdy”, „wcale nie”) i konstrukcje pokrewne (np. „trudno powiedzieć, by”) są kluczowe, ponieważ odwracają polaryzację wypowiedzi. Badania w tej dziedzinie skupiają się na identyfikacji wskaźnika negacji oraz zakresu negacji czyli fragmentu tekstu, którego dotyczy odwrócenie znaczenia. Złożone, wielokrotne negacje pozostają trudne do interpretacji.
- **Wykrywanie Sarkazmu i Ironii:** Jest to powszechnie uznawane za jedno z najtrudniejszych zadań w NLP. Sarkazm polega na celowym użyciu języka o pozytywnej polaryzacji do wyrażenia negatywnego sentymentu (np. „Świetna robota!” w odpowiedzi na błąd). Skuteczne wykrywanie jest często niemożliwe bez dostępu do szerszego kontekstu.

Aktualnie analiza wydźwięku rozwija się w kierunku rozwiązania tych problemów, ale przez specyfikę ludzkiej ekspresji, prawdopodobnie nigdy nie zostaną w pełni okiełznane.

2.3 Metodyka CRISP-DM w projektowaniu systemów analitycznych

CRISP-DM (Cross Industry Standard Process for Data Mining) to jedna z najczęściej stosowanych metodyk zarządzania projektami analitycznymi. Opracowana w latach dziewięćdziesiątych przez konsorcjum firm (IBM, Daimler-Benz, NCR i inne), do dziś pozostaje uniwersalnym standardem wykorzystywanym przy wdrażaniu rozwiązań opartych na analizie danych. Metodyka ta zakłada iteracyjny, uporządkowany i zrozumiały proces, który może być stosowany w dowolnej branży, niezależnie od rodzaju przetwarzanych danych liczbowych, tekstowych czy obrazowych. CRISP-DM składa się z sześciu głównych etapów, które prowadzą użytkownika od zdefiniowania problemu biznesowego, aż po wdrożenie gotowego rozwiązania. W kontekście projektowania systemów analitycznych, takich jak narzędzia do eksploracji opinii użytkowników, metodyka ta umożliwia efektywne zarządzanie zarówno stroną inżynierską, jak i organizacyjną przedsięwzięcia (Chapman, et al., 2000). Schematyczny przebieg procesu według metodyki CRISP-DM przedstawia Rysunek 2.



Rysunek 2 Schemat działania CRISP-DM
Źródło: (Chapman, et al., 2000)

Pierwszym i kluczowym krokiem jest jasne zdefiniowanie celu projektu z punktu widzenia użytkownika końcowego lub organizacji. Należy określić, jaką decyzję ma wspierać system analityczny, jakie są główne pytania badawcze oraz jakie wartości mają zostać dostarczone interesariuszom. W przypadku systemów analizujących opinie graczy, celem może być np. identyfikacja najczęstszych przyczyn krytyki produktu, ocena wpływu poziomu trudności na zadowolenie użytkownika czy porównanie postrzegania gier różnych producentów.

Drugi etap to zrozumienie danych, gdzie analizuje się dostępność, jakość i strukturę danych, które mają zostać użyte w projekcie. Dla systemów analitycznych opartych na danych tekstowych oznacza to m.in. identyfikację źródeł opinii, ocenę ich kompletności, języka, formy (długie recenzje, komentarze jednozdaniowe) oraz określenie potencjalnych ograniczeń - takich jak ironia, slang czy wielojęzyczność. Często przeprowadza się także wstępną eksplorację danych, by rozpoznać ich rozkład, długość oraz podstawowe statystyki opisowe.

Trzecim etapem są wszystkie działania niezbędne do przekształcenia surowych danych w zestaw gotowy do analizy. W przypadku danych tekstowych typowe czynności to: oczyszczanie tekstu ze znaków specjalnych, linków czy emotikon, tokenizacja, usunięcie słów nieinformatywnych, a także filtrowanie recenzji o zbyt małej wartości analitycznej. Jest to często najbardziej czasochłonny etap całego procesu, ale też kluczowy dla sukcesu kolejnych faz.

Następnie w czwartym kroku dobiera się i buduje modele analityczne w zależności od wcześniej określonych celów. Może to być np. klasyfikator wydźwięku recenzji (model sentymentu), klasteryzacja wypowiedzi o podobnej treści (modelowanie tematów), wykrywanie wzorców. W projektowaniu systemów tekstowych popularne są modele, takie jak regresja logistyczna, maszyny wektorów nośnych (SVM), drzewa decyzyjne, a także modele głębokie (RNN, BERT). CRISP-DM zakłada iteracyjność tego etapu, gdzie modele są testowane, porównywane, stroi się ich parametry i wybiera te najlepiej spełniające założenia projektu.

Kluczowym etapem jest ewaluacja modeli, czyli ocena jakości modelu nie tylko pod względem technicznym (np. dokładność, F1-score, precision-recall), ale przede wszystkim z perspektywy założeń biznesowych. Analizuje się, czy wytrenowany model odpowiada na

potrzeby zdefiniowane w pierwszym kroku, czy dostarcza wartościowych informacji dla użytkownika końcowego oraz czy jego interpretacja jest zrozumiała. W przypadku systemów tekstowych warto również przeprowadzić ocenę jakości danych wejściowych oraz skuteczność preprocessingu.

Ostatnim i często pomijanym w pracach naukowych fragmentem jest wdrożenie, które obejmuje przeniesienie rozwiązania z fazy testowej do środowiska produkcyjnego. W zależności od skali i celu projektu może to oznaczać stworzenie interaktywnego dashboardu, integrację modelu z aplikacją webową lub wygenerowanie raportu analitycznego dla zespołu projektowego. CRISP-DM podkreśla również znaczenie dokumentacji procesu, zapewnienia powtarzalności i przygotowania systemu do przyszłych aktualizacji, ponieważ dane i potrzeby biznesowe zmieniają się dynamicznie.

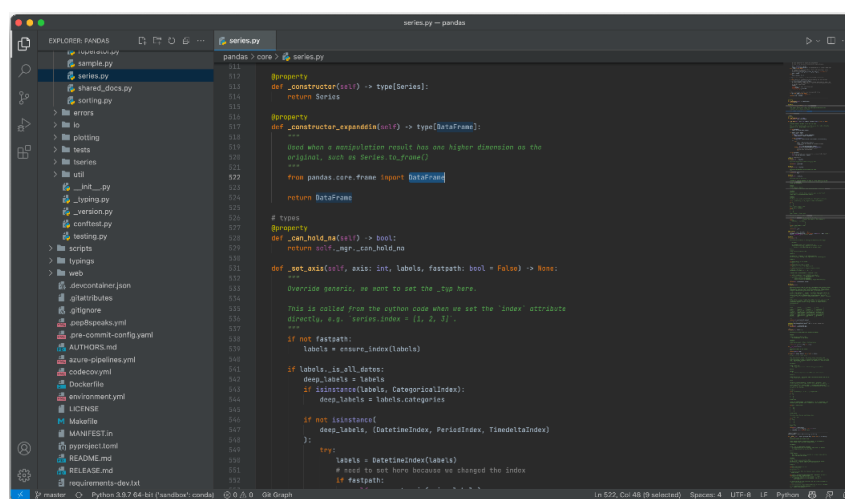
2.4 Przegląd technologii

2.4.1 Technologia do akwizycji i eksploracji danych

W dziedzinie data science i eksploracji danych, w szczególności analizy tekstu (text mining), dominują dwa kluczowe środowiska programistyczne - Python i R. Wybór między nimi jest fundamentalną decyzją na etapie projektowania architektury systemu analitycznego.

Python jest wieloplatformowym, interpretowanym językiem programowania ogólnego przeznaczenia, znanym ze swojej intuicyjnej składni. Chociaż pierwotnie nie był narzędziem statystycznym, jego ogromna elastyczność i rosnący ekosystem wyspecjalizowanych bibliotek uczyniły go dominującym narzędziem w procesach analizy danych.

W kontekście pozyskiwania danych, siła Pythona leży w jego zdolności do łatwej automatyzacji oraz integracji z systemami webowymi. Biblioteki takie jak Requests i BeautifulSoup umożliwiają efektywne pobieranie i parsowanie treści statycznych stron internetowych, a Selenium dynamicznych. Połączenie tych bibliotek stanowi fundament web scrapingu przy użyciu języka Python. Przykładowe środowisko programistyczne dla języka Python PyCharm JetBrains ukazano na Rysunku 3.



Rysunek 3 Środowisko PyCharm jako przykładowe IDE języka Python

Źródło: <https://github.com/nicohlr/vscode-pycharm-theme>

Po zebraniu danych, kluczową rolę w ich czyszczeniu i eksploracji odgrywa biblioteka Pandas. Jest ona przemysłowym standardem w zakresie manipulacji i analizy danych tabelarycznych, pozwalając na sprawne przekształcanie, filtrowanie i agregowanie danych

w celu przygotowania ich do analizy. Wszechstronność Pythona umożliwia także implementację złożonych algorytmów przy relatywnie niskim nakładzie pracy programistycznej. W kontekście przetwarzania języka naturalnego kluczową rolę odgrywa biblioteka NLTK (Natural Language Toolkit), która dostarcza niezbędnych narzędzi do lingwistycznej analizy tekstu. W ramach niniejszej pracy biblioteka ta stanowi fundament etapu preprocessingu, oferując szeroki wachlarz funkcji importowanych z poziomu głównego pakietu. Szczególne znaczenie mają tu dwa podmoduły - `nltk.corpus` oraz `nltk.stem`. Pierwszy z nich wykorzystywany jest do pobierania stopwords, co umożliwia redukcję szumu informacyjnego poprzez usunięcie wyrazów nieniosących unikalnego znaczenia. Z kolei moduł `nltk.stem` udostępnia klasę `WordNetLemmatizer`, która pozwala na przeprowadzenie procesu lematyzacji.

Procesy te są często realizowane w ramach interaktywnego środowiska Jupyter Notebook. Pozwala ono na łączenie kodu, jego wyników (np. fragmentów tabel, statystyk) oraz wizualizacji w jednym dokumencie. Umożliwia to iteracyjną eksplorację danych, szybkie testowanie hipotez i efektywną dokumentację procesu analitycznego. Wygląd interfejsu programistycznego wykorzystywanego w pracy przedstawiono na Rysunku 4.

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6
7 df = pd.read_csv('dark_souls_3_reviews.csv')
8 df.head()
9
10
11 df['review_text'] = df['review_text'].str.replace("Posted:.*?", "", regex=True)
12 df['date_posted'] = df['date_posted'].str.replace("Posted: ", "", regex=False)
13 df.head()
14
15
16 df['recommendation'] = df['recommendation'].replace({'Recommended': True, 'Not Recommended': False})
17

```

username	review_text	date_posted	recommendation
0 Aezarith	Posted: 13 October\nPlay this game before I se...	Posted: 13 October	Recommended
1 Big Fellas	Posted: 13 October\nReally great so far!	Posted: 13 October	Recommended
2 812	Posted: 13 October\nfire keeper <3	Posted: 13 October	Recommended
3 Aeth	Posted: 13 October\nGreat classic.	Posted: 13 October	Recommended
4 voltorstone	Posted: 13 October\nDifficult but worth every ...	Posted: 13 October	Recommended

username	review_text	date_posted	recommendation
0 Aezarith	Play this game before I send a homeless man ar...	13 October	Recommended
1 Big Fellas	Really great so far!	13 October	Recommended
2 812	fire keeper <3	13 October	Recommended
3 Aeth	Great classic.	13 October	Recommended
4 voltorstone	difficult but worth every agony, its like the ...	13 October	Recommended

Rysunek 4 Wygląd interfejsu środowiska PyCharm jako przykładowego IDE dla języka Python

Źródło: opracowanie własne

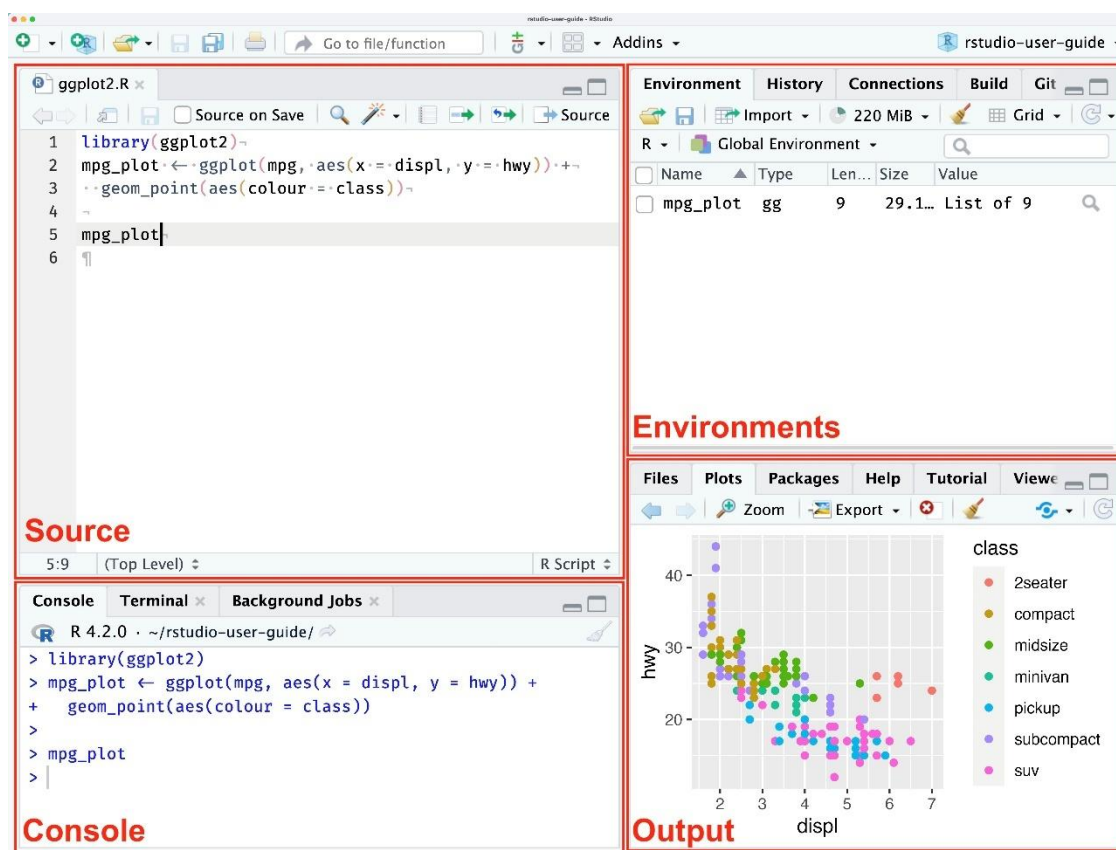
Połączenie tych elementów czyni Pythona wysoce efektywnym narzędziem do budowania kompletnych potoków przetwarzania danych, obejmujących etapy od akwizycji, przez czyszczenie, aż po ich wstępną eksplorację.

R jest językiem programowania i środowiskiem pierwotnie zaprojektowanym do analizy statystycznej i wizualizacji danych. Chociaż jego korzenie są głęboko akademickie i statystyczne, jego ogromna elastyczność i rozbudowany ekosystem pakietów (CRAN) uczyniły go potężnym narzędziem w szeroko rozumianej analizie danych. W kontekście pozyskiwania danych, siła R przejawia się w wyspecjalizowanych pakietach. Narzędzia takie jak `rvest` (często używany w połączeniu z `httr`) umożliwiają efektywne pobieranie i parsowanie treści stron internetowych, co jest fundamentem procesów web scrapingu. Po zebraniu danych, kluczową rolę w ich czyszczeniu i eksploracji odgrywa ekosystem Tidyverse, a w szczególności pakiet `dplyr`. Jest on nowoczesnym standardem w zakresie

manipulacji danymi, pozwalając na sprawną i czytelną transformację (dzięki operatorowi "pipe"), filtrowanie i agregowanie danych w celu przygotowania ich do analizy.

W domenie analizy tekstu (text mining), środowisko R oferuje dwa główne podejścia realizowane przez pakiety tm (Text Mining Package) oraz tidytext. Biblioteka tm jest klasycznym narzędziem służącym do tworzenia zbiorów tekstowych, zarządzania metadanymi oraz wykonywania standardowych operacji czyszczenia, takich jak usuwanie słów przystankowych (stopwords) czy stemming. Z kolei pakiet tidytext integruje analizę tekstu z paradygmatem Tidyverse, umożliwiając traktowanie tekstu jako uporządkowanych tabel (jedna leksem na wiersz). Takie podejście pozwala na płynne wykorzystanie znanych narzędzi do manipulacji danymi, jak dplyr czy ggplot2, do wizualizacji częstości słów czy analizy sentymentu, jednak może być mniej wydajne przy wdrażaniu złożonych potoków produkcyjnych w porównaniu do rozwiązań pythonowych.

Procesy te są najczęściej realizowane w ramach zintegrowanego środowiska programistycznego RStudio. Zapewnia ono potężne narzędzia łączące edytor kodu, konsolę, okna wizualizacji i zarządzanie plikami. Interfejs środowiska RStudio zaprezentowano na Rysunku 5. Połączenie tych elementów czyni R wysoce efektywnym narzędziem do budowania kompletnych potoków przetwarzania danych, obejmujących etapy od akwizycji, przez czyszczenie, aż po ich wstępną eksplorację.



Rysunek 5 Środowisko RStudio jako przykładowe IDE języka R

Źródło: <https://docs.posit.co/ide/user/ide/get-started/>

W kontekście niniejszej pracy spośród dwóch środowisk wybrany został Python. Ostateczny wybór Pythona jako wiodącego narzędzia w niniejszej pracy podyktowany jest jego uniwersalnością oraz zdolnością do obsługi całego procesu inżynierii danych w jednolitym środowisku programistycznym. Decyzja ta wynika z konieczności wyjścia poza ramy standardowej analizy statystycznej na rzecz budowy elastycznego rozwiązania

inżynierskiego. W przyjętym kontekście Python posłuży przede wszystkim do zaprojektowania i implementacji dedykowanego webscrapera, co pozwoli na zautomatyzowane pozyskiwanie materiału badawczego. Ponadto, to właśnie w tym środowisku zrealizowana zostanie zasadnicza część eksploracji zgromadzonych danych, zarówno o charakterze numerycznym, jak i tekstowym. Wykorzystanie bogatego ekosystemu bibliotek NLP umożliwi efektywne przeprowadzenie kluczowych etapów wstępnego przetwarzania, obejmujących detekcję i usunięcie treści typu spam, stopwords oraz lematyzację. Taka architektoniczna spójność, eliminująca konieczność migracji między różnymi programami, sprawia, że Python zostanie efektywnie wykorzystany również do realizacji pozostałych celów pracy.

2.4.2 Technologia do modelowania predykcyjnego

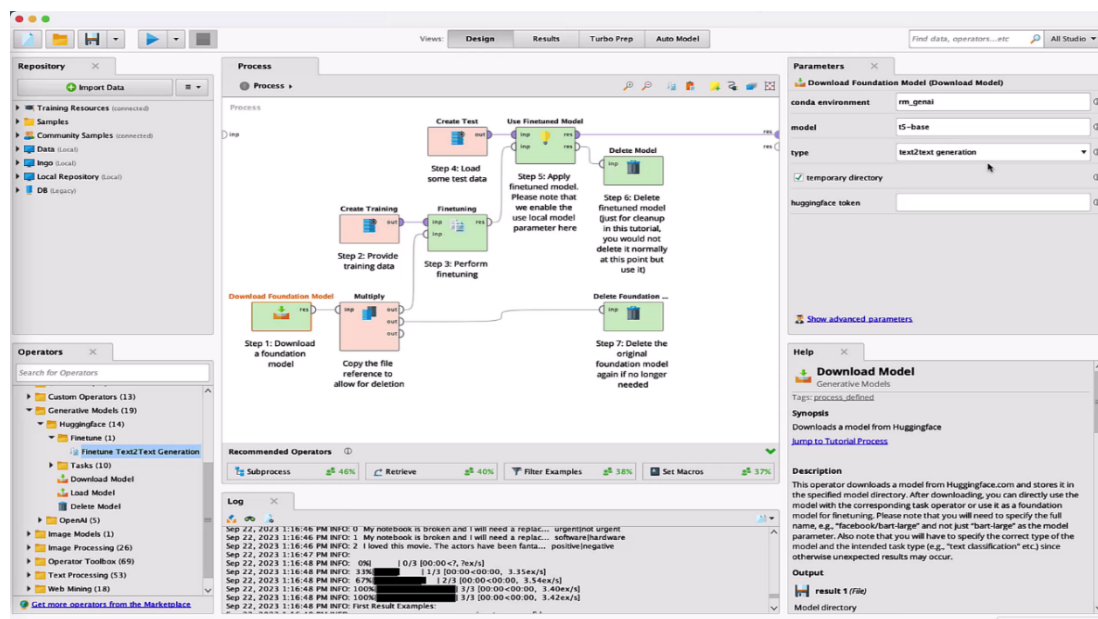
W kontekście niniejszej pracy kluczowym etapem w procesie analizy sentymentu jest modelowanie predykcyjne, oparte na algorytmach klasyfikacyjnych. Istotą tego procesu nie jest wyłącznie sama kategoryzacja dokumentów, ale mechanizm uczenia maszynowego, który pozwala na zidentyfikowanie zależności między konkretnymi słowami a oceną końcową. Model, analizując zbiór opinii oznaczonych jako pozytywne lub negatywne, uczy się przypisywać wagi poszczególnym tokenom. W rezultacie możliwe staje się wygenerowanie listy słów i fraz o jednoznacznie pozytywnym lub negatywnym zabarwieniu emocjonalnym. Na rynku technologii analitycznych dominują obecnie dwa odmienne podejścia do realizacji tego zadania: podejście programistyczne, oparte na języku Python i jego rozbudowanym ekosystemie bibliotek, lub wykorzystanie środowisk typu low-code/no-code, reprezentowanych przez platformy takie jak Altair AI Studio.

W podejściu programistycznym wiodącą rolę odgrywa biblioteka Scikit-learn (sklearn). Jest to kompleksowe narzędzie dla języka Python, dostarczające zoptymalizowanych implementacji algorytmów uczenia maszynowego oraz metod transformacji danych. Fundamentalnym elementem oferowanym przez tę bibliotekę jest możliwość wektoryzacji tekstu, realizowana przez TfidfVectorizer. To rozwiązanie implementuje metodę TF-IDF, która pozwala na przekształcenie surowego tekstu w reprezentację numeryczną, zrozumiałą dla algorytmów. Technika ta waży znaczenie słów, promując terminy specyficzne dla danego dokumentu, a redukując wpływ słów powszechnych. Narzędzie to umożliwia analizę nie tylko pojedynczych słów, ale także sekwencji sąsiadujących wyrazów (ngramów), co pozwala na zachowanie kontekstu i wyłapywanie fraz zmieniających znaczenie (np. negacji).

W ramach biblioteki Scikit-learn dostępny jest szeroki wachlarz algorytmów klasyfikacyjnych, wśród których szczególną rolę w analizie tekstu odgrywa Regresja Logistyczna. Jest to liniowy model klasyfikacyjny, służący do szacowania prawdopodobieństwa przynależności obserwacji do jednej z dwóch klas (binarna klasyfikacja sentymentu). Istotną przewagą Regresji Logistycznej nad bardziej złożonymi algorytmami, takimi jak RandomForest czy SVM, jest jej transparentność. W przeciwieństwie do modeli typu black box, nie udostępniającymi informacji o ich wewnętrznym działaniu, regresja logistyczna jest modelem interpretowalnym. Każdemu tokenowi przypisuje ona jawną wagę (współczynnik). Dodatnia wartość współczynnika wskazuje na korelację ze zmienną celu (sentyment pozytywny), natomiast ujemna na korelację odwrotną (sentyment negatywny). Ta właściwość sprawia, że technologia ta jest idealnym narzędziem do budowy słowników sentymentu, pozwalając na bezpośrednie odczytanie siły i wpływu poszczególnych słów.

Alternatywą dla środowisk programistycznych są platformy klasy Visual Data Science, których czołowym przedstawicielem jest Altair AI Studio. Jest to środowisko typu low-code/no-code, które umożliwia projektowanie zaawansowanych procesów analitycznych za

pomocą graficznego interfejsu użytkownika. Oprogramowanie oferuje gotowe, konfigurowalne bloki operacyjne do wczytywania danych, przetwarzania tekstu oraz trenowania modeli, w tym również regresji logistycznej. Interfejs takiego rozwiązania na przykładzie Altair AI Studio widoczny jest na Rysunku 6.



Rysunek 6 Interfejs Altair AI Studio

Źródło: <https://www.g2.com/products/rapidminer-studio/reviews>

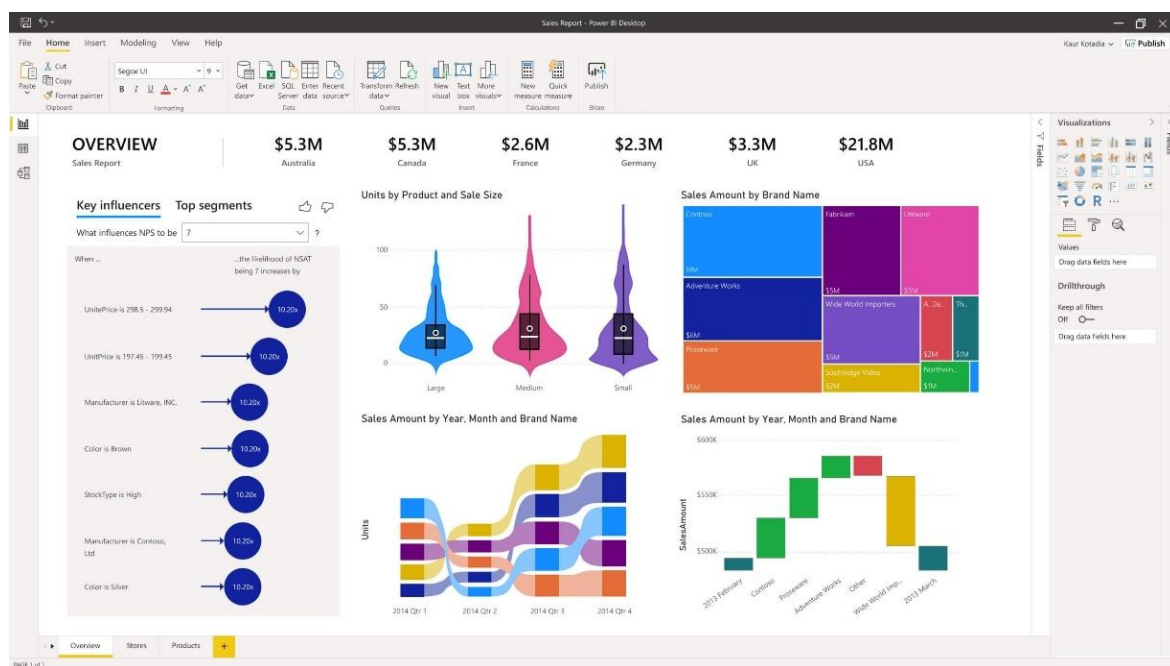
Rozwiązanie to pozwala na szybkie prototypowanie, wizualizację przepływu danych oraz weryfikację wyników bez konieczności pisania złożonego kodu, co czyni je popularnym wyborem w zastosowaniach biznesowych oraz jako narzędzie weryfikacyjne dla rozwiązań programistycznych.

Pomimo wygody użytkowania struktury low-code/no-code reprezentowanej przez Altair AI Studio w niniejszej pracy wybrano język Python do części analitycznej. Decyzja ta podyktowana jest przede wszystkim higienie pracy opartej na kodzie, która zapewnia znacznie głębszą kontrolę nad strukturą i parametrami modelu w porównaniu do gotowych rozwiązań wizualnych. Ponadto środowisko to charakteryzuje się wyższą wydajnością obliczeniową. Eliminacja narzutu zasobów niezbędnych do obsługi graficznego interfejsu użytkownika platform typu Altair, pozwala na zminimalizowanie czasu obliczeń. Co równie istotne, wybór ten gwarantuje pełną spójność technologiczną z opracowanymi wcześniej modułami akwizycji i manipulacji danymi. Utrzymanie całego potoku przetwarzania w ramach jednego ekosystemu sprawia, że poszczególne elementy rozwiązania idealnie ze sobą współgrają, eliminując problemy kompatybilności.

2.4.3 Technologia do wizualizacji danych

Warstwa prezentacji wyników stanowi finalny i krytyczny element architektury każdego systemu analitycznego, pełniąc funkcję interfejsu pomiędzy surowymi wynikami obliczeń a odbiorcą końcowym. Jej fundamentalnym zadaniem jest translacja wielowymiarowych zbiorów danych na skondensowane i interpretowalne wnioski, które mogą stanowić podstawę do podejmowania decyzji strategicznych. Na rynku rozwiązań klasy Business Intelligence (BI) dominują obecnie dwa wiodące ekosystemy - Microsoft Power BI oraz Tableau, różniące się filozofią przetwarzania danych i naciskiem na poszczególne etapy procesu analitycznego.

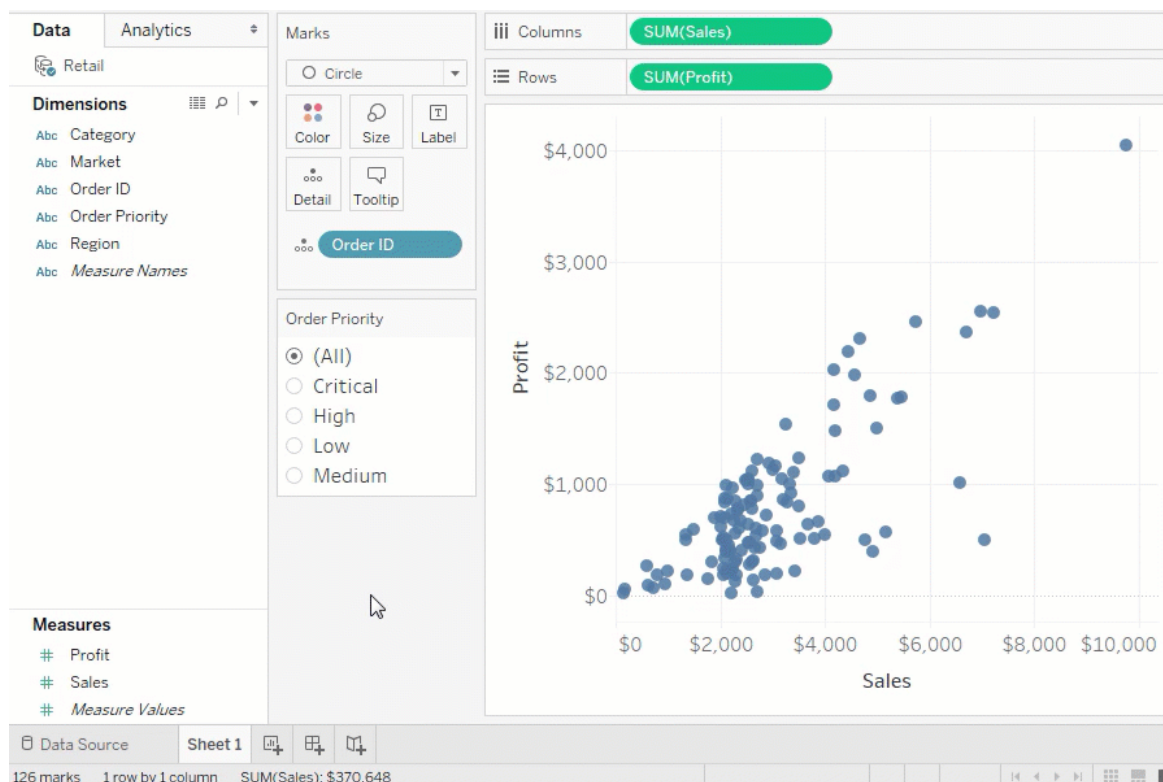
Microsoft Power BI to kompleksowa, zintegrowana platforma analityczna klasy Business Intelligence, stanowiąca strategiczną odpowiedź Microsoft na rosnące potrzeby rynku analitycznego. Jej absolutnie największą przewagą konkurencyjną jest natywna, głęboka integracja z całym ekosystemem Microsoft. Obejmuje to bezproblemową współpracę z Excelem (możliwość analizy modeli z Excela w Power BI i odwrotnie), źródłami danych w chmurze Azure (np. Azure SQL Database, Synapse Analytics) oraz narzędziami kolaboracji, jak Teams czy SharePoint, co ułatwia dystrybucję raportów w organizacji. Przykładowy dashboard analityczny w środowisku MS Power BI przedstawia Rysunek 7.



Rysunek 7 Wygląd interfejsu MS Power BI
Źródło: <https://omegacode.pl/produkty/power-bi>

Power BI jest ceniony za swoje potężne, wiodące na rynku możliwości w zakresie modelowania danych. Silnik Power Query (wspólny z Excelem, oparty na języku M) dostarcza zaawansowane funkcje ETL (Extract, Transform, Load) do czyszczenia i transformacji danych z wielu źródeł. Z kolei silnik modelujący oparty na języku DAX (Data Analysis Expressions) pozwala na tworzenie złożonych modeli relacyjnych i kalkulacji, podobnych do tych z SSAS Tabular. Power BI jest więc narzędziem zaprojektowanym jako kompletne, zunifikowane rozwiązanie BI, łączące funkcje ETL, modelowania i wizualizacji.

Z kolei **Tableau Desktop** jest narzędziem powszechnie uznawanym za historycznego i obecnego lidera w dziedzinie wizualizacji danych i eksploracyjnej analizy. W przeciwieństwie do Power BI, którego fundamentalna siła leży w integracji ekosystemowej i zaawansowanym modelowaniu (ETL/DAX), Tableau historycznie koncentrowało się na byciu absolutnie najlepszym w swojej klasie narzędziem do wizualnej, interaktywnej eksploracji. Filozofia Tableau opiera się na umożliwieniu użytkownikom "analizy w tempie myśli". Interfejs środowiska Tableau Desktop, wspierający to podejście, widoczny jest na Rysunku 8.



Rysunek 8 Wygląd środowiska Tableau Desktop
 Źródło: https://www.tableau.com/__test__/table-of-contents

Sercem platformy jest opatentowany silnik VizQL, który tłumaczy intuicyjne operacje użytkownika typu "przeciągnij i upuść" (drag-and-drop) bezpośrednio na zoptymalizowane zapytania do bazy danych. Pozwala to na płynną i błyskawiczną eksplorację. Platforma ta jest znana ze swojej wyjątkowej zdolności do efektywnej obsługi bardzo dużych i złożonych zbiorów danych. Tableau pozwala na tworzenie wysoce interaktywnych, niezwykle estetycznych i w pełni elastycznych pulpitów menedżerskich. Jest często preferowane w środowiskach, gdzie kładzie się nacisk na "storytelling" oparty na danych oraz tam, gdzie wymagana jest najwyższa jakość graficzna i elastyczność wizualizacji.

Do wizualizacji wyników w niniejszej pracy wybrano środowisko Tableau. Decyzja ta wynika bezpośrednio z architektury przyjętego rozwiązania, w którym ciężar pobierania, czyszczenia i strukturyzacji danych został przeniesiony na wcześniejsze etapy realizowane w języku Python. Dane wyjściowe z modelu analitycznego posiadają już uporządkowaną, tabelaryczną strukturę, gotową do bezpośredniej interpretacji. W takim scenariuszu rozbudowane funkcje ETL i integratory oferowane przez Power BI, służące do obsługi niedoskonałych źródeł danych, stają się zbędnym narzutem technologicznym. Tableau, ze swoim naciskiem na estetykę i szybkość eksploracji wizualnej gotowych zbiorów, stanowi w tym kontekście narzędzie bardziej adekwatne i efektywne.

3. Projekt narzędzia

3.1 Uwagi wstępne

Niniejszy rozdział przedstawia koncept techniczny i metodologiczny projektowanego narzędzia, którego zadaniem jest rozwiązanie problemu biznesowego zdefiniowanego szczegółowo w rozdziale 1.1. W odpowiedzi na zidentyfikowaną lukę analityczną, polegającą na braku zautomatyzowanych rozwiązań zdolnych do poprawnej interpretacji specyficznego języka graczy gatunku soulslike, celem pracy jest stworzenie narzędzia, które pozwoli na identyfikację kluczowych czynników wpływających na odbiór gier studia FromSoftware. Projekt narzędzia oparto na metodyce CRISP-DM, która strukturyzuje proces od pozyskania danych po wizualizację wniosków.

W kontekście niniejszego narzędzia, pojęcie „kluczowych czynników sukcesu” jest rozumiane jako najczęściej przejawiające się w opiniach pozytywne aspekty wyekstrahowane przez narzędzie z opinii graczy wystawianych na platformie Steam. Nie są one tutaj rozumiane jako ogólne wskaźniki finansowe, lecz jako konkretne aspekty gry (np. mechanika, fabuły, warstwy audio), które wykazują silną, mierzalną korelację z pozytywną oceną użytkownika w dalszej części pracy nazywanymi czynnikami. Z technicznego punktu widzenia, siła oddziaływania danego czynnika na sukces gry jest definiowana jako siła oddziaływania czynnika. Jest ona obliczana jako suma iloczynów wag sentymentu (wyznaczonych przez model regresji) oraz częstości występowania tokenów, które zostały uprzednio przypisane do kategorii tematycznych w procesie tworzenia słownika (np. kategoria grafika składa się ze słów „grafika”, „krajobraz”, „wygląd”. Zależność tę opisuje poniższy wzór:

$$S_c = \frac{\sum_{i=1}^n (w_i * x_i)}{\sum_{i=1}^n x_i} \quad (1)$$

Gdzie:

S_c – siła oddziaływania czynnika

n – liczba unikalnych tokenów przypisanych do czynnika c

w_i – waga wydźwięku tokenu składającego się na czynnik c

x_i – ilość wystąpień tokenu składającego się na czynnik c

Powyższa definicja pozwala na przekształcenie subiektywnych opinii w wymierne dane liczbowe, rozwiązując tym samym problem niejednoznaczności w odbiorze gier o niszowym charakterze dając menedżerom miarę sukcesu poszczególnych elementów gry. Wzór przedstawia siłę oddziaływania czynnika jako średnią ważoną ocen czynników. W oparciu o tak skonstruowane założenie przyjęto zasadę, że każda kategoria tematyczna, dla której siła oddziaływania czynnika przyjmuje wartość większą od zera, jest klasyfikowana jako czynnik sukcesu. Wartość ta oznacza bowiem przewagę pozytywnego wydźwięku nad negatywnym w ogólnym bilansie opinii graczy.

Aby wykorzystać powyższy model i uzyskać wartości zmiennych niezbędnych do obliczeń, czyli wagę wydźwięku oraz liczebność wystąpień, konieczne jest zaprojektowanie i wdrożenie dedykowanego narzędzia analitycznego. Jego architektura będzie miała spełniać powyższe kroki:

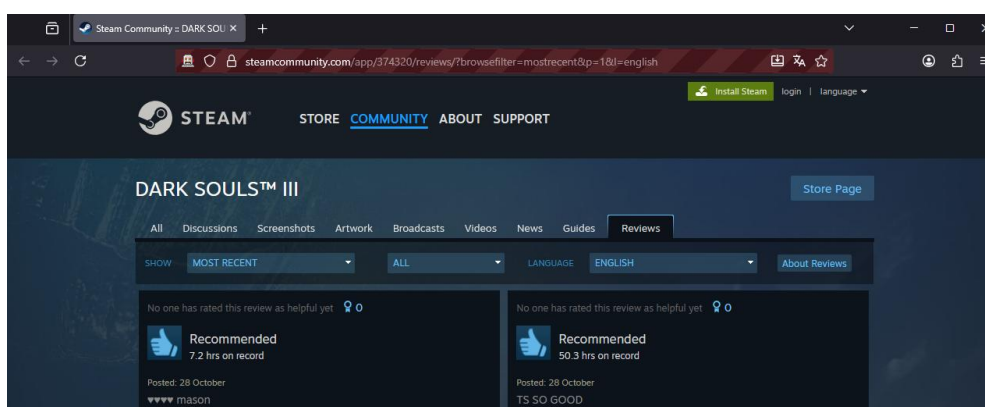
- akwizycja danych
- przygotowanie danych
- trenowanie i walidacja modelu klasyfikacyjnego

Kluczowym jest wykorzystanie wewnętrznych parametrów wytrenowanego modelu wag sentymentu. Oznacza to, że wiedza zaszyta w strukturze modelu posłuży do ewaluacji znaczenia słów. Stąd też potrzeba wykorzystania modelu o otwartej strukturze, jak regresja logistyczna. Z kolei drugi składnik równania, czyli ilość wystąpień, zostanie wyliczony bezpośrednio z uprzednio oczyszczonego i ustrukturyzowanego zbioru danych tekstowych.

Zwieńczeniem całego procesu analitycznego jest transformacja wygenerowanych danych numerycznych w wiedzę użytkową. Zagregowane efekty działania narzędzia, w tym obliczone wskaźniki sentymentu dla poszczególnych czynników, zostaną zaprezentowane w formie interaktywnych pulpitów menedżerskich. Wizualizacje te mają na celu syntezę złożonych wyników modelu do czytelnej postaci graficznej, stanowiąc tym samym bezpośrednie i praktyczne wsparcie dla osób decyzyjnych w studiu FromSoftware. Dzięki temu menedżerowie zyskają narzędzie umożliwiające szybką diagnozę odbioru produkcji oraz podejmowanie strategicznych decyzji projektowych w oparciu o obiektywne dane rynkowe.

3.2 Akwizycja i zrozumienie danych

Jest to początkowy etap technicznej części metodyki CRISP-DM, który odpowiedzialny jest za programistyczne pozyskiwanie surowych danych wejściowych. Komponenty tej warstwy będą łączyć się z zewnętrznym źródłem danych jakim jest Steam, w celu pobrania recenzji tekstowych predefiniowanej listy gier (zarówno tytułów FromSoftware, jak i gier konkurencyjnych). Dane wyjściowe tej warstwy będą miały postać nieustrukturyzowaną, zawierającą treść recenzji oraz kluczowe zmienne jak ocena (rekomendowane/nierekomendowane) i data. Fundamentem części technologicznej będzie web scraping oparty o język Python wraz z jego bibliotekami Pandas i Selenium. Zbudowany skrypt będzie automatyzował ręczny proces zbierania opinii, czyli włączanie strony, kopiowanie części tekstowej recenzji, nazwę użytkownika, ocenę i datę, a następnie przewinie stronę w celu pobierania kolejnych danych. Taki system będzie funkcjonował dzięki elementowi biblioteki Selenium, czyli WebDriver. WebDriver pozwala automatyzować ruch skryptu po przeglądarce. Jego działanie jest widoczne dzięki charakterystycznemu czerwonemu paskowi adresu, jak na Rysunku 9, który informuje użytkownika skryptu, że jego przeglądarka została przejęta przez działanie kodu.



Rysunek 9 Strona internetowa przejęta przez WebDriver

Źródło: opracowanie własne w oparciu o stronę:

<https://steamcommunity.com/app/374320/reviews/?l=english&browsefilter=toprated>

Wybór biblioteki Selenium jest nieprzypadkowy, ponieważ kiedy komunikacja z API może zakończyć się odrzuceniem zapytania ze strony skryptu, tak Selenium nie komunikuje się z wewnętrznym interfejsem strony, lecz odtwarza ruchy jakie zrobiłby prawdziwy

użytkownik ręcznie pobierający dane. Efektem jest niezawodność procesu akwizycji, lecz jest wolniejszy od bezpośredniego kontaktu z API, ponieważ pobranie 10000 opinii trwa około 45 minut do półtorej godziny w zależności od budowy strony, łącza internetowego użytkownika, czy wersji i rodzaju przeglądarki.

Tak pobrane dane zostają zapisane do formy pliku o rozszerzeniu .csv i są możliwe do otworzenia za pomocą środowiska tekstowego w formie nieustrukturyzowanej lub za pomocą oprogramowania analitycznego, takiego jak MS Excel czy Altair AI Studio w formie ustrukturyzowanej. Możliwe też jest wykorzystanie biblioteki pandas w pythonie i importowanie danych wykorzystując funkcję read_csv() Na Rysunku 10 pokazano zaimportowaną próbkę danych do środowiska MS Excel.

username	review_text	date_posted	recommendation
Mr.beam	Posted: 13 October amazing game everything about it was perfect	Posted: 13 October	Recommended
Backline Nautilus	Posted: October 13 My favourite game of all time, it's just simply I	Posted: October 13	Recommended
Drink the gas so your car can't	Posted: October 12 i like the game it looks beautiful the boss fight	Posted: October 12	Not Recommended
kiddac24	Posted: October 11 Too many bugs, game crashes an average of 1	Posted: October 11	Not Recommended
Hirohito ㊦	Posted: October 11 Why is this game \$80. Its been out for nearly	Posted: October 11	Not Recommended
AufAbwegen	Posted: October 11 Dark Souls III is my favorite FromSoft game. B	Posted: October 11	Recommended

Rysunek 10 Surowe pobrane dane ze Steam

Źródło: opracowanie własne

Wyekstrahowane recenzje składają się z 4 kolumn i 10 tysięcy recenzji dla każdej z analizowanych gier dostępnych na platformie Steam - Dark Souls, Dark Souls II, Dark Souls III, Sekiro i Elden Ring. Kolumny zbudowane są według poniższej logiki.

- **Username** – nazwa użytkownika składającego recenzję. Użytkownik nie może wystawić więcej niż jednej opinii co niweluje wielokrotne wystawianie opinii w konsekwencji zmieniając recenzje w spam. Potencjalne duplikaty mogłyby świadczyć o niepoprawnie działającym web scraperze
- **Review_text** – tekst opinii, który jest głównym obiektem zainteresowania w analizie. Pobrany z pomocą skryptu jest widocznie zanieczyszczony przez rozpoczynanie się zawsze od daty zamieszczenia opinii. Usunięcie tej wady będzie pierwszym krokiem analizy danych.
- **Date_posted** – czas wstawienia opinii. W przypadku wystawienia opinii w roku bieżącym wyświetlana w formacie DD/MM albo MM/DD, lecz w przypadku wystawienia opinii w zeszłych latach forma zmienia się na MM/DD/YYYY.
- **Recommendation** – zmienna określająca pozytywny lub negatywny charakter opinii. Wstępnie jest zmienną tekstową, lecz zostanie zmieniona na binarną w kolejnych etapach projektu.

Tak pobrane i zdefiniowane dane są następnie przekazywane do etapu przygotowanie danych.

3.3 Przygotowanie danych

Po pozyskaniu surowych danych kolejnym krokiem jest ich przygotowanie i przetworzenie. Etap ten realizowany jest poprzez wieloetapową eksploracyjną analizę danych (EDA) oraz procedury czyszczenia i transformacji (preprocessing). Analiza pozyskanych danych ma dostarczyć wiedzy na temat wzorców zawartych w nich zawartych, struktury, niedoskonałości oraz przygotowania danych pod właściwe modelowanie. W ramach niniejszego projektu, proces ten dzieli się na następujące etapy:

- Identyfikacja niedoskonałości i transformacja kluczowych zmiennych
- Analiza struktury danych i rozkładu zmiennych
- Wstępna wizualizacja danych w celu identyfikacji trendów czasowych

Operacje te zostaną przeprowadzone z pomocą języka Python w środowisku Project Jupyter. Wykorzystanie tego środowiska podyktowane jest jego elastycznością w procesie transformacji danych. Interaktywny charakter Jupytera pozwala na iteracyjne czyszczenie danych dzięki jego rozkładowi skryptu na poszczególne komórki. W pierwszej kolejności dane są czyszczone z oczywistych niedoskonałości. W przeciwieństwie do modeli analitycznych, na tym etapie nie usuwa się danych, lecz poddaje się je transformacji. Wstępny import danych do środowiska Python w formie ramki danych ukazano na Rysunku 11.

	username	review_text	date_posted	recommendation	game
0	Mr.beam	Posted: 13 October\namazing game	Posted: 13 October	Recommended	Dark Souls 3
1	Backline Nautilus	Posted: October 13\nMy favourite	Posted: October 13	Recommended	Dark Souls 3
2	Drink the gas so your car can't	Posted: October 12\ni like the g	Posted: October 12	Not Recommended	Dark Souls 3
3	kiddac24	Posted: October 11\nToo many bug	Posted: October 11	Not Recommended	Dark Souls 3
4	Hirohito @	Posted: October 11\nWhy is this	Posted: October 11	Not Recommended	Dark Souls 3
5	AufAbwegen	Posted: October 11\nDark Souls I	Posted: October 11	Recommended	Dark Souls 3

Rysunek 11 Załadowana próbka danych do środowiska Python

Źródło: opracowanie własne

Kluczową rzeczą do wyczyszczenia jest oczywista niedoskonałość jaką jest nieprawidłowy tekst recenzji review-text, który rozpoczyna się od daty i warto też przy tym kroku usunąć wszelkie znaki tabulacji zniekształcające tekst. Date-posted natomiast zamiast być zmienną czasu to jest traktowana jako string rozpoczynający się od słów „Posted: „. Efekt oczyszczenia danych z tych podstawowych niedoskonałości prezentuje Rysunek 12.

	username	review_text	date_posted	recommendation	game
0	Mr.beam	amazing game everything ab	13 October	Recommended	Dark Souls 3
1	Backline Nautilus	My favourite game of all t	October 13	Recommended	Dark Souls 3
2	Drink the gas so your car can't	i like the game it looks b	October 12	Not Recommended	Dark Souls 3
3	kiddac24	Too many bugs, game crashe	October 11	Not Recommended	Dark Souls 3
4	Hirohito @	Why is this game \$80. Its	October 11	Not Recommended	Dark Souls 3
5	AufAbwegen	Dark Souls III is my favor	October 11	Recommended	Dark Souls 3

Rysunek 12 Oczyszczone dane z niedoskonałości

Źródło: opracowanie własne

W pierwszej kolejności kluczowe zmienne zostaną poddane transformacji, aby były gotowe do analizy. Zmienna czasu (data) zostanie rozbита na bardziej szczegółowe warstwy, a zmienna recommendation (ocena) zostanie przekodowana z formy tekstowej na binarną, gdzie „1” oznacza rekomendację, a „0” jej brak. Wynik tego etapu przygotowania danych w formie tabelarycznej widoczny jest na Rysunku 13.

username	review_text	recommendation	game	date
0 Mr.beam	amazing game everything a	True	Dark Souls 3	2025-10-13
1 Backline Nautilus	My favourite game of all	True	Dark Souls 3	2025-10-13
2 Drink the gas so your car can't	i like the game it looks	False	Dark Souls 3	2025-10-12
3 kiddac24	Too many bugs, game crash	False	Dark Souls 3	2025-10-11
4 Hirohito @	Why is this game \$80. Its	False	Dark Souls 3	2025-10-11
5 AufAbwegen	Dark Souls III is my favo	True	Dark Souls 3	2025-10-11

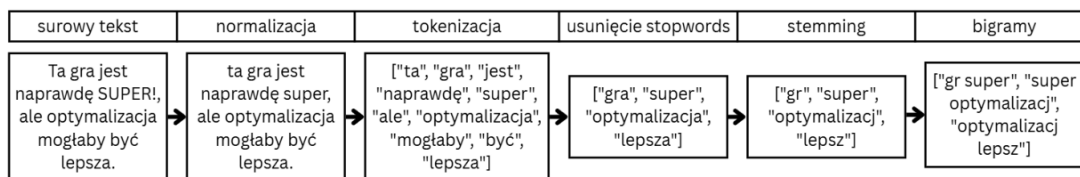
Rysunek 13 Wynik przygotowania danych

Źródło: opracowanie własne

Po wstępnym przygotowaniu ogólnych charakterystyk danych, projekt przejdzie do fazy przetwarzania tekstu, które przygotuje dane do użytku w trakcie procesu modelowania. W tym kroku surowe recenzje zostaną poddane kluczowym operacjom NLP, takim jak:

- Normalizacja znaków
- Usunięcie Stopwords
- Lematyzacja
- Utworzenie n-gramów

Celem jest uproszczenie tekstu recenzji, którego wynikiem będzie esencja sensu zawartego w opinii przez eliminację słów, które nie wnoszą żadnego znaczenia i są tylko wykorzystywane w składni języka (stopwords) i usunięcia końcówek leksykalnych. Przykładowy schemat tego procesu przygotowania danych tekstowych zobrazowano na Rysunku 14.



Rysunek 14 Przykładowy proces przygotowania danych tekstowych

Źródło: opracowanie własne

Tak przeprowadzony proces w pełni przygotowuje dane do głównej części analitycznej. Takie rozwiązanie nie wymaga zapisywania danych w pliku zewnętrznym, tylko przekazywane są lokalnie. Za przykład może posłużyć jeden wiersz z próbki danych, którego treść przed przetworzeniem przedstawia Rysunek 15.

My favourite game of all time, it's just simply beautiful, the perfect blend of boss design, aesthetics, and world design that manage to create the perfect feeling for me. The game has flaws, of course, but most of them contribute to the charm that it possesses. It's not just about progressing through the game or having fun, although there is much of that to be had, this game is a unique experience, that I think everyone should try to invest themselves into and see at least once in their lifetime. Peak.

Rysunek 15 Treść przykładowej opinii

Źródło: <https://steamcommunity.com/id/bihqpp/recommended/374320/> [dostęp 11.11.2025]

W procesie normalizacji powyższy tekst zostaje w pierwszej kolejności sprowadzony do formy, która ignoruje wielkości znaków, liczby czy znaki specjalne, takie jak przecinki. Celem jest transformacja tekstu w ciąg słów, który jest następnie dzielony na tokeny, czyli pojedyncze jednostki informacji tekstowych, dopiero teraz interpretowanych przez skrypt, jako oddzielne słowa tak jak ukazane na poniższym Rysunku 16.

```
["my", "favourite", "game", "of", "all", "time", "its", "just", "simply", "beautiful", "the", "perfect", "blend",
"of", "boss", "design", "aesthetics", "and", "world", "design", "that", "manage", "to", "create", "the",
"perfect", "feeling", "for", "me", "the", "game", "has", "flaws", "of", "course", "but", "most", "of", "them",
"contribute", "to", "the", "charm", "that", "it", "possesses", "its", "not", "just", "about", "progressing",
"through", "the", "game", "or", "having", "fun", "although", "there", "is", "much", "of", "that", "to", "be",
"had", "this", "game", "is", "a", "unique", "experience", "that", "i", "think", "everyone", "should", "try", "to",
"invest", "themselves", "into", "and", "see", "at", "least", "once", "in", "their", "lifetime", "peak"]
```

Rysunek 16 Efekt normalizacji i tokenizacji
Źródło: opracowanie własne

Tak przygotowany tekst jest wciąż niegotowy do dalszej analizy, ponieważ zawiera stopwords, które zanieczyszczałyby dane i powodowały znaczące odchylenia w analizie sentymentu czy przy tworzeniu list słów. Wynika to z natury stopwords, czyli ich braku znaczenia w kontekście sensu zdania. Są one nieodłączną częścią leksyki języka, jednak są używane do upłynnienia mowy i są mediatorem pomiędzy częściami zdania niosącymi za sobą sens. Przykładem może być zdanie „Gra jest super”, gdzie słowo „jest” uznawane jest za takie, które jest tylko łącznikiem między słowami „Gra” i „super”. Kluczem do wykonania tego kroku jest jasne określenie, jakie wypełniacze są potrzebne do analizy, a jakie są zbędne. Przy podstawowej analizie sprawdzającej tylko częstość występowania słów stosuje się gotowe już słowniki, które usuwają około 40% do 50% tekstu. Wyjątkiem jest jednak analiza sentymentu, gdzie dużym czynnikiem wpływającym na trafność modeli jest negacja. W przypadku usunięcia wszystkich stopwords ze słów „not bad” pozostałoby tylko „bad” co wypacza znaczenie opinii i wprowadza model w błąd. Finalny zbiór po usunięciu stopwords zaprezentowano na Rysunku 17.

```
[ 'favourite', 'game', 'time', 'simply', 'beautiful', 'perfect', 'blend', 'boss', 'design', 'aesthetics', 'world',
'design', 'manage', 'create', 'perfect', 'feeling', 'game', 'flaws', 'course', 'contribute', 'charm', 'possesses',
'progressing', 'game', 'fun', 'although', 'much', 'game', 'unique', 'experience', 'think', 'everyone', 'try',
'invest', 'see', 'least', 'lifetime', 'peak' ]
```

Rysunek 17 Zbiór tokenów po usunięciu stopwords
Źródło: opracowanie własne

Przykładowa próbka po usunięciu stopwords zredukowała swój rozmiar o około 60%, co jest wynikiem większym, niż w przykładach literaturowych, ale jest to zrozumiałe przez nieformalny i często chaotyczny sposób pisania recenzji w przestrzeni internetowej. Tak przygotowane dane możemy wykorzystać do następnych kroków analizy. Pierwszym z nich jest wykonanie prostej listy słów, które występują w recenzji. Przykładowa lista słów została zaprezentowana na przykładzie Tabeli 1.

Tabela 1 Przykładowa lista 5 najczęściej występujących słów
Źródło: opracowanie własne

token	count
game	4
perfect	2
design	2
favourite	1
time	1

Tak przygotowana lista jest dobrym narzędziem do wstępnej analizy, co najważniejsze było w ogóle analizowanych recenzji. Poza oczywistością, jaką jest słowo „game” w przypadku gry, warto zauważyć, że do góry tabeli będą wpływać z narastającą ilością opinii najważniejsze aspekty gry, które dla graczy są najważniejsze. W przypadku pojedynczych recenzji, jak ta omówiona wcześniej i przedstawiona listą słów w Tabeli 1 powyżej, możemy co najwyżej przyjmować założenia, że im częściej dane słowo występowało, tym ważniejszy był to aspekt dla opiniodawcy. Warto również wspomnieć w kontekście do listy słów, że częstym problemem takich list jest nieujednolicanie słów, ze względu na morfologię ich pochodzenia. W tym miejscu przydaje się stemming czy lematyzacja, czyli usuwanie końcówek leksykalnych słów. Przykładem pary słów, gdzie stemming jest niezastąpiony jest „przeciwnik” i „przeciwnicy”. Oba słowa dotyczą oponentów, z którymi przyjdzie nam się mierzyć w grze, ale są rozdzieleni na liczbę pojedynczą i mnogą, czego bez odpowiedniej instrukcji komputer nie rozumie, dlatego traktuje je jako dwa osobne słowa. Zastosowanie stemmingu pozwala na ich unifikację do wspólnego rdzenia, co przedstawia Tabela 2.

Tabela 2 Przykładowa lista słów z zastosowanym stemmingiem

Źródło: opracowanie własne

token	count
gam	4
perfect	2
design	2
favourit	1
time	1
simpli	1
beauti	1
blend	1
boss	1
aesthet	1

Tak zmodyfikowana lista słów zamiast „beautiful” zawiera skróconą wersję „beauti”, na które składają się słowa, jak „beautiful” czy „beauty” agregując je do wspólnej listy. Rozwinięciem konceptu listy słów jest też generowanie n-gramów, proces bardzo czasochłonny i zasobochłonny, lecz dostarczający cennych informacji na temat opinii. Rozwija on analizę o zlepki wyrazowe, które zależne są od ustalonej liczby n, czyli ilości słów przypisywanych do n-gramu. Przykładowe n-gramy dla n=2 (bigramy) wraz z ich liczebnością zestawiono w Tabeli 3.

Tabela 3 Przykładowe n-gramy dla n = 2

Źródło: opracowanie własne

bi-gram	count
('boss', 'design')	1
('world', 'design')	1
('favourite', 'game')	1
('perfect', 'blend')	1
('unique', 'experience')	1

Tak pogłębiona analiza daje nam więcej kontekstu do każdego użytego słowa. Przy małej próbce badawczej przez różnorodność języka i sposobów pisanie, zauważamy znikome ilości powtarzających się n-gramów, ale proporcjonalnie z ilością opinii zwiększamy szansę na powtórzenie się ich, co zdecydowanie zwiększa sens identyfikowanych danych. Dzięki tej metodzie dostajemy bezpośrednią informację, jaki „design” jest dla gracza najważniejszy, czy jaki aspekt gry uznaje za „unique”.

Najważniejszym elementem analizy tekstu opinii to zdecydowanie analiza sentymentu, która na podstawie utworzonego modelu przez uczenie maszynowe będzie wykrywała słowa, które mają pozytywny lub negatywny wpływ na opinie.

Tak zagregowane słowa ze względu na sentyment są cenną informacją, która bezpośrednio przekłada się na odpowiedź na problem biznesowy. W przypadku danych tak specyficznych, jak opinie graczy wymagane będzie utworzenie dedykowanego modelu, który będzie automatycznie oceniał słowa kluczowe, takie jak „projekt świata” czy „poziom trudności” na podstawie zostawionych przez recenzentów opinii. Ideę przykładowej analizy sentymentu na danych tekstowych przedstawia Rysunek 18.

Consumer comment	Meaningful words	Sentiment weight	Sentiment score
"Employees are always so polite and helpful . They have almost everything larger stores have, which is good enough for me! They even have white sweet potatoes which you usually only find at specialty grocers. Great experience!"	Always Polite Helpful Good Great	1 2 2 1 2	8
" Nice store. Seems clean and well organized . It is a bit on the smaller size as far as other stores go, so prepare for it to perhaps feel a little crowded , even if there aren't that many people in the store. This also means that their stock of certain items may be a bit smaller than the larger locations, but that's to be expected."	Nice Clean Well Organized Smaller Crowded Larger	2 2 1 2 -1 (x2) -1 1	5
"Since a new manager took over this once best supermarket around has fallen way off. They barely have anything, produce is always expired , and the store is not kept up very well."	New Best Fallen Barely Expired	1 2 -1 -1 -2	-1

Rysunek 18: Przykładowa analiza sentymentu na przykładowych danych

Źródło: <https://www.bellomy.com/blog/three-text-sentiment-analysis-methods-and-why-ours-better>

Na etap analizy wydźwięku poświęcony jest następny etap, czyli tworzenie modeli analizujących wydźwięk opinii.

3.4 Ekstrakcja czynników sukcesu

Wartym podkreślenia jest fakt, że implementacja algorytmu klasyfikacyjnego w niniejszym projekcie nie jest spełnieniem celu samego w sobie. Choć dążymy do uzyskania wysokich wskaźników jakości modelu, sam mechanizm zdolny do poprawnego odróżniania opinii pozytywnych od negatywnych nie stanowi finalnego produktu projektu, lecz pełni kluczową funkcję narzędzia walidacyjnego. Fundamentalnym założeniem przyjętym w pracy jest teza, że jeżeli model skutecznie „rozumie” specyfikę recenzji, to parametry decyzyjne, na których oparł swoje werdykty (wagi przypisane poszczególnym cechom), stanowią wierne odzwierciedlenie rzeczywistych preferencji graczy.

W konsekwencji, sprawnie działający klasyfikator jest w istocie jedynie produktem pochodnym, służącym do wyekstrahowania zidentyfikowanych czynników. Aby jednak uznać te czynniki za odpowiedź na postawiony problem badawczy, proces ekstrakcji musi zostać poddany weryfikacji z pomocą miar efektywności działania modelu. Głównymi miarami poprawności interpretacji języka graczy przez model będą wskaźniki skuteczności klasyfikacji - Accuracy, Kappa Cohena oraz AUC. Biorąc pod uwagę silne niezbalansowanie zbioru danych, sam wysoki poziom dokładności Accuracy, przewidywany na wysokim poziomie, mógłby być mylący. Dlatego kluczowym kryterium walidacyjnym, potwierdzającym, że model nie faworyzuje jedynie klasy większościowej, będzie osiągnięcie satysfakcjonującej wartości statystyki Kappa Cohena na poziomie powyżej 0.5 oraz wskaźnika AUC na poziomie minimum 0.85. Akceptacja umiarkowanego poziomu zgodności Kappy podyktowana jest specyfiką danych tekstowych pobranych z nieustrukturyzowanych źródeł, które ze względu na wysokie nasycenie ironią i żargonem środowiskowym są trudne w interpretacji. W świetle literatury NLP dotyczącej analizy sentymentu w niejednoznacznych kontekstach, wartości pomiędzy Kappy 0.4 – 0.5 uznaje się za wystarczające, gdyż odzwierciedlają one złożoność semantyczną opinii, a nie błąd statystyczny modelu (Lefever, et al., 2018). Ostatecznie uprzednio przedstawione minimalne wyniki stanowić będą matematyczny dowód na to, że zidentyfikowane przez algorytm słowa kluczowe i tematy nie są dziełem przypadku, lecz stanowią rzeczywistą, biznesową wartość ukrytą w opiniach użytkowników.

Etap modelowania zostanie zrealizowany w środowisku języka Python z wykorzystaniem biblioteki scikit-learn. Ta technologia zapewnia niezbędną optymalizację wydajności oraz precyzyjną kontrolę nad procesem inżynierii cech.

W celu rzetelnej weryfikacji zdolności uogólniania modelu, projekt zakłada podział zbioru danych na część uczącą oraz testową w proporcji 70:30. Zagwarantuje to, że proporcja klas pozytywnych do negatywnych w obu podzbiorach będzie identyczna, jak w oryginalnym zbiorze danych, co jest niezbędne dla zachowania statystycznej reprezentatywności próbki. Ze względu na spodziewaną nierównowagę klas, planuje się wdrożenie strategii równoważenia zbioru, która zostanie zaaplikowana wyłącznie do danych treningowych. Zastosowana zostanie technika downsamplingu (redukcji) klasy większościowej, ustalająca stosunek recenzji pozytywnych do negatywnych na poziomie 2:1. Jest to bardziej naturalne podejście od upsamplingu, które nie sprawdza się w przypadku danych tekstowych. Taka operacja pozwoli na redukcję szumu informacyjnego przy jednoczesnym zachowaniu wystarczającej liczby przykładów uczących, nie naruszając przy tym naturalnego rozkładu danych w zbiorze testowym, na którym przeprowadzana będzie ewaluacja.

Kolejnym planowanym krokiem jest wektoryzacja tekstu, czyli transformacja recenzji w macierz numeryczną przy użyciu metody TF-IDF. W konfiguracji wektoryzatora uwzględnione zostaną n-gramy rzędu (1, 2), co pozwoli na analizę zarówno pojedynczych słów, jak i par wyrazów (bigramów), umożliwiając uchwycenie szerszego kontekstu wypowiedzi.

Jako algorytm klasyfikacyjny zostanie wykorzystana regresja logistyczna. Decyzja o wyborze modelu liniowego wynika z jego wysokiej interpretowalności. W przeciwieństwie do modeli o złożonej strukturze, regresja logistyczna pozwoli na bezpośrednią analizę wagi wydźwięku tokenów. Umożliwi to nie tylko przewidywanie wydźwięku, ale także identyfikację konkretnych słów i fraz najsilniej korelujących z pozytywną lub negatywną oceną, co stanowi bezpośrednią realizację celu biznesowego pracy.

Końcowa ewaluacja modelu zostanie przeprowadzona na odseparowanym zbiorze testowym przy użyciu metryk dokładności klasyfikacji oraz Kappy Cohena. Dodatkowo wygenerowany zostanie raport klasyfikacyjny, który pozwoli ocenić precyzję i recall klasy mniejszościowej (negatywnej) modelu dla każdej z klas decyzyjnych osobno.

Model spełniający założone warunki utworzy wewnętrznie listę słów, które posiadają zróżnicowaną wagę oraz charakter wydźwięku. Tak wygenerowany zestaw mógłby stanowić samodzielną podstawę do analiz, jednak w celu pogłębienia wiedzy ukrytej w danych, zdecydowano się na agregację wyników w ramach predefiniowanych obszarów tematycznych. Podstawową metodyką NLP do ekstrakcji obszarów tematycznych jest automatyczna ekstrakcja tematów, przy użyciu takich metod jak LDA czy BERTopic. Problemem w tym przypadku są opinie pobierane z platformy Steam, które charakteryzują się wysokim stopniem zaszumienia, chaotyczną strukturą gramatyczną, stosowaniem skrótów myślowych oraz specyficznego żargonu graczy. W takich warunkach algorytmy nienadzorowane mają trudność z wyodrębnieniem spójnych i interpretowalnych klastrów tematycznych. Stąd w miejsce zawodnej automatyzacji, opracowana została autorska lista odgórnie ustalonych kategorii tematycznych, nazywanych czynnikami. Tokeny składające się na poszczególne kategorie będą dobierane i weryfikowane manualnie przez autora w trakcie optymalizacji procesu, co pozwoli na precyzyjne uchwycenie kontekstu specyficznego dla gier typu soulslike. Klucz dobierania tokenów do danych grup przedstawiono w Tabeli 4.

Tabela 4 Powiązania czynników ze składnikami
Źródło: opracowanie własne

Czynnik	Powiązany rodzaj tokenów
Grafika	warstwa wizualna, styl artystyczny, projekt świata
Udźwiękowanie	warstwa audio, muzyka, efekty dźwiękowe, głosy postaci
Fabula	narracja, historia, głębia świata, postacie niezależne
Rozgrywka	mechanika gry, system walki, poziom trudności, satysfakcja z gry
Sterowanie	interakcje gracza z systemami, responsywność postaci, peryferia
Optymalizacja	techniczna warstwa gry, stabilność aplikacji
Wsparcie	błędy, wsparcie deweloperów
Cena	koszt zakupu, stosunek ceny do jakości
Tryb Wieloosobowy	infrastruktura sieciowa, interakcje online, mechaniki kooperacji

Na podstawie klucza utworzono wstępny słownik widoczny w Tabeli 5., który aktualizowany będzie z przebiegiem pracy.

Tabela 5: Przykładowe składowe czynników
Źródło: opracowanie własne

Czynnik	Składowe
Cena	price,money,cost
Udźwiękowanie	sound,music,soundtrack
Fabula	story,lore,narrative
Wsparcie	support,bug,fix
Sterowanie	control,keyboard,mouse
Rozgrywka	action,challenge,combat
Grafika	enviroment,art,map
Tryb Wieloosobowy	multiplayer,online,server
Optymalizacja	optimize,performance,freeze

Wynikiem działania narzędzia będą dwa pliki zawierającymi dane na temat opinii wybranych produkcji studia FromSoftware. Pierwszy, nazwany `clean_df.xlsx`, zawierać będzie zbiór już wyczyszczonych ze stopwords opinii, rodzaj rekomendacji, datę wstawienia i grę, której dotyczy. Ten jest zbiorem udostępniającym ogólne informacje o zgromadzonych danych i jest podstawowym źródłem informacji na temat rozkładu wydźwięku opinii o grach Fromsoftware. Przykładowy wiersz struktury pliku `clean_df.xlsx` prezentuje Tabela 6.

Tabela 6 Przykład wiersza pliku `clean_df.xlsx`
Źródło: opracowanie własne

cleaned_review	recommendation	date	game_name
one best game time	TRUE	24.11.2025	Dark Souls 1

Z kolei drugi plik wyjściowy, oznaczony jako `wynik.xlsx` stanowi esencję przeprowadzonego procesu modelowania i zawiera zagregowane wyniki analizy regresji logistycznej. Struktura tego zbioru została zaprojektowana w taki sposób, aby bezpośrednio odpowiadać na zdefiniowany cel pracy, jakim jest identyfikacja kluczowych czynników sukcesu gier studia FromSoftware. W pliku tym znajdują się wyselekcjonowane przez wektoryzator tokeny przyjmujące postać unigramów lub bigramów, wraz z przypisanymi im parametrami statystycznymi. Zmienna `Coefficient` reprezentuje wyuczoną przez algorytm wagę, determinującą siłę oraz kierunek wpływu danego słowa na sentyment (gdzie wartości dodatnie wskazują na atuty, a ujemne na wady produkcji), natomiast zmienna `Count` obrazuje skalę występowania danego zjawiska w całej kolumnie opinii. Przykładowe wiersze pliku `wynik.xlsx` wraz z obliczonymi współczynnikami ukazano w Tabeli 7.

Tabela 7 Przykładowe wiersze pliku `wynik.xlsx`
Źródło: opracowanie własne

token	coefficient	count	topics
60fps	-0,969	72	Performance
interesting story	0,185	25	Story
atmosphere	0,074	700	Graphics
boring	-3,844	491	Story

Zbiór ten jest zatem gotowym, ustrukturyzowanym produktem analitycznym, który syntetyzuje wiedzę wydobytą z surowych danych i jest przygotowany do bezpośredniej implementacji w warstwie prezentacji wyników w środowisku Tableau.

3.5 Projekt pulpitów

Zaprojektowana warstwa wizualna narzędzia analitycznego ma na celu transformację surowych wyników modelu predykcyjnego w ustrukturyzowaną wiedzę, umożliwiającą podejmowanie decyzji w czasie rzeczywistym. Proces projektowy został przeprowadzony w oparciu o architekturę hierarchiczną, realizowaną w środowisku Tableau Desktop, która prowadzi użytkownika od ogólnego obrazu kondycji produktu do szczegółowej analizy poszczególnych tokenów (drażenie danych). Konstrukcja interfejsów oparta została na metodyce analityki deskryptywnej i jest bezpośrednio podporządkowana konieczności udzielenia odpowiedzi na trzy kluczowe pytania badawcze:

- Jakie są kluczowe czynniki sukcesu wydawanego produktu?
- Jakie czynniki sukcesu najbardziej wpływają na sukces produktu?
- Jakie elementy składają się na pozytywny lub negatywny wydźwięk czynników?

W oparciu o powyższe pytania zdefiniowano nadrzędny cel, którym jest budowa pulpitów transformujących surowe wyniki modelu analitycznego w syntetyczną wiedzę, umożliwiającą podejmowanie decyzji w czasie rzeczywistym. Narzędzie ma za zadanie umożliwić błyskawiczną diagnozę priorytetów poprzez odróżnienie problemów niskowych od systemowych, co stanowi kluczową przesłankę dla optymalizacji alokacji zasobów przy rozwoju produktu.

Jako głównych odbiorców systemu zidentyfikowano kadrę zarządzającą oraz producentów wykonawczych. Sposób korzystania z narzędzia przewiduje dwa poziomy interakcji:

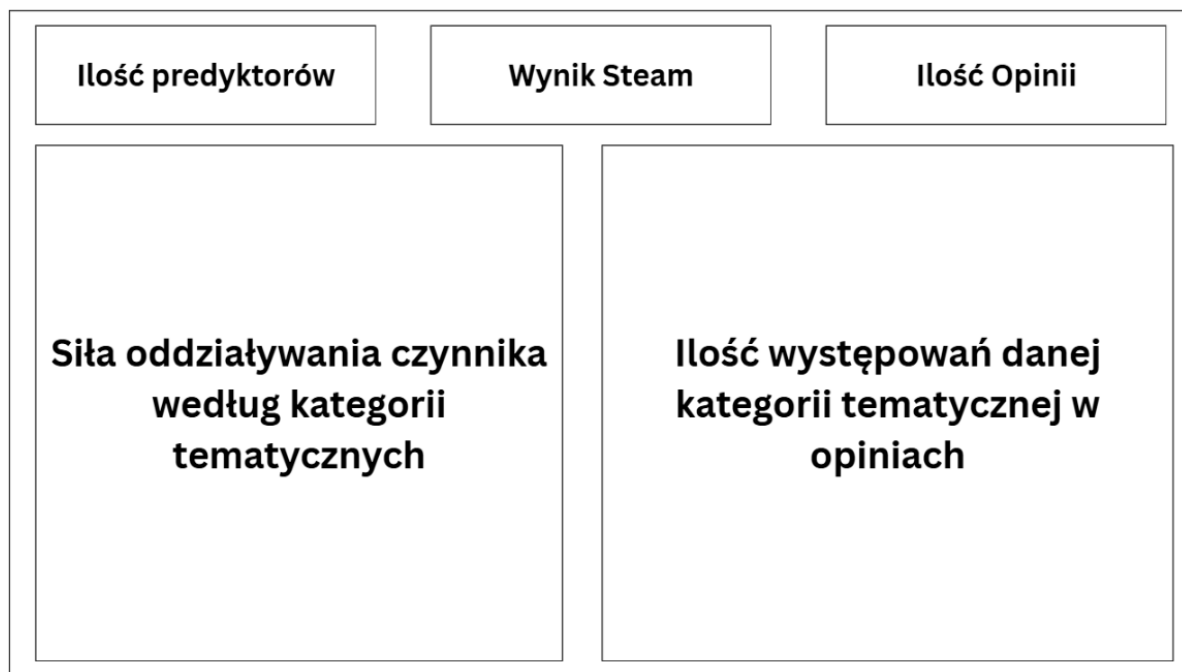
- strategiczny - służący do szybkiej oceny ogólnej kondycji produktu
- analityczny - wykorzystywany do pogłębionej weryfikacji przyczyn zaistniałych trendów w opiniach graczy.

Na etapie określania wskaźników wydajności (KPI) wybrano metryki bezpośrednio korespondujące ze zdefiniowanymi pytaniami badawczymi. Dla oceny czynników sukcesu zdefiniowano miarę siły oddziaływania czynnika oraz miarę częstotliwości występowania danej kategorii tematycznej w zbiorze recenzji. Kluczowym wskaźnikiem jakościowym pełniącym rolę ogólnego obrazowania sytuacji badanych danych jest „Wynik Steam” (procentowy udział sentymentu pozytywnego), wspierany przez wskaźniki ilościowe, takie jak całkowita „Ilość Opinii” oraz „Ilość predyktorów”.

Realizacja tych wskaźników wymagała sporządzenia listy niezbędnych źródeł danych. System opiera się na wynikach wyjściowych modelu analitycznego, który dostarcza zarówno dane ogólne z pliku `clean_df.xlsx`, jak i dane dotyczące tokenów z przydzieloną przez model wagą wydźwięku w pliku `wynik.xlsx`. Kluczowym wymogiem dla źródeł danych jest ich kompletność oraz spójność, polegająca na jednoznacznym przypisaniu tokenów do kategorii tematycznych oraz wartości sentymentu.

W procesie projektowania filtrów przyjęto, że wybór czynnika na poziomie ogólnym ma determinować zakres danych prezentowanych w widokach szczegółowych. Mechanizm ten jest ściśle powiązany z funkcją drażenia danych. Zastosowano architekturę hierarchiczną, umożliwiającą użytkownikowi płynne przejście od zagregowanych wykresów słupkowych do dekompozycji kategorii na czynniki pierwsze, w postaci szczegółowego widoku punktów, gdzie każdym punktem będzie konkretnym tokenem

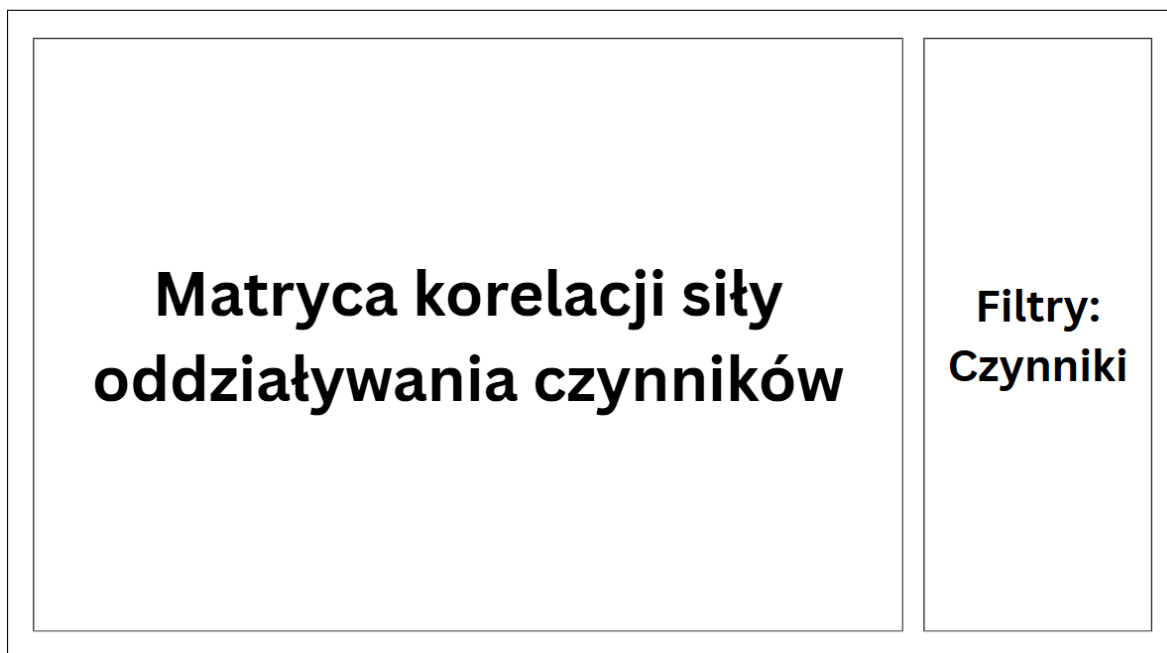
przyjmowanym do analizy na kanwie pulpitu scatter plot. Projekt układu pulpitu podstawowych informacji przedstawia Rysunek 19.



Rysunek 19 Pulpit podstawowych informacji
Źródło: opracowanie własne

Pierwszy pulpit będzie pełnił rolę centrum dowodzenia dla kadry zarządzającej i producentów wykonawczych. Jego zadaniem będzie udzielenie natychmiastowej odpowiedzi na dwa pierwsze pytania analityczne poprzez zestawienie jakości sentymentu z ilością generowanych opinii. W górnej części dashboarda zaprezentowane zostaną zagregowane metryki, takie jak ogólny wynik procentowy sentymentu, całkowita liczba przeanalizowanych opinii oraz liczba zidentyfikowanych unikalnych tokenów (predyktorów). Główny wykres odpowiadający na pytanie - „Jakie są kluczowe czynniki sukcesu?” znajdzie się w lewym dolnym rogu pulpitu. Będzie to wykres słupkowy, którego zadaniem będzie zaprezentowanie obliczonej wartości siły oddziaływania dla każdego czynnika. Słupki skierowane w górę wskażą obszary budujące sukces gry, natomiast słupki skierowane w dół zidentyfikują obszary krytyczne, obniżające ocenę końcową. Dołączony zostanie do niego z prawej strony komplementarny wykres słupkowy, odpowiadający na pytanie: „Jakie czynniki najbardziej wpływają na sukces?”. Zobrazuje on, jak często dany temat jest poruszany przez graczy. Zestawienie tych dwóch wykresów obok siebie pozwoli na błyskawiczną diagnozę priorytetów. Użytkownik końcowy będzie mógł odróżnić problemy niszowe od problemów systemowych, co jest kluczowe dla alokacji zasobów przy tworzeniu nowego produktu.

Drugi pulpit został zaprojektowany jako narzędzie pogłębionej analizy. Jego celem jest dekompozycja ogólnych kategorii na czynniki pierwsze, co pozwoli odpowiedzieć na trzecie pytanie analityczne - „Jakie elementy składają się na wydzźwięk czynników?”. Projekt matrycy korelacji siły oddziaływania czynników widoczny jest na Rysunku 20.



Rysunek 20 Matryca korelacji siły oddziaływania czynników
Źródło: opracowanie własne

Wymagania dla tego pulpitu zdefiniowano w oparciu o konieczność precyzyjnego rozróżnienia tematów głośnych, często poruszanych, ale o niskim wpływie decyzyjnym, od tematów kluczowych, które mogą pojawiać się rzadziej, ale determinują sentyment recenzji. Głównym interfejsem będzie wykres punktowy scatter plot, operujący na dwóch wymiarach. Na osi odciętych zostanie odwzorowana liczebność wystąpień danego tokenu. Oś rzędnych reprezentuje natomiast siłę oddziaływania czynnika, wyliczoną z wykorzystaniem wartości obliczonych przez model klasyfikacyjny. Warstwa wizualna zostanie wzbogacona o kodowanie kolorystyczne oparte na skali przechodzącej od czerwieni do zieleni, gdzie punkty zielone oznaczają tokeny o dodatnim wpływie na sukces gry, natomiast punkty czerwone wskazują na elementy obniżające ocenę. Interaktywność pulpitu zapewnią będzie filtr, umożliwiający odbiorcy końcowemu zawężenie widoku do konkretnego czynnika. Dzięki takiemu projektowi, matryca pozwoli na natychmiastową identyfikację anomalii oraz priorytetyzację zadań, wskazując rzadkie błędy krytyczne, które przy tradycyjnej analizie częstościowej mogłyby zostać pominięte.

Tak utworzone pulpity menedżerskie utworzą kompletny i spójny ekosystem analityczny. Zestaw ten będzie odpowiadał na wszelkie potrzeby zespołu menedżerów, prowadząc ich przez najogólniejsze informacje na temat czynników, po indywidualny udział czynników w opiniach graczy i części składowe tych, że czynników.

4. Implementacja

4.1 Budowa skryptu pobierającego dane z platformy Steam

Niniejszy rozdział przedstawia szczegółową analizę i implementację modułu odpowiedzialnego za automatyczną akwizycję danych (web scraping) recenzji użytkowników. Opracowane narzędzie funkcjonuje jako aplikacja konsolowa, umożliwiająca użytkownikowi interaktywny wybór badanych tytułów oraz precyzyjne określenie wielkości próby badawczej. Moduł zaimplementowano w języku Python z wykorzystaniem biblioteki Selenium, opierając działanie na sterowniku przeglądarki Google Chrome.

Fundamentem działania skryptu jest import niezbędnych bibliotek. W celu zapewnienia przenośności kodu pomiędzy różnymi środowiskami uruchomieniowymi wykorzystano menedżer pakietów ChromeDriverManager. Automatyzuje on proces pobierania i instalacji odpowiedniej wersji sterownika chromedriver, eliminując konieczność ręcznej konfiguracji ścieżek systemowych. Import niezbędnych bibliotek zaimplementowano w sposób przedstawiony na Rysunku 21.

```
import time
import pandas as pd
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service as ChromeService
from webdriver_manager.chrome import ChromeDriverManager
```

Rysunek 21 Import bibliotek

Źródło: opracowanie własne

W celu zapewnienia skalowalności rozwiązania zastosowano zewnętrzną strukturę konfiguracyjną. Słownik GAMES_CONFIG mapuje wybór użytkownika na ustrukturyzowane metadane - nazwę gry, jej unikalny identyfikator aplikacji (AppID) w serwisie Steam oraz nazwę docelowego pliku wyjściowego. Taka architektura pozwala na łatwe dodawanie kolejnych tytułów do analizy bez konieczności ingerencji w logikę pętli pobierającej. Strukturę słownika konfiguracyjnego mapującego gry ukazano na Rysunku 22.

```
GAMES_CONFIG = {
    "1": {"name": "Dark Souls 1 (Remastered)", "id": "570940",
"filename": "DS1.csv"},
    "2": {"name": "Dark Souls 2 (SotFS)", "id": "335300", "filename":
"DS2.csv"},
    "3": {"name": "Dark Souls 3", "id": "374320", "filename": "DS3.csv"},
    "4": {"name": "Sekiro", "id": "814380", "filename": "SEKIRO.csv"},
    "5": {"name": "Elden Ring", "id": "1245620", "filename": "ER.csv"}}
```

Rysunek 22 Mapowanie słownika identyfikowanych gier

Źródło: opracowanie własne

Właściwa ekstrakcja danych realizowana jest przez funkcję get_reviews(), której struktura została przedstawiona na Rysunku 23. Została ona zaprojektowana w sposób modułowy, przyjmując jako argumenty obiekt sterownika, liczbę recenzji do pobrania oraz nazwę gry w celach logowania. Na początku funkcji następuje inicjalizacja zmiennych

pomocniczych oraz pobranie początkowej wysokości dokumentu, co jest niezbędne do obsługi dynamicznego ładowania treści.

```
def get_reviews(driver, num_reviews, game_name):
    reviews_data = []
    reviews_scraped = 0
    last_height = driver.execute_script("return
document.body.scrollHeight")

    print(f"--- Rozpoczynam pobieranie {num_reviews} opinii dla:
{game_name} ---")
```

Rysunek 23 Zdefiniowanie funkcji get_reviews()

Źródło: opracowanie własne

Ze względu na zastosowanie na platformie Steam mechanizmu "nieskończonego przewijania" (infinite scroll), implementacja wymagała zastosowania pętli while. Wewnątrz niej skrypt ukazany na Rysunku 24 wykonuje kod JavaScript przewijający widok do dołu strony, co wymusza na serwerze przesłanie kolejnej partii danych. Zastosowano również opóźnienie czasowe (time.sleep), pozwalające na poprawne wyrenderowanie nowych danych na stronie.

```
while reviews_scraped < num_reviews:
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(2) # Czas na załadowanie nowych opinii

    reviews_on_page = driver.find_elements(By.CLASS_NAME, "apphub_Card")
```

Rysunek 24 Mechanizm przewijania strony i lokalizacja elementów

Źródło: opracowanie własne

Aby zoptymalizować proces i uniknąć duplikowania danych, skrypt operuje na bieżącej partii załadowanych elementów. Zmienna current_batch jest odpowiednio przycinana, jeśli liczba dostępnych recenzji przekracza zdefiniowany przez użytkownika limit num_reviews. Logikę limitowania przetwarzanych danych prezentuje Rysunek 25.

```
current_batch = reviews_on_page

if len(current_batch) > num_reviews:
    current_batch = current_batch[:num_reviews]
```

Rysunek 25 Logika limitowania przetwarzanych danych

Źródło: opracowanie własne

Właściwa ekstrakcja danych tekstowych odbywa się w pętli for o strukturze ukazanej na Rysunku 26. Algorytm iteruje przez elementy, zaczynając od indeksu ostatnio pobranej recenzji (reviews_scraped), co zapewnia przetwarzanie wyłącznie nowych wpisów. Kluczowym elementem jest blok try-except, który zapewnia odporność skryptu na błędy wynikające z braku poszczególnych atrybutów w kodzie HTML (np. brak treści recenzji), pozwalając na kontynuowanie pracy bez przerywania działania programu.

```

for i in range(reviews_scraped, len(current_batch)):
    if reviews_scraped >= num_reviews:
        break
    review = current_batch[i]
    try:
        username = review.find_element(By.CLASS_NAME,
"apphub_CardContentAuthorName").text
        review_text = review.find_element(By.CLASS_NAME,
"apphub_CardTextContent").text
        date_posted = review.find_element(By.CLASS_NAME,
"date_posted").text
        recommendation = review.find_element(By.CLASS_NAME,
"title").text
        reviews_data.append({
            "username": username,
            "review_text": review_text,
            "date_posted": date_posted,
            "recommendation": recommendation,
        })
        reviews_scraped += 1
    except Exception:
        continue

```

Rysunek 26 Pętla ekstrahująca dane z obsługą błędów
Źródło: opracowanie własne

Sterowanie całym procesem odbywa się w funkcji `main()`, która pełni rolę interfejsu użytkownika. Program prosi o podanie parametrów wejściowych i na podstawie wyboru użytkownika buduje listę `selected_games`. Obsłużono dwa tryby, pierwszy to wybór wszystkich gier (opcja "0") oraz drugi, który jest wyborem selektywnym inicjalizowanym poprzez podanie ciągu cyfr odpowiadających kluczom w konfiguracji. Funkcję główną programu oraz obsługę wyboru gier przez użytkownika obrazuje Rysunek 27.

```

def main():
    if selection == "0":
        print("Wybrano opcję: Opinie o wszystkich grach.")
        for key in GAMES_CONFIG:
            selected_games.append(GAMES_CONFIG[key])
    else:
        for char in selection:
            if char in GAMES_CONFIG:
                selected_games.append(GAMES_CONFIG[char])
        driver =
webdriver.Chrome(service=ChromeService(ChromeDriverManager().install()))

```

Rysunek 27 Funkcja główna i konfiguracja słownika
Źródło: opracowanie własne

W ostatniej fazie skrypt iteruje przez wybrane gry, dynamicznie generując adres URL na podstawie identyfikatora aplikacji. Zaimplementowano tu również automatyczną obsługę

tak zwanej "bramki wiekowej", która jest wymagana dla gier przeznaczonych dla dorosłych odbiorców. Po zakończeniu pobierania dane są konwertowane na ramkę danych pandas i zapisywane do pliku CSV. Blok finally gwarantuje poprawne zwolnienie zasobów przeglądarki. Pętlę wykonawczą i zapis danych przedstawia Rysunek 28.

```
try:
    for game in selected_games:
        url =
f"https://steamcommunity.com/app/{game['id']}/reviews/?browsefilter=mostrecent&p=1&l=english"
        driver.get(url)
        time.sleep(3)

    try:
        driver.find_element(By.ID, "view_product_page_btn").click()
    except Exception:
        pass

    reviews = get_reviews(driver, NUM_REVIEWS, game['name'])

    df = pd.DataFrame(reviews)
    df.to_csv(game['filename'], index=False)
    print(f" -> SUKCES: Zapisano do '{game['filename']}'")

finally:
    driver.quit()
```

Rysunek 28 Pętla wykonawcza i zapis danych
Źródło: opracowanie własne

Z pomocą scrapera pobrano pięć zestawów opinii o grach FromSoftware liczących po 10000 opinii.

4.2 Wstępne oczyszczenie danych

Po zaimportowaniu plików wynikowych do środowiska dokonano wizualnej inspekcji pierwszych rekordów w celu weryfikacji poprawności procesu ekstrakcji. Jak przedstawiono na Rysunku 29, dane w swojej surowej postaci obarczone są znacznym szumem informacyjnym i formatowaniem specyficznym dla struktury źródłowej strony HTML. W obecnym kształcie zbiór nie nadaje się do dalszego przetwarzania. Konieczne jest zatem przeprowadzenie procesu czyszczenia danych, mającego na celu usunięcie artefaktów ekstrakcji, normalizację formatów oraz wyodrębnienie czystych cech niezbędnych do dalszej eksploracji. Dodatkowo zbiory są rozdzielone co uniemożliwia solidną analizę.

	username	review_text	date_posted	recommendation
0	Aezarith	Posted: 13 October\nPlay this game before I send a homeless man...	Posted: 13 October	Recommended
1	Big Fellas	Posted: 13 October\nReally great so far!	Posted: 13 October	Recommended
2	812	Posted: 13 October\nfire keeper <3	Posted: 13 October	Recommended
3	Aeth	Posted: 13 October\nGreat classic.	Posted: 13 October	Recommended
4	voltorstone	Posted: 13 October\nDifficult but worth every agony, its like t...	Posted: 13 October	Recommended
5	Crazyi	Posted: 13 October\nGiving it a thumbs up, but only on sale bec...	Posted: 13 October	Recommended
7	The Koolony On Mars	Posted: 13 October\nMy god, this game is good but it has no clo...	Posted: 13 October	Not Recommended
8	RAT STEW!!!!	Posted: 13 October\nthe thing that really elevated this game to...	Posted: 13 October	Recommended
9	ryanakaikae5	Posted: 13 October\nserious thoughts create me. I'm like trying...	Posted: 13 October	Not Recommended
10	Trota	Posted: October 13\nNothing to say, it's a must to play.	Posted: October 13	Recommended

Rysunek 29 Wygląd danych po imporcie

Źródło: opracowanie własne

Import bibliotek pandas oraz numpy stanowi standardowy krok inicjalizujący środowisko pracy. W kontekście czyszczenia danych, pandas jest niezbędny do wczytania pliku CSV do obiektu ramki danych i wykonywania operacji na wierszach i kolumnach. Biblioteka numpy zostaje zaimportowana pomocniczo, przede wszystkim w celu obsługi wartości numerycznych oraz, co kluczowe w tym etapie, do zarządzania brakującymi danymi, co pozwala na ich późniejszą identyfikację i uzupełnianie. W celu przeprowadzenia wstępnej eksploracyjnej analizy danych oraz graficznej prezentacji wyników środowisko uzupełniono o biblioteki matplotlib oraz seaborn. Narzędzia te umożliwiają wizualizację rozkładów zmiennych, co jest niezbędne dla oceny jakości danych. Warstwa przetwarzania języka naturalnego oparta została na bibliotece nltk oraz module string. Wykorzystanie specyficznych komponentów, takich jak lista słów nieznaczących oraz lematyzator, pozwala na skuteczną redukcję szumu informacyjnego poprzez usunięcie zbędnych elementów i sprowadzenie wyrazów do ich form podstawowych, natomiast moduł collections służy do analizy częstości występowania tokenów. Fundamentem etapu modelowania jest biblioteka scikit-learn. Zaimplementowano z niej szereg modułów odpowiedzialnych za kompleksową obsługę procesu uczenia maszynowego, począwszy od podziału zbioru danych i obsługi nierównowagi klas, przez wektoryzację tekstu metodą TF-IDF, aż po implementację wybranego algorytmu regresji logistycznej oraz ewaluację jego skuteczności za pomocą raportów klasyfikacyjnych i metryk statystycznych. Import bibliotek niezbędnych do tego etapu widoczny jest na Rysunku 30.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from collections import Counter
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from collections import Counter
```

Rysunek 30 Import bibliotek

Źródło: opracowanie własne

Aby rozwiązać problem rozdzielonych danych posłużono się funkcją `concat` biblioteki `pandas`, która umożliwia łączenie ramek danych. Zaimportowane dane do osobnych ramek danych zostały także oznaczone dodaniem kolumny `[game]`, która definiuje jakiego produktu dotyczą opinie. Następnie zostały połączone z wykorzystaniem funkcji `concat`. Kod realizujący wczytanie i łączenie zbiorów danych z różnych plików w jedną ramkę danych przedstawia Rysunek 31.

```
ds1 = pd.read_csv('DS1.csv'); ds1["game"] = "Dark Souls"  
ds2 = pd.read_csv('DS2.csv'); ds2["game"] = "Dark Souls 2"  
ds3 = pd.read_csv('DS3.csv'); ds3["game"] = "Dark Souls 3"  
sek = pd.read_csv('SEKIRO.csv'); sek["game"] = "Sekiro"  
er = pd.read_csv('ER.csv'); er["game"] = "Elden Ring"  
x = [ds1, ds2, ds3, sek, er]  
df = pd.concat(x, ignore_index=True)
```

Rysunek 31 Łączenie zbiorów danych z różnych plików w jedną ramkę danych
Źródło: opracowanie własne

Istotnym defektem surowego zbioru danych było zanieczyszczenie kolumny z treścią recenzji nadmiarowymi metadanymi daty publikacji, które zostały błędnie zinterpretowane jako integralna część opinii. W celu eliminacji tego szumu informacyjnego zastosowano transformację opartą na wyrażeniach regularnych (RegEx), precyzyjnie identyfikującą i usuwającą powtarzalny nagłówek techniczny zakończony znakiem nowej linii. Zabieg ten pozwolił na skuteczne wyizolowanie właściwej, semantycznej warstwy tekstu od artefaktów procesu ekstrakcji danych.

Kolumna `date_posted` w surowej postaci zawierała prefiks tekstowy "Posted: ", który uniemożliwiał interpretację tej kolumny jako zmiennej czasowej. Zastosowano metodę `str.replace` zaprezentowaną na Rysunku 32, aby usunąć stały fragment tekstu. Jest to krok preprocessingu niezbędny do późniejszej konwersji typu danych (casting) ze string na `datetime`, co pozwoli na analizę trendów w czasie.

```
df['review_text'] =  
df['review_text'].str.replace(r'Posted:.*?\n', '', regex=True)  
df['date_posted'] = df['date_posted'].str.replace('Posted: ',  
'', regex=False)
```

Rysunek 32 Usuwanie artefaktów technicznych z tekstu recenzji i daty
Źródło: opracowanie własne

W ramach normalizacji danych przeprowadzono binaryzację zmiennej decyzyjnej `recommendation`, transformując jej wartości z formatu tekstowego na typ logiczny (Boolean). Operacja ta polegała na zmapowaniu etykiet „Recommended” oraz „Not Recommended” odpowiednio na wartości `True` i `False` zgodnie z logiką przedstawioną na Rysunku 33. Zabieg ten nie tylko zoptymalizował wykorzystanie pamięci operacyjnej, ale również umożliwił bezpośrednie zastosowanie operacji arytmetycznych, takich jak obliczanie średniej w celu wyznaczenia wskaźnika pozytywnych opinii.

```
df['recommendation'] =  
df['recommendation'].replace({'Recommended': True, 'Not  
Recommended': False})
```

Rysunek 33 Binaryzacja zmiennej rekomendacji
Źródło: opracowanie własne

Podczas eksploracji pobranego zbioru danych zidentyfikowano istotną heterogeniczność w kolumnie `date_posted`, wynikającą ze specyfiki wyświetlania dat przez platformę źródłową. Występują tam trzy odmienne standardy zapisu, pełna data z rokiem (dla wpisów archiwalnych) oraz dwa warianty skrócone (dla wpisów z bieżącego roku), różniące się inwersją dnia i miesiąca. Przykłady dat przedstawiono w pierwszych liniach Rysunku 34 przedstawiającego logikę funkcji parsującej datę. W celu normalizacji tych danych utworzono funkcję `parse_universal_date`. Logika funkcji zawiera:

- **Tokenizacja i czyszczenie:** Ciąg znaków jest pozbawiany znaków interpunkcyjnych i dzielony na składowe (tokeny).
- **Detekcja formatu:** Algorytm analizuje liczbę tokenów. W przypadku trzech elementów (miesiąc, dzień, rok) data jest interpretowana bezpośrednio.
- **Obsługa brakującego roku:** Dla formatów dwuelementowych (brak roku w źródle) system stosuje przyjmuje za rok 2025
- **Dynamiczne rozpoznawanie składowych:** Funkcja weryfikuje typ danych poszczególnych pierwszego tokenu (metoda `isdigit()`), aby

Tak przygotowana funkcją wykorzystywana jest w dalszej części kodu do automatyzacji procesu transformacji danych z kolumny `date_posted`.

```
def parse_universal_date(date_str):  
    default_year = 2025  
    if not isinstance(date_str, str):  
        return (None, None, None)  
    parts = date_str.replace(',', ' ').split()  
    day, month_name, year = None, None, None  
    if len(parts) == 3:  
        month_name = parts[0]  
        day = int(parts[1])  
        year = int(parts[2])  
    elif len(parts) == 2:  
        year = default_year  
        if parts[0].isdigit():  
            day = int(parts[0])  
            month_name = parts[1]  
        else:  
            month_name = parts[0]  
            day = int(parts[1])  
    return (day, month_name, year)
```

Rysunek 34 Funkcja normalizująca zróżnicowane formaty daty
Źródło: opracowanie własne

Wykorzystując `apply`, zdefiniowana wcześniej funkcja `parse_universal_date` została wywołana dla każdego rekordu w kolumnie `date_posted`. Wynik został rozpakowany do trzech niezależnych kolumn składowych - `day`, `month_name` oraz `year`. Biblioteka Pandas

w metodzie konstruującej obiekty czasowe, wymaga numerycznej reprezentacji miesięcy. Ponieważ dane źródłowe zawierały pełne nazwy angielskie (np. "October"), konieczne było stworzenie słownika mapującego (month_map). Za pomocą wektorowej metody map, wartości tekstowe w kolumnie month_name zostały przetłumaczone na odpowiadające im liczby całkowite (1-12), tworząc nową kolumnę month. Kluczowym etapem była agregacja przygotowanych kolumn numerycznych (year, month, day) do pojedynczego obiektu typu datetime64[ns] za pomocą funkcji pd.to_datetime. Zastosowanie parametru errors='coerce' stanowi zabezpieczenie algorytmu w przypadku wystąpienia daty niemożliwej (np. 30 lutego), system wstawi wartość NaT (Not a Time) zamiast przerwać działanie błędem. Proces zakończono usunięciem kolumn pośrednich (month_name, year, month, day) oraz pierwotnej kolumny tekstowej (date_posted). Zastosowanie konwersji daty w ramce danych przedstawia kod na Rysunku 35.

```
date_parts = df['date_posted'].apply(parse_universal_date)
df[['day', 'month_name', 'year']] =
pd.DataFrame(date_parts.tolist(), index=df.index)
month_map = {'January': 1, 'February': 2, 'March': 3, 'April':
4, 'May': 5, 'June': 6, 'July': 7, 'August': 8, 'September': 9,
'October': 10, 'November': 11, 'December': 12}
df['month'] = df['month_name'].map(month_map)
df['date'] = pd.to_datetime(df[['year', 'month', 'day']],
errors='coerce')
df = df.drop(columns=['month_name'])
df = df.drop(columns=['date_posted'])
df = df.drop(columns=['day'])
df = df.drop(columns=['month'])
df = df.drop(columns=['year'])
```

Rysunek 35 Zastosowanie konwersji daty w ramce danych
Źródło: opracowanie własne

Eliminacja redundancji danych pozwoliła na uzyskanie czystego, znormalizowanego schematu tabeli przedstawionego na Rysunku 36, gotowego do analizy dalszej analizy. Po szeregu powyżej zdefiniowanych operacji wynikiem jest ramka danych, którą dzieli jedynie etap przekształcenia tekstu do bycia zdantą do dalszych analiz.

username	review_text	recommendation	game_name	date
0 Szechuan Sauce	One of the best games of all time.	• True	Dark Souls 1	2025-11-24
1 komsigashmiga	dark souls 1 is THE game to play	• True	Dark Souls 1	2025-11-24
2 PindaNinja	its dark souls, so yea... its awesome!!	• True	Dark Souls 1	2025-11-24
3 SenpaiJesus	Good game	• True	Dark Souls 1	2025-11-24
4 Hxymer	This game super fun	• True	Dark Souls 1	2025-11-24
5 Qual	Опять серебряный рыцарь сложнее чем боссы 50 урона критом из 50	• True	Dark Souls 1	2025-11-24
6 BIGCUMMER	GET ON DARK SOULS	• True	Dark Souls 1	2025-11-23
7 The Elden Wombat	Made me wanna die in real life 5 stars	• True	Dark Souls 1	2025-11-23
8 deIarey	One of the best.	• True	Dark Souls 1	2025-11-23
9 PhantomHalo451	I must say probably one of my most favourite gaming experiences...	• True	Dark Souls 1	2025-11-24

Rysunek 36 Dane po transformacjach
Źródło: opracowanie własne

Pierwszym mankamentem danych tekstowych jest spam, który jest czynnikiem zanieczyszczającym wcześniej zmienione zmienne tekstowe. Celem usunięcia spamu zostały użyte w tej części podstawowe sposoby usuwania bez konstruktywnej treści tj. usuwanie znaków ASCII i pozbywanie się skrajnie krótkich opinii. Algorytm w pierwszej kolejności wyznacza całkowitą długość każdego wpisu oraz zlicza wystąpienia znaków specjalnych, niebędących literami ani cyframi, wykorzystując do tego wyrażenia regularne. Na podstawie tych wartości kalkulowany jest współczynnik zaszumienia tekstu, który służy do automatycznej identyfikacji tak zwanych grafik ASCII oraz treści o niskiej wartości semantycznej. Właściwa selekcja danych odbywa się poprzez nałożenie maski logicznej, która eliminuje rekordy charakteryzujące się współczynnikiem zaszumienia przekraczającym 40% oraz wpisy skrajnie krótkie, liczące poniżej 10 znaków. Na skutek filtru spamu ze zbioru znika około 20% rekordów. Funkcję filtrującą spam zaimplementowano w sposób widoczny na Rysunku 37. Tak wstępnie oczyszczone dane są następnie poddawane są usuwaniu stopwords.

```
def filter_spam_optimized(df):
    lengths = df['review_text'].str.len()
    non_alnum_counts = df['review_text'].str.count(r'^a-zA-Z0-9\s')
    ratios = non_alnum_counts / lengths
    mask = (ratios <= 0.40) & (lengths > 10)
    return df[mask]
df = filter_spam_optimized(df)
```

Rysunek 37 Funkcja filtrująca spam

Źródło: opracowanie własne

Implementacja mechanizmu czyszczenia tekstu przedstawionarozpoczęła się od redefinicji standardowego zbioru słów stopwords. W celu zachowania kontekstu sentymentu, kluczowego dla dalszej klasyfikacji, wykorzystano operację różnicy zbiorów, wykluczając z domyślnej listy NLTK zdefiniowany podzbiór słów negujących. Równolegle zainicjowano obiekt tablicy translacji przy użyciu metody statycznej str.maketrans, dedykowany do wysokowydajnej eliminacji znaków interpunkcyjnych bez konieczności stosowania wolniejszych wyrażeń regularnych. Konfigurację niestandardowej listy stopwords oraz tabeli translacji przedstawia Rysunek 38.

```
words_to_keep =
{'no', 'not', 'nor', 'neither', 'none', "don't", "couldn't"}
custom_stopwords = set(stopwords.words('english')) -
words_to_keep
stop_words = custom_stopwords
punct_table = str.maketrans(' ', ' ', string.punctuation)
```

Rysunek 38 Konfiguracja niestandardowej listy stopwords

Źródło: opracowanie własne

Właściwa transformacja danych, przedstawiona na Rysunku 39, została zrealizowana poprzez metodę apply, aplikującą funkcję lambda do każdego rekordu serii danych. Wewnątrz funkcji zagnieżdżono sekwencyjny potok operacji. Potok inicjuje normalizację wielkości znaków (lower), mapowanie translacyjne usuwające interpunkcję, tokenizację

metodą split oraz ostateczną filtrację tokenów z wykorzystaniem wcześniej ustalonej listy stopwords. Algorytm wyposażono w walidację typu danych (isinstance), zapewniającą odporność na błędy w przypadku wystąpienia wartości null lub nieliterowych. Proces zakończono optymalizacją struktury ramki danych poprzez usunięcie kolumny źródłowej.

```
df['cleaned_review'] = df['review_text'].apply(lambda x: " ".join(
    [word for word in x.lower().translate(punct_table).split()
     if word not in stop_words])
    if isinstance(x, str) else ""
)
df = df.drop(columns=['review_text'])
```

Rysunek 39 Implementacja czyszczenia i tokenizacji tekstu
Źródło: opracowanie własne

Po usunięciu stopwords zauważamy na Rysunku 40 znaczną redukcję ilości tekstu w kolumnie odpowiadającej treści opinii jak i widać efekt zastosowanej funkcji .lower(), normalizującej tekst usuwając wielkie litery.



```
cleaned_review
one best game time
dark soul 1 game play
dark soul yea awesome
game super fun
get dark soul
made wanna die real life 5 star
one best
must say probably one favourite gaming expe...
tbh thought liked ds3elden ring youll like ...
dispite people might tell dsr best one soul...
```

Rysunek 40 Kolumna cleaned_review po usunięciu stopwords
Źródło: opracowanie własne

Następnym krokiem w potoku przetwarzania tekstu była lematyzacja, mająca na celu redukcję przestrzeni cech poprzez unifikację form fleksyjnych słownictwa. Proces ten przedstawiono na Rysunku 41, gdzie zrealizowano go przy użyciu obiektu WordNetLemmatizer, który wykorzystuje leksykalną bazę danych WordNet do identyfikacji form bazowych wyrazów. Transformację zaaplikowano do kolumny z oczyszczonymi recenzjami, wykorzystując metodę apply do iteracji po wszystkich rekordach ramki danych. Wewnątrz funkcji anonimowej zaimplementowano logikę polegającą na tokenizacji ciągu znaków metodą split, a następnie mapowaniu każdego tokenu odpowiadającemu odpowiednikowi lematyzacyjnemu.

```

lemmatizer = WordNetLemmatizer()
df['cleaned_review'] = df['cleaned_review'].apply(lambda x: "
".join([lemmatizer.lemmatize(word) for word in x.split()])))

```

Rysunek 41 Przeprowadzenie lematyzacji tekstu
Źródło: opracowanie własne

Tak przygotowana struktura danych, pozbawiona artefaktów technicznych i form fleksyjnych, stanowi zbiór danych gotowych do procesu wektoryzacji i trenowania modelu predykcyjnego. W celu łatwiejszej optymalizacji i dostosowywania modelu do spełnienia wcześniej założonych celów najpierw wykonywana jest eksploracyjna analiza danych

4.3 Eksploracyjna analiza danych

Mając do dyspozycji przetworzony i oczyszczony zbiór danych, kolejnym logicznym krokiem w procesie badawczym jest jego eksploracyjna analiza. Etap utworzony jest na podstawie przykładowych danych 10000 opinii, o każdym produkcie i jego wyniki mogą się nieznacznie różnić w zależności od czasu pobrania opinii z pomocą webscrapera. W związku z tym przedstawione w poniższym dziale analizy są ogólnym opisem danych, a nie wchodzeniem w szczegóły dotyczące konkretnie opisywanych danych. Etap ten służy nie tylko ilościowej charakterystyce zgromadzonego materiału badawczego, ale przede wszystkim weryfikacji założeń dotyczących rozkładu zmiennych, co jest fundamentem dla doboru odpowiednich metod modelowania. W niniejszym podrozdziale analiza koncentruje się na dwóch kluczowych wymiarach - zbadaniu balansu klas decyzyjnych (rekomendacji pozytywnych i negatywnych) oraz analizie statystyk opisowych dotyczących długości i struktury samych opinii. Wnioski płynące z tej fazy pozwolą na ocenę potencjału analitycznego zbioru oraz zidentyfikowanie ewentualnych anomalii mogących wpłynąć na stabilność trenowanych w dalszej kolejności algorytmów klasyfikacyjnych. Ramka danych obejmuje 39749 obserwacji opisanych czterema atrybutami kluczowymi dla dalszej eksploracji zgodnie z Rysunkiem 42. Zmienne recommendation (typ logiczny), date (format datetime64) oraz game_name (typ obiektowy) charakteryzują się pełną kompletnością. Jedynie w kolumnie text, przechowującej przetworzoną treść recenzji, zidentyfikowano marginalny ubytek danych (10 wartości pustych), które powstały na skutek usunięcia stopwords. Te braki automatycznie zostaną usunięte w etapie modelowania.

```

RangeIndex: 39749 entries, 0 to 39748
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   text            39739 non-null  object
1   recommendation  39749 non-null  bool
2   date            39749 non-null  datetime64[ns]
3   game_name       39749 non-null  object

```

Rysunek 42 Wynik funkcji .info()
Źródło: opracowanie własne

Analiza najczęściej występujących tokenów, przedstawiona na Rysunku 43, ujawnia specyficzną strukturę danych. Na szczycie listy znajduje się słowo „game”, które jako termin generyczny niesie znikomą wartość informacyjną. Jednocześnie obecność w czołówce słów o silnym ładunku pozytywnym, takich jak „good” i „best”, koresponduje z zaobserwowaną wcześniej przewagą recenzji rekomendujących, co wstępnie sugeruje ogólne zadowolenie społeczności graczy i może być w fazie modelowania dobrym wskaźnikiem wydźwięku. Z perspektywy budowy modelu analitycznego, kluczowym wnioskiem jest wysoka pozycja tokenu „not” oraz wieloznacznego słowa „like”. Powszechność negacji sygnalizuje ryzyko błędnej klasyfikacji sentymentu przez proste modele leksykalne, co uzasadnia konieczność zastosowania n-gramów do przechwytywania szerszego kontekstu, przykładowo odróżnienia „good” od „not good”

	token	count
0	game	46572
1	soul	15821
2	like	9553
3	not	9264
4	dark	9264
5	good	6999
6	time	6969
7	one	6729
8	best	5993
9	play	5531

Rysunek 43: Lista słów w opiniach
Źródło: opracowanie własne

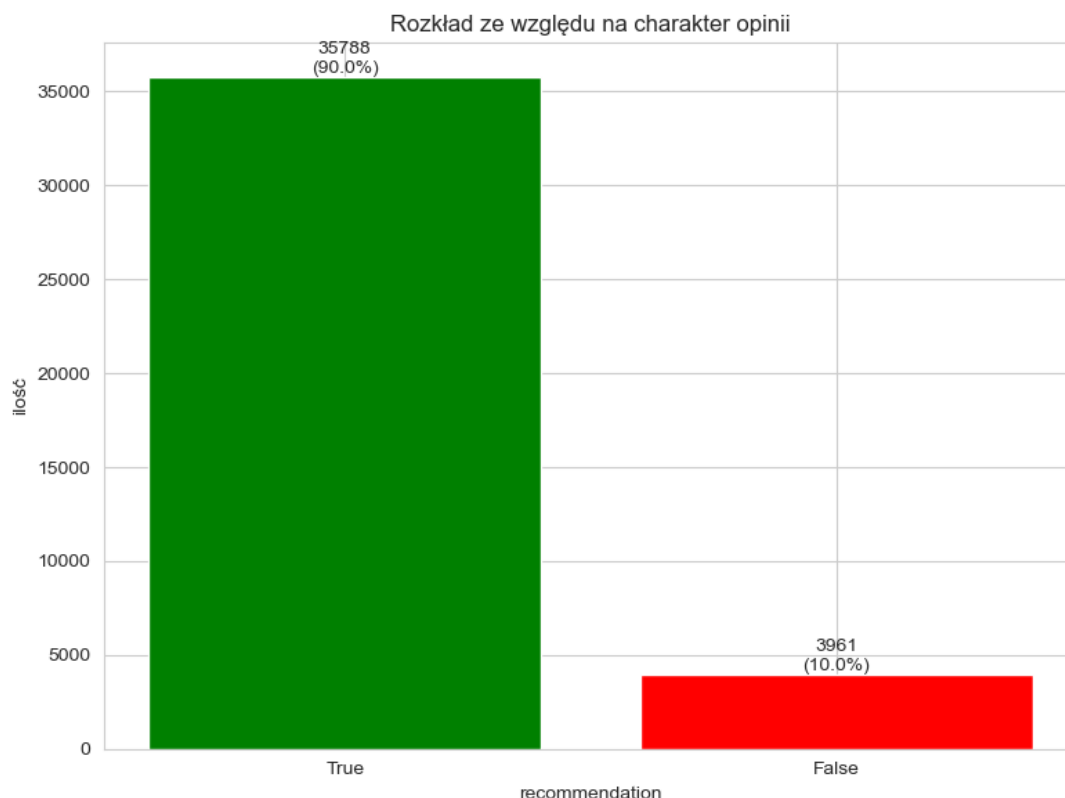
W celu uchwycenia szerszego kontekstu wypowiedzi, analizę rozszerzono o generowanie bigramów. Jak wynika z przedstawionego zestawienia na Rysunku 44, powtarzalność fraz dwuwyrzowych jest naturalnie znacznie rzadsza niż pojedynczych tokenów, co skutkuje dominacją nazw własnych i określeń gatunkowych w czołówce rankingów.

	token	count
0	dark soul	8455
1	soul game	3758
2	elden ring	3564
3	best game	1841
4	game ever	1675
5	soul 2	1497
6	feel like	1463
7	good game	1352
8	great game	1205
9	love game	1131

Rysunek 44 Lista bigramów w opiniach
Źródło: opracowanie własne

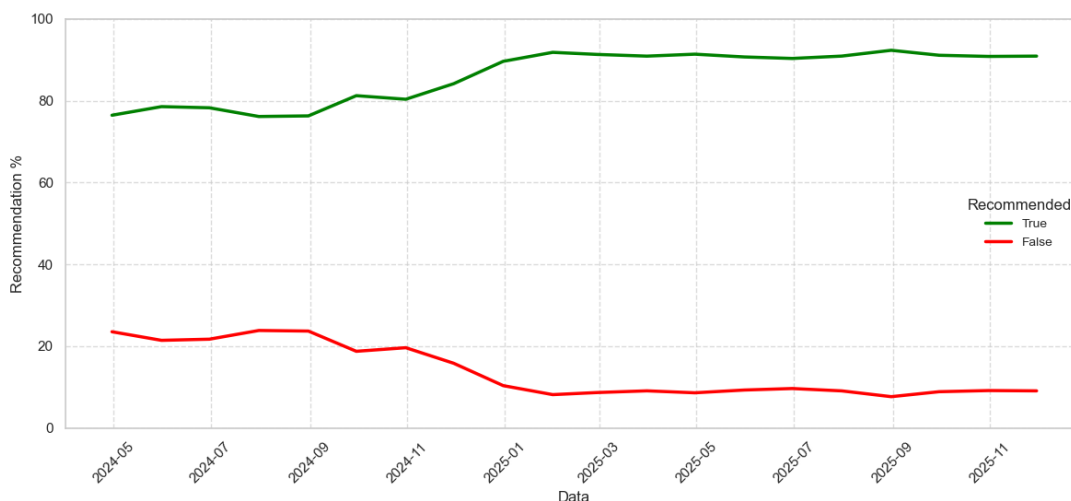
Analiza rozkładu zmiennej decyzyjnej recommendation, wizualnie przedstawiona na Rysunku 45, ujawnia skrajne niezbalansowanie klas. Dominującą grupę stanowią opinie

pozytywne (oznaczone kolorem zielonym), których odnotowano 35 788, co stanowi aż 90% całego zbioru. W kontraście do nich, opinie negatywne (oznaczone kolorem czerwonym) są reprezentowane przez zaledwie 3 961 próbek, czyli 10% zbioru. Taka dysproporcja (stosunek 9:1) stanowi krytyczne wyzwanie dla procesu uczenia maszynowego. Trenowanie modelu na surowych danych o takiej strukturze niesie wysokie ryzyko, że algorytm będzie dążył do maksymalizacji ogólnej trafności poprzez faworyzowanie klasy większościowej, ignorując jednocześnie sygnały płynące z recenzji negatywnych.



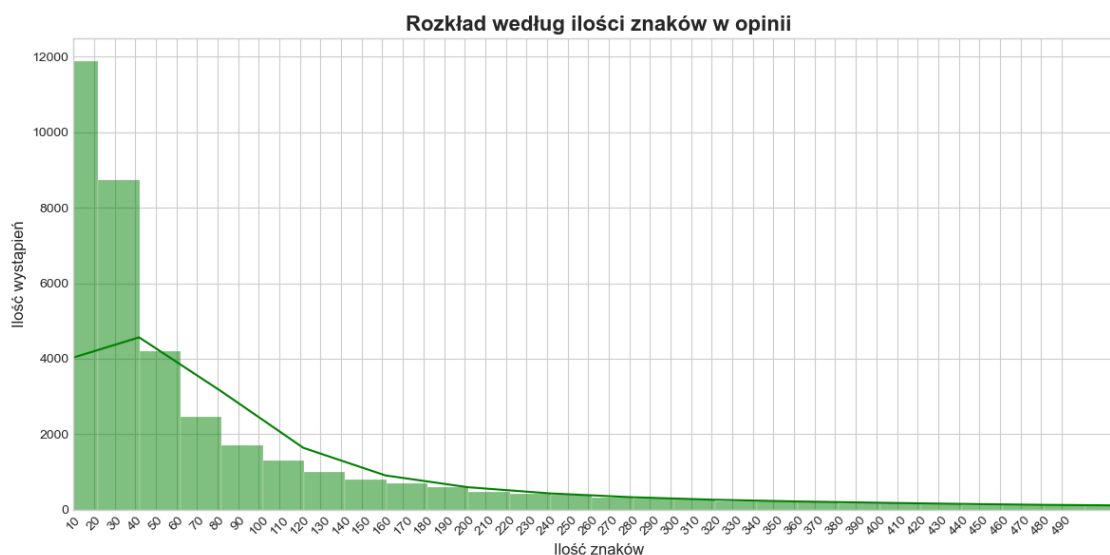
Rysunek 45 Rozkład charakteru opinii
Źródło: opracowanie własne

Celem pogłębienia analizy utworzono wykres poszerzający rozkład o analizę w czasie. Dysproporcja pozytywnych do negatywnych opinii utrzymuje się w czasie, co bez względu na czas pobrania danych jest zagrożeniem dla działania modelu. Aby temu zapobiec i wymusić na modelu poprawną identyfikację krytycznych uwag graczy, w etapie przygotowania danych do modelowania konieczne będzie zastosowanie techniki downsamplingu klasy większościowej. Zredukowanie liczby próbek pozytywnych pozwoli na wyrównanie proporcji klas i skuteczniejszy proces uczenia. Rozkład charakteru opinii w ujęciu czasowym prezentuje Rysunek 46.



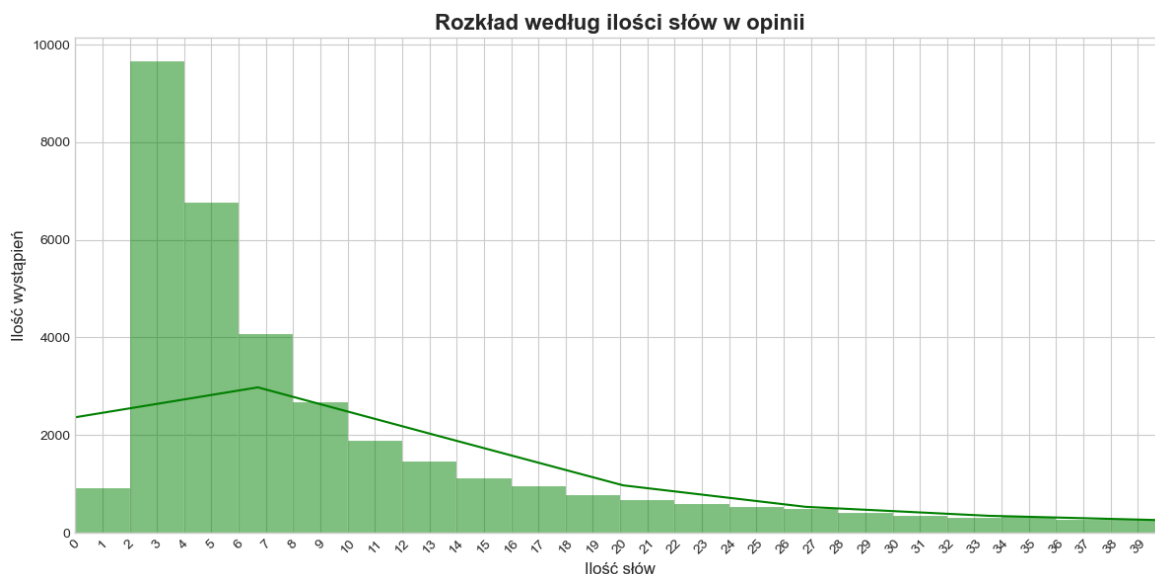
Rysunek 46 Rozkład ze względu na charakter opinii w czasie
Źródło: opracowanie własne

Analiza histogramu rozkładu długości recenzji wyrażonej w liczbie znaków ujawnia silną asymetrię rozkładu danych. Jak widać na Rysunku 47, dominującą grupę w zbiorze stanowią wpisy skrajnie krótkie. Najwyższa frekwencja, sięgająca blisko 12 000 wystąpień, odnotowana została dla przedziału poniżej 20 znaków, a znacząca większość wszystkich opinii mieści się w granicach do 100 znaków. Taka charakterystyka zbioru niesie ze sobą istotne implikacje dla doboru metod przetwarzania tekstu. Przewaga krótkich form (często jednozdaniowych lub będących pojedynczymi wyrazami) sugeruje, że znaczna część danych to komunikaty o wysokim ładunku emocjonalnym, ale niskiej gęstości informacyjnej, co jest korzystne dla prostej klasyfikacji sentymentu.



Rysunek 47 Histogram ilości znaków
Źródło: opracowanie własne

Uzupełnieniem analizy liczby znaków jest zbadanie rozkładu długości opinii wyrażonej w liczbie słów. Przedstawiony na Rysunku 48 wykres potwierdza silną prawostronną asymetrię danych. Znaczna większość słów mieści się w przedziale od 1 do 10 słów, co może wskazywać na impulsywny i emocjonalny charakter opinii graczy.



Rysunek 48 Histogram ilości słów

Źródło: opracowanie własne

Przeprowadzona powyżej eksploracyjna analiza danych pozwoliła na zdiagnozowanie fundamentalnych cech zgromadzonego zbioru. Zgodnie z założeniami metodyki CRISP-DM, etap zrozumienia danych nie jest jedynie formalnością, lecz krytycznym krokiem decydującym o dalszych częściach procesu. Eksploracja dostarczyła kluczowego wglądu w strukturę opinii graczy, obnażając takie wyzwania jak silne niezbalansowanie klas decyzyjnych czy przewaga krótkich, emocjonalnych form wypowiedzi. Zidentyfikowanie tych właściwości na tym etapie pozwala na świadomą kalibrację algorytmów w dalszej części procesu. Posiadając zweryfikowaną wiedzę na temat jakości i charakterystyki materiału badawczego, projekt jest gotowy do przejścia do zasadniczego etapu analitycznego, jakim jest właściwa analiza sentymentu oraz ekstrakcja czynników sukcesu.

4.4 Analiza sentymentu

4.4.1 Ekstrakcja wydźwięku czynników

W oparciu o wnioski płynące z przeprowadzonej w poprzednim podrozdziale eksploracyjnej analizy danych, która ujawniła zarówno znaczną dysproporcję klas decyzyjnych, jak i specyficzną strukturę leksykalną opinii, projekt przechodzi do kluczowej fazy modelowania. Jak wykazano wcześniej, prosta analiza częstotliwościowa tokenów jest niewystarczająca do precyzyjnego określenia czynników sukcesu, ponieważ nie uwzględnia ona kontekstu ani wagi emocjonalnej poszczególnych fraz. W tym celu zostaje utworzony model klasyfikacyjny, którego predyktory zostaną wykorzystane jako waga dźwięku opinii.

W pierwszym kroku następuje przygotowanie zmiennych do procesu uczenia maszynowego. Zmienna decyzyjna *recommendation*, która wcześniej została zbinaryzowana do wartości logicznych (True/False), jest teraz rzutowana na typ całkowity (integer), gdzie 1 oznacza rekomendację pozytywną, a 0 negatywną. Następnie wyodrębniane są zmienne, gdzie objaśniająca *X_raw* zawiera oczyszczony tekst recenzji, oraz objaśniana *y_raw*, stanowiąca cel predykcji. Kod przygotowania zmiennych wejściowych i decyzyjnych do modelowania widoczny jest na Rysunku 49.

```
df['target'] = df['recommendation'].astype(int)
X_raw = df['cleaned_review']
y_raw = df['target']
```

Rysunek 49 Przygotowanie zmiennych wejściowych i decyzyjnych do modelowania
Źródło: opracowanie własne

Zgodnie z założeniami projektowymi, zbiór danych dzielony jest na część uczącą i testową w proporcji 70:30. Kluczowym parametrem jest tutaj `stratify=y_raw`, który gwarantuje, że proporcja klas pozytywnych do negatywnych w obu podzbiorach będzie identyczna jak w oryginalnym zbiorze danych, co zapewnia statystyczną reprezentatywność próbki. Podział danych na zbiór treningowy i testowy zrealizowano kodem przedstawionym na Rysunku 50.

```
X_train_raw, X_test, y_train_raw, y_test = train_test_split(
    X_raw, y_raw, test_size=0.30, random_state=42, stratify=y_raw
)
train_data = pd.DataFrame({'text': X_train_raw, 'target':
    y_train_raw})
```

Rysunek 50 Podział danych na zbiór treningowy i testowy
Źródło: opracowanie własne

W celu przygotowania do procesu równoważenia danych, zbiór treningowy zostaje tymczasowo rozdzielony na dwie niezależne ramki danych:

- `neg_train` - zawierającą wyłącznie recenzje negatywne
- `pos_train` - zawierającą recenzje pozytywne.

Jest to krok niezbędny do wykonania operacji na konkretnej klasie decyzyjnej. Separację klas decyzyjnych w zbiorze treningowym pokazuje Rysunek 51.

```
neg_train = train_data[train_data['target'] == 0]
pos_train = train_data[train_data['target'] == 1]
```

Rysunek 51 Separacja klas decyzyjnych w zbiorze treningowym
Źródło: opracowanie własne

Ze względu na zidentyfikowaną w analizie eksploracyjnej silną dysproporcję klas (90% opinii pozytywnych), zastosowano technikę redukcji klasy większościowej. Algorytm losowo redukuje liczbę recenzji pozytywnych, ustalając ich stosunek do recenzji negatywnych na poziomie 2:1 (`n_samples=len(neg_train) * 2`). Jest to podejście preferowane nad upsamplingiem w przypadku danych tekstowych, pozwalające na redukcję szumu przy zachowaniu wystarczającej liczby przykładów uczących. Kod zabezpieczony jest przez wyrażenie warunkowe, ponieważ istnieje szansa, że zbiór negatywnych opinii większy. W takim przypadku balansowanie nie następuje. Strukturę kodu przedstawia Rysunek 52.

```

if len(pos_train) > len(neg_train):
    pos_train_downsampled = resample(pos_train,
                                     replace=False,
                                     n_samples=len(neg_train) * 2,
                                     random_state=42)
    balanced_train = pd.concat([neg_train, pos_train_downsampled])
else:
    balanced_train = train_dat

```

Rysunek 52 Równoważenie klas metodą downsamplingu

Źródło: opracowanie własne

Po zbalansowaniu proporcji klas, dane są ponownie rozdzielane jak widać na Rysunku 53 na serie `X_train_bal` (tekst) i `y_train_bal` (target). Na tym etapie zbiór treningowy jest już gotowy do procesu wektoryzacji, mając strukturę wymuszającą na modelu zwracanie większej uwagi na opinie negatywne.

```

X_train_bal = balanced_train['text']
y_train_bal = balanced_train['target']

```

Rysunek 53 Finalne przygotowanie zbalansowanego zbioru treningowego

Źródło: opracowanie własne

Kolejnym krokiem jest wektoryzacja kolumny tekstowej poprzez inicjalizację obiektu `TfidfVectorizer`, który posłuży do transformacji tekstu w macierz numeryczną metodą TF-IDF. Kluczowym ustawieniem jest `ngram_range=(1, 2)`, co pozwala na analizę zarówno pojedynczych słów, jak i bigramów (par wyrazów). Umożliwia to uchwycenie szerszego kontekstu wypowiedzi, co jest niezbędne np. przy obsługiwaniu negacji. `Max_features` maksymalną ilość tokenów ważnych dla modelu. `Min_df` ustawione na 5 gwarantuje, że do modelu nie będzie dostawał się szum, który znajduje się poniżej 5% wszystkich opinii. Konfigurację wektoryzatora TF-IDF przedstawia Rysunek 54.

```

tfidf = TfidfVectorizer(
    max_features=20000,
    ngram_range=(1, 2),
    min_df=5,

```

Rysunek 54 Konfiguracja wektoryzatora TF-IDF

Źródło: opracowanie własne

Następnie zachodzi właściwa wektoryzacja przedstawiona na Rysunku 55. Metoda `fit_transform` jest aplikowana wyłącznie na zbiorze treningowym (`X_train_bal`), aby "nauczyć" wektoryzator słownika tylko na podstawie danych uczących. Zbiór testowy jest następnie transformowany (`transform`) przy użyciu tego samego słownika, co zapobiega wyciekowi danych z testów do treningu.


```
X_train_vec = tfidf.fit_transform(X_train_bal)
X_test_vec = tfidf.transform(X_test)
```

Rysunek 55 Transformacja tekstu na macierze numeryczne

Źródło: opracowanie własne

Jako algorytm klasyfikacyjny wykorzystano regresję logistyczną. Decyzja ta wynika z jej wysokiej interpretowalności. W przeciwieństwie do bardziej złożonych modeli, których predyktory nie są jasno określone. Regresja logistyczna pozwala na bezpośrednią analizę wag przypisanych poszczególnym słowom, co jest niezbędne do realizacji celu pracy, czyli identyfikacji czynników sukcesu. Wewnętrzne ustawienia regresji logistycznej przedstawiono na Rysunku 56.

```
model = LogisticRegression(max_iter=5000, solver='liblinear')
model.fit(X_train_vec, y_train_bal)
```

Rysunek 56 Trenowanie modelu regresji logistycznych

Źródło: opracowanie własne

Model dokonuje predykcji sentymentu dla danych ze zbioru testowego (`X_test_vec`), którego model wcześniej "nie widział". Wyniki zgodnie z Rysunkiem 57 są zapisywane w zmiennej `y_pred` i posłużą do niezależnej ewaluacji jakości klasyfikatora. Predykcja dokonuje się z

```
y_pred = model.predict(X_test_vec)
```

Rysunek 57 Generowanie predykcji dla zbioru testowego

Źródło: opracowanie własne

Obliczane są trzy główne metryki. Dokładność (Accuracy), Kappa Cohena i AUC (Area under Curve). Ze względu na fakt, że zbiór testowy (w przeciwieństwie do treningowego) pozostał niezbalansowany (odzwierciedla rzeczywistość), sama dokładność mogła być myląca. Kod obliczający i jednocześnie wyświetlający metryki został przedstawiony poniżej na Rysunku 58.

```
y_pred_proba = model.predict_proba(X_test_vec)[: , 1]
auc_score = roc_auc_score(y_test, y_pred_proba)
acc = accuracy_score(y_test, y_pred)
kappa = cohen_kappa_score(y_test, y_pred)
print(f"Accuracy: {acc:.4f}")
print(f"Cohen's Kappa: {kappa:.4f}")
print(f"AUC: {auc_score:.4f}")
```

Rysunek 58 Obliczanie i wyświetlanie metryk skuteczności modelu

Źródło: opracowanie własne

Dodatkowo generowany jest pełny raport klasyfikacyjny, który pozwala ocenić precyzję (Precision) i czułość (Recall) dla każdej z klas osobno. Kod generujący raport przedstawia Rysunek 59, a jego przykładowy wygląd Rysunek 60.

```
print("\n--- Model Performance ---")
print(classification_report(y_test, y_pred,
target_names=['Negative', 'Positive']))
```

Rysunek 59 Generowanie i wyświetlanie szczegółowego raportu klasyfikacyjnego

```
Accuracy:      0.9067
Cohen's Kappa: 0.5241

--- Model Performance ---
              precision    recall  f1-score   support

   Negative      0.53      0.64      0.58      1188
   Positive      0.96      0.94      0.95     10737

 accuracy              0.91      11925
 macro avg      0.74      0.79      0.76      11925
weighted avg      0.92      0.91      0.91      11925
```

Rysunek 60 Wygląd przykładowych statystyk modelu

Źródło: opracowanie własne

Ostatni i najważniejszy krok z perspektywy biznesowej. Z wytrenowanego modelu ekstrahowane są nazwy tokenów oraz przypisane im współczynniki. Zgodnie z Rysunkiem 61 tworzona jest ramka danych `word_sentiment`, która zawiera informację o sile i kierunku wpływu każdego słowa na ocenę gry.

```
feature_names = tfidf.get_feature_names_out()
coefs = model.coef_[0]
word_sentiment = pd.DataFrame({'token': feature_names,
'coefficient': coefs})
```

Rysunek 61 Ekstrakcja wag wydźwięku z modelu

Źródło: opracowanie własne

Uzyskane wyniki widoczne na Rysunku 62 świadczą o tym, że surowe dane tekstowe zostały z powodzeniem przetransformowane w ustrukturyzowany zbiór metryk analitycznych. Wyekstrahowane tokeny wraz z ich wagami nie są już tylko zbiorem ciągów znaków, lecz wymiernymi wskaźnikami sentymentu, gotowymi do dalszej agregacji.

token	coefficient
best	5.160067
1010	4.600200
amazing	3.633406
love	3.475534
peak	2.928534
masterpiece	2.904293
good	2.882305
great	2.614183
favorite	2.308827
best game	2.177658

Rysunek 62 Przykładowe wagi wydźwięku
Źródło: opracowanie własne

Zbiór także zgodnie z założeniami projektu rozróżnia negacje i jest w stanie rozdzielić przykładowo słowo „recommend” i „cant recommend” nadając im polaryzujące wagi jak ukazano w poniższej Tabeli 8.

Tabela 8 Porównanie sentymentu tokenów zawierających słowo "recommend"
Źródło: opracowanie własne

token	coefficient
definitely recommend	0,307
recommend	0,911
cannot recommend	-0,292
cant recommend	-0,932

Utworzony zbiór jest zestawem tokenów o przypisanej zmiennej numerycznej „coefficient”, która definiuje wagę wydźwięku danego tokenu. Im wyższy wskaźnik coefficient tym większy wpływ frazy na sumaryczny sentyment opinii.

4.4.2 Segregacja tematyczna czynników

Zbiór już na tym etapie mógłby zostać wykorzystany do powierzchownej identyfikacji kluczowych czynników sukcesu gier studia FromSoftware, lecz w celu ułatwienia tego procesu tokeny zostaną dopasowane do zdefiniowanych kategorii tematycznych z wykorzystaniem predefiniowanego słownika jak i dodana do nich również zostanie ilość wystąpień w dokumentach. W tym celu w pierwszej kolejności na podstawie danych z ramki danych zawierających informacje o danych zostaną utworzone dwie ramki zawierające słowa pojedyncze słowa i bigramy.

Skrypt zliczający rozpoczyna się od agregacji wszystkich słów zawartych w kolumnie cleaned_review do jednej, płaskiej listy wordlist. Operacja ta realizowana jest poprzez iteracyjne dzielenie treści każdego wpisu na pojedyncze tokeny, co pozwala na spłaszczenie struktury danych. Następnie, wykorzystując klasę Counter z biblioteki collections, algorytm zlicza wystąpienia każdego unikalnego wyrazu w całym zbiorze opinii. Finalnie, uzyskany słownik mapujący słowa na ich liczebność zostaje przekonwertowany do postaci ustrukturyzowanej ramki danych df_counts, składającej się z kolumn określających sam

token oraz liczbę jego wystąpień. Zliczanie częstości występowania pojedynczych słów realizuje kod na Rysunku 63.

```
wordlist = [
    word
    for text in df['cleaned_review']
    for word in str(text).split()
]
token_counts = Counter(wordlist)
df_counts = pd.DataFrame(token_counts.items(), columns=['token',
'count'])
df_counts = df_counts.sort_values(by='count',
ascending=False).reset_index(drop=True)
```

Rysunek 63 Zliczanie częstości występowania pojedynczych słów
Źródło: opracowanie własne

Identycznie do poprzedniego działu skrypt budujący listę bigramów. Różnica pomiędzy dwoma sposobami wynika z rozpoczęcia pętli, która zaczyna się od funkcji zip, która sekwencyjnie łączy w pary słowa składające się na rekord wiersza. Po tym ich liczebność przekazywana jest do ramki danych df_bigrams tak samo jak w przypadku unigramów. Generowanie i zliczanie częstości występowania bigramów przedstawia Rysunek 64.

```
all_bigrams = [
    " ".join(bigram)
    for text in df['cleaned_review']
    for words in [str(text).split()]
    for bigram in zip(words, words[1:])
]

bigram_counts = Counter(all_bigrams)
df_bigrams = pd.DataFrame(bigram_counts.items(),
columns=['token', 'count'])
df_bigrams = df_bigrams.sort_values(by='count',
ascending=False).reset_index(drop=True)
```

Rysunek 64 Generowanie i zliczanie częstości występowania bigramów
Źródło: opracowanie własne

Następnie ramki zliczające unigramy i bigramy są łączone w jedną wielką ramkę df_counters, która będzie podłączona do ramki zawierającej wydźwięk tokenów. Agregację liczników słów i bigramów w jedną ramkę danych pokazuje Rysunek 65.

```
y = [df_counts, df_bigrams]
df_counters = pd.concat(y,ignore_index=True)
df_counters = df_counters.sort_values(by='count',
ascending=False).reset_index(drop=True)
```

Rysunek 65 Agregacja liczników słów i bigramów w jedną ramkę danych

Źródło: opracowanie własne

Połączenie zbiorów odbywa się dzięki funkcji merge, która łączy ze sobą ramki danych na podstawie wybranego klucza. W tym przypadku kluczem jest kolumna token. Jeśli token znajduje się i w ramce word_sentiment i df_counters to są łączone i zapisują się w nowo utworzonej ramce danych merged. Integrację wag sentymentu z danymi o liczebności realizuje kod na Rysunku 66.

```
merged = word_sentiment.merge(df_counters, on='token',
how='left')
```

Rysunek 66 Integracja wag sentymentu z danymi o liczebności

Źródło: opracowanie własne

Po dołączeniu liczebności do wydźwignię narzędzie przystępuje etapu wyszukiwania tematu słów. Przykładowy wynik ramki merged prezentuje Rysunek 67.

token	coefficient	count
010	-1.083284	49.0
05	0.246770	14.0
10	-0.000273	815.0
10 10	0.190728	53.0
10 enemy	0.116722	11.0
10 game	-0.031927	27.0
10 hour	-0.358415	63.0
10 minute	-0.648730	23.0
10 time	0.165835	45.0
10 year	0.172286	79.0

Rysunek 67 Przykładowy wynik ramki merged

Źródło: opracowanie własne

Na podstawie zbioru słów autor zaktualizował słownik o dodatkowe słowa składające się na tematy. Tak skonfigurowany słownik zostaje zapisany jako dict{} w formie:

- topic_dictionary = {temat:[element_1,element_2,element_n]}

A następnie wykonywane są operacje mające na celu połączyć słowa z danym tematem. Kompletną zawartość słownika tematów przedstawia Tabela 9.

Tabela 9 Słownik tematów
Źródło: opracowanie własne

Topic	Składowe
Cena	'price', 'cost', 'money', 'cheap', 'expensive', 'sale', 'worth', 'dollar', 'euro', 'buy', 'refund'
Udźwiękowanie	'sound', 'music', 'soundtrack', 'audio', 'voice', 'ost', 'song', 'dub', 'hearing', 'sfx'
Fabula	'story', 'lore', 'narrative', 'plot', 'ending', 'npc', 'quest', 'writing', 'character', 'dialogue', 'cutscene', 'cinema', 'cutscenes', 'storytelling', 'memorable', 'end', 'ending', 'experience', 'boring'
Wsparcie	'support', 'bug', 'fix', 'patch', 'dev', 'developer', 'update', 'broken', 'error', 'issue', 'glitch', 'devs', 'dlc'
Sterowanie	'control', 'keyboard', 'mouse', 'controller', 'gamepad', 'keybind', 'input', 'clunky', 'camera', 'bind'
Rozgrywka	'action', 'adaptability', 'aggressive', 'beating', 'challenging', 'defeat', 'defeated', 'hitboxes', 'replay', 'replayability', 'combat', 'boss', 'fight', 'mechanic', 'dodge', 'parry', 'difficult', 'hard', 'easy', 'challenge', 'fun', 'gameplay', 'bos', 'attack', 'level', 'weapon', 'die', 'death', 'explore', 'exploration', 'magic', 'spell', 'build'
Grafika	'graphic', 'visual', 'art', 'map', 'world', 'beautiful', 'ugly', 'texture', 'look', 'style', 'view', 'scenery', 'atmosphere', 'environment', 'animation', 'lighting', 'area', 'shadow'
Tryb Wieloosobowy	'multiplayer', 'pvp', 'coop', 'summon', 'invade', 'invasion', 'online', 'server', 'lag', 'connection', 'co-op', 'community', 'gank', 'ganks'
Optymalizacja	'performance', 'fps', 'optimize', 'crash', 'stutter', 'frame', 'run', 'pc', 'freeze', 'optimization', 'framerate', 'drop', '60fps', 'buggy'

W celu zoptymalizowania procesu przypisywania kategorii, dokonano transformacji struktury danych z pomocą kodu przedstawionego na Rysunku 68. Utworzono zbiór płaski `all_topic_values` zawierający wszystkie słowa kluczowe, co umożliwia błyskawiczne sprawdzanie ich obecności. Równolegle skonstruowano odwrócony słownik `value_to_topic`, który pozwala na natychmiastowe zmapowanie konkretnego słowa (np. "bug") na jego kategorię nadrzędną (np. "Wsparcie") bez konieczności iterowania po całym słowniku głównym.

```
all_topic_values =
set([value for values in topic_dictionary.values() for value in values])
value_to_topic =
{value: topic for topic, values in topic_dictionary.items() for value in values}
```

Rysunek 68 Optymalizowanie słownika tematów (spłaszczanie struktury)
Źródło: opracowanie własne

Zainicjowano pustą listę `topics_list` z pomocą linijki przedstawionej na Rysunku 69, która posłuży jako kontener na wyniki kategoryzacji. Będzie ona przechowywać przypisany temat dla każdego wiersza w ramce danych, zachowując kolejność zgodną z oryginalnym zbiorem tokenów.

```
topics_list = []
```

Rysunek 69 Inicjalizacja listy na wyniki kategoryzacji

Źródło: opracowanie własne

Główna część programu, przedstawiona na Rysunku 70, działa za pomocą pętli for. Pętla algorytmu realizuje zadanie przypisywania słów określonych przez model do zdefiniowanych wcześniej kategorii. Algorytm analizuje każdy element kolumny token (który może być unigramem lub bigramem). W przypadku bigramów (np. "fps drop"), fraza jest dzielona na składowe, a algorytm poszukuje dopasowania dla każdej z części. Jeśli zidentyfikowany zostanie przynajmniej jeden temat, jest on przypisywany do danego rekordu (w przypadku niejednoznaczności wybierany jest pierwszy alfabetycznie dla zachowania determinizmu). W przypadku braku dopasowania wstawiana jest wartość None.

```
for token_str in merged['token']:
    if not isinstance(token_str, str):
        topics_list.append(None)
        continue

    parts = token_str.split()
    found_topics = set()

    if len(parts) == 1:
        if parts[0] in all_topic_values:
            found_topics.add(value_to_topic[parts[0]])

    elif len(parts) >= 2:
        for part in parts:
            if part in all_topic_values:
                found_topics.add(value_to_topic[part])

    if found_topics:
        first_topic = sorted(list(found_topics))[0]
        topics_list.append(first_topic)
    else:
        topics_list.append(None)
```

Rysunek 70 Algorytm pętli przypisującej kategorie tematyczne do tokenów

Źródło: opracowanie własne

Ostatnim krokiem jest połączenie wcześniej utworzonej listy merged z tematami.

```
merged['topics'] = topics_list
```

Rysunek 71 Finalne przypisanie zidentyfikowanych tematów do ramki danych

Źródło: opracowanie własne

Efektom są zagregowane rezultaty widoczne na poniższym Rysunku 72. Wygenerowana lista tematów zostaje dołączona do głównej ramki danych jako nowa kolumna topics. Dzięki temu każdy wyekstrahowany przez model token posiada teraz nie tylko wagę (współczynnik) i liczebność, ale także interpretowalną etykietę kategorii, co jest niezbędne do konstrukcji mapy czynników sukcesu na pulpitach menedżerskich.

	token	coefficient	count	topics
100	worth	0.368251	32.0	Price
1tb	hard	0.070350	57.0	Gameplay
1v1	combat	-0.156615	6.0	Gameplay
60	fps	-0.544434	95.0	Performance
60	fps	-0.968519	72.0	Performance
	able run	0.069812	9.0	Performance
absolute	cinema	1.170947	244.0	Story
across	map	-0.202332	15.0	Graphics
	action	0.136983	456.0	Gameplay
action	game	0.110476	102.0	Gameplay

Rysunek 72 Ostateczna struktura danych

Źródło: opracowanie własne

Ostatnim krokiem jest zapis przetworzonej ramki danych do pliku wynik.xlsx dzięki funkcji `to_excel()` przedstawionej na Rysunku 73

```
merged.to_excel("wynik.xlsx", index = False)
```

Rysunek 73 Eksport przetworzonych danych analitycznych do pliku Excel

Źródło: opracowanie własne

Plik ten stanowi esencję przeprowadzonego procesu modelowania i zawiera zagregowane wyniki analizy regresji logistycznej wraz z przypisanymi tematami i liczebnością tokenów.

4.5 Budowa pulpitów menedżerskich

Zwieńczeniem prac implementacyjnych oraz finalnym produktem opracowanego narzędzia analitycznego jest utworzony w Tableau Desktop interaktywny zestaw pulpitów menedżerskich. Stanowi on warstwę prezentacyjną całego systemu, dedykowaną bezpośrednio dla zespołu decyzyjnego studia deweloperskiego. To właśnie na tym etapie sformalizowane wyniki modelu regresji logistycznej i analizy leksykalnej, zawarte w pliku wynikowym, ulegają transformacji w czytelną wiedzę pomagającą rozwiązać problem biznesowy studia FromSoftware.

Zgodnie z projektem pulpitów z rozdziału 3. pierwszy utworzony pulpit jest najważniejszym wynikiem pracy. Jego zadaniem jest natychmiastowo dać odpowiedź zespołowi analizującemu opinie graczy jakie są kluczowe czynniki sukcesu ich gier, a także jaka jest ich waga przy ogólnej opinii. Aby utworzyć pulpit widoku strategicznego w pierwszej kolejności kreowane są jego elementy składowe.

Górna część wizualizacji zawiera syntetyczne podsumowanie analizowanego zbioru danych, wyrażone w trzech kluczowych liczbach:

1. **Opinie:** Informuje o całkowitym wolumenie przetworzonych recenzji, co świadczy o skali badawczej i wiarygodności statystycznej próby. Tworzony przez zliczenie ilości rekordów w pliku `clean_df.xlsx` funkcją `CNT()`
2. **Wynik Steam:** Reprezentuje odsetek recenzji pozytywnych w całym zbiorze. Jest to bezpośredni miernik ogólnego zadowolenia klientów. Utworzenie go wymaga dwóch zmiennych pomocniczych. Pierwsza, przedstawiona na Rysunku 74, to „Boolean” zmieniającą zmienna „Recommendation” na cyfry 0 dla „false” i 1 dla „true” jak przedstawiono na Rysunku.

```
IF [recommendation] == TRUE THEN 1 ELSE 0 END
```

Rysunek 74 Pole obliczeniowe dla wartości binarnej
Źródło: opracowanie własne

1. Następnie tworzone jest kolejne pole obliczeniowe, nazwane „Steam%”, którego struktura przedstawiona na Rysunku 75 oblicza iloraz sumy zmiennej „Boolean” i jej liczebności obliczoną funkcją `CNT()` jak przedstawiono na Rysunku 75.

```
SUM([Boolean])/COUNT([Boolean])
```

Rysunek 75 Obliczanie procentowego wyniku Steam
Źródło: opracowanie własne

3. **Predyktory:** Wskazuje na liczbę unikalnych tokenów uznanych przez model, które mają istotny wpływ na klasyfikacje sentymentu. Tworzony przez zliczenie ilości rekordów w pliku `wynik.xlsx` funkcją `CNT()`

Wynikiem jest pokazany poniżej na Rysunku 76 panel wskaźników wydajności gotowy do implementacji

Predyktory	Wynik Steam	Opinie
8 343	90,03%	39 749

Rysunek 76 Panel wskaźników wydajności
Źródło: opracowanie własne

Kolejnym etapem tworzenia pierwszego pulpitu jest utworzenie paneli opartych o siłę oddziaływania czynnika obliczonego zgodnie ze wzorem określonym w rozdziale 3.1. W tym celu utworzono zmienną „Siła oddziaływania czynnika”, która liczona jest jako średnia ważona czyli iloraz sumy iloczynów wag wydźwiewu i ilości wystąpień do sumy wszystkich składowych czynnika. Formułę pola obliczeniowego dla siły oddziaływania czynnika przedstawia poniższy Rysunek 77.

$$\frac{\text{SUM}([\text{coefficient}] * [\text{count}])}{\text{SUM}(\text{count})}$$

Rysunek 77 Formuła siły oddziaływania czynnika

Źródło: opracowanie własne

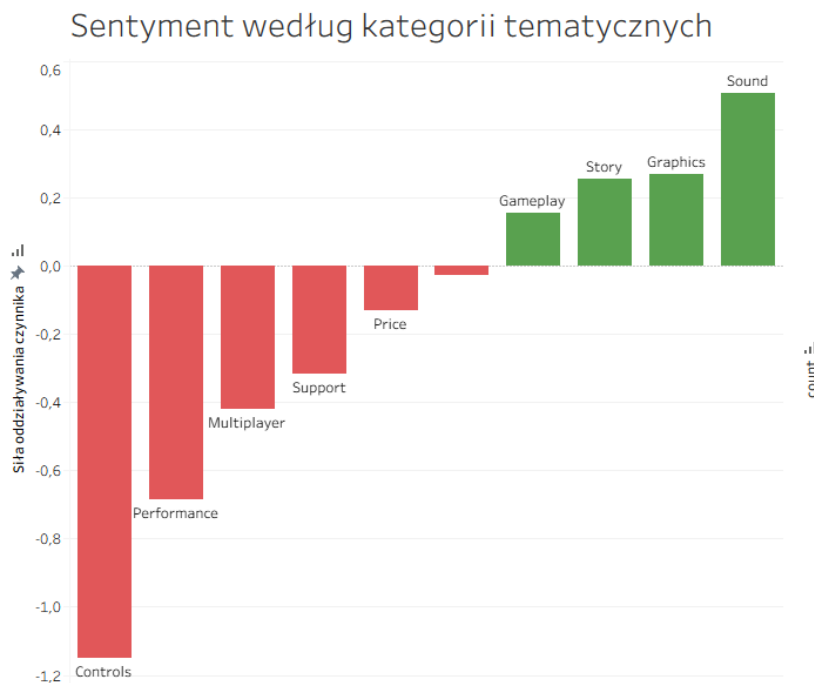
Tak obliczona zmienna jest następnie wykorzystywana do utworzenia dwóch wizualizacji, którymi są „Sentymet według kategorii tematycznych” i „Kategorie tematyczne w opiniach”. Do utworzenia ich została również stworzona dodatkowa zmienna, która porządkuje dane o czynnikach. Zaimportowane dane o czynnikach zawierają uproszczenie w kolumnie poświęconej czynnikom, które jest niedogodnością przy tworzeniu wizualizacji. Przypisane tematy były tylko do czynników zawarte w słowniku tematów. Efektem tego są tokeny bez przypisanego tematu, które Tableau Desktop definiuje jako „null”. Rozwiązaniem jest stworzenie zmiennej Czynniki_Clean ukazanej na Rysunku 78, która przyjmuje dla rekordów bez przypisanego tematu przyjmuje wartość „Ogólne”.

$$\text{ifnull}([\text{Czynniki}], \text{"Ogólne"})$$

Rysunek 78 Obsługa wartości pustych w kolumnie czynników

Źródło: opracowanie własne

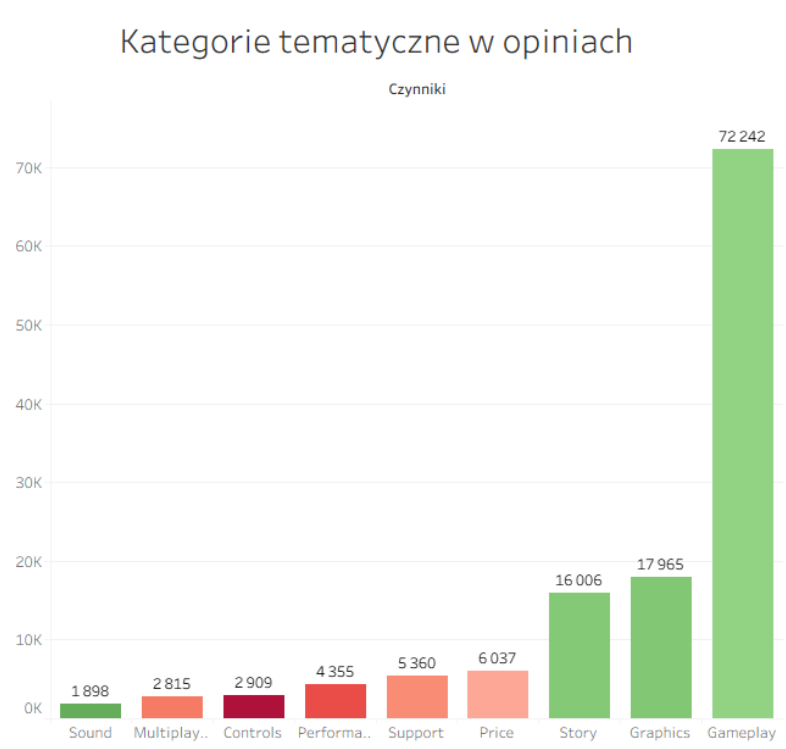
Tak przygotowane pola obliczeniowe posłużyły do stworzenia wizualizacji na rysunku 79, która stanowi bezpośrednią odpowiedź na pytanie o czynniki sukcesu i porażki. Wykres przedstawia średnią siłę oddziaływania poszczególnych kategorii na ostateczną ocenę gry. Oś pionowa reprezentuje siłę oddziaływania czynnika, gdzie wartości dodatnie oznaczają czynniki budujące sukces, a ujemne obszary problematyczne



Rysunek 79 Wykres sentymentu według kategorii tematycznych

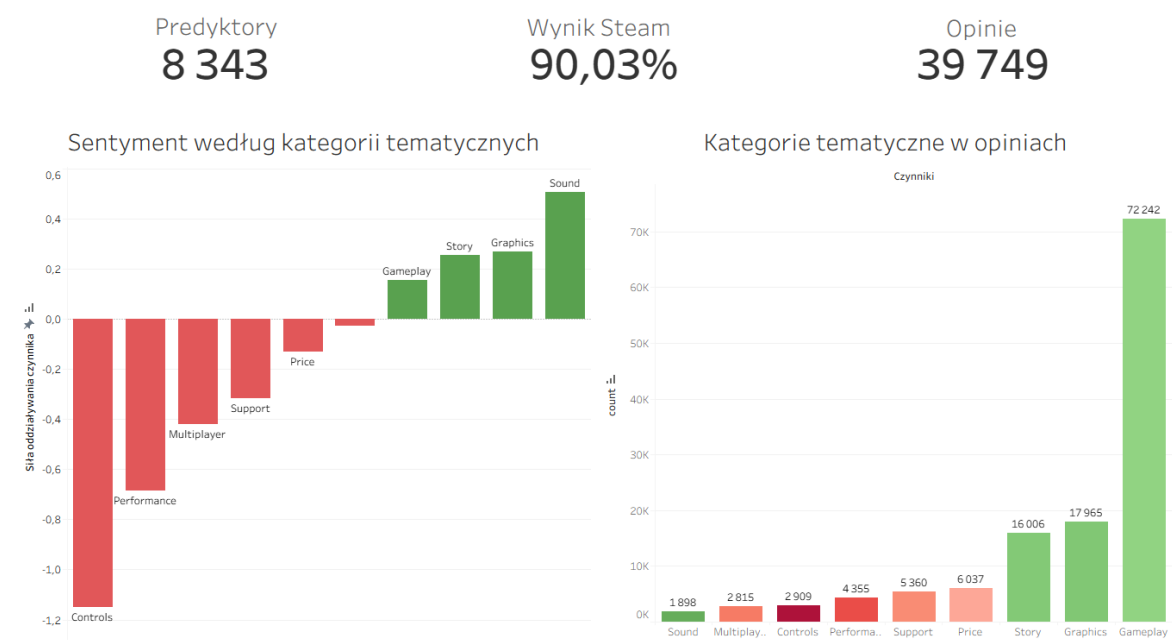
Źródło: opracowanie własne

Jednakże powyższy wykres byłby bezużyteczny bez dodania kontekstu, który przedstawia wizualizacja przedstawiająca ilość wspomnień danego czynnika w opiniach graczy. Wykres przedstawiono na Rysunku 80



Rysunek 80 Wykres liczebności kategorii tematycznych w opiniach
Źródło: opracowanie własne

Wszystkie utworzone wizualizacje są następnie agregowane jako jeden pulpit menedżerski przedstawiony na poniższym Rysunku 81.

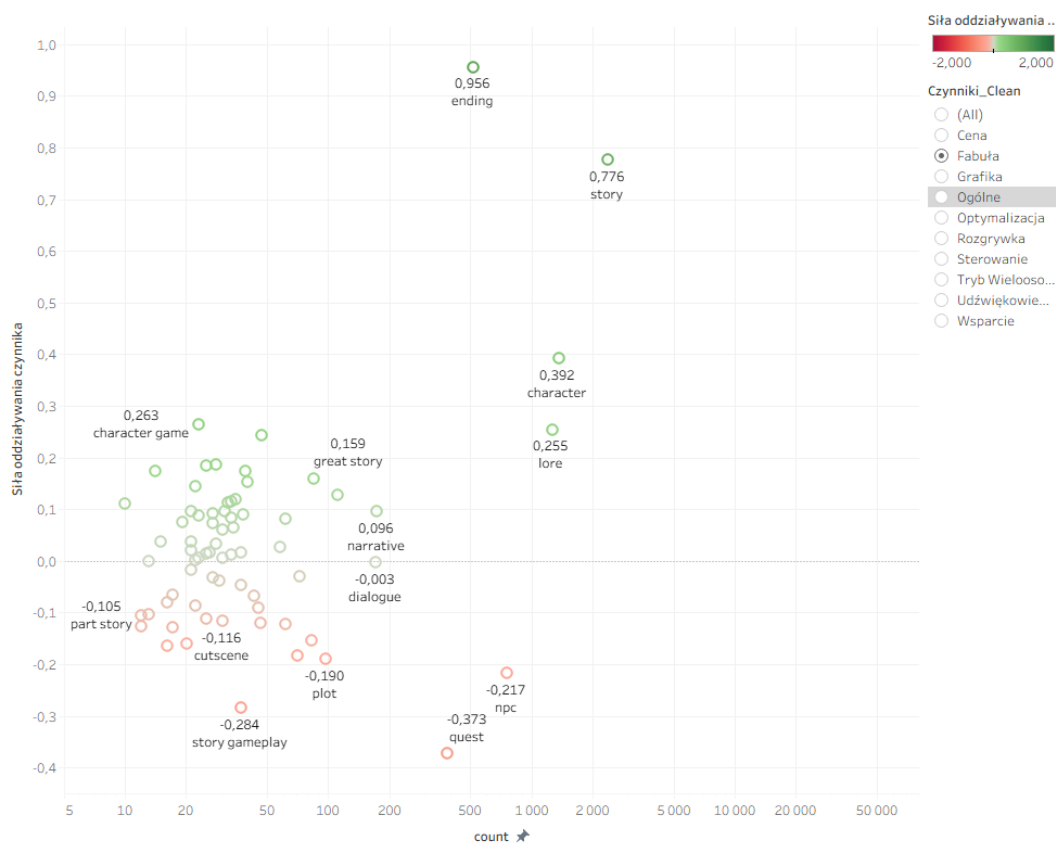


Rysunek 81 Strategiczny pulpit menedżerski
Źródło: opracowanie własne

Prezentowany pulpit został zaprojektowany w sposób hierarchiczny, umożliwiając odbiorcy przejście od ogólnych wskaźników efektywności do szczegółowej analizy czynnikowej. Jego strukturę można podzielić na trzy kluczowe sekcje, którymi są panel

wskaźników numerycznych oraz dwa komplementarne wykresy słupkowe definiujące charakterystykę opinii.

Uzupełnieniem ogólnego obrazu zaprezentowanego na głównym dashboardzie, jest matryca korelacji siły oddziaływania czynnik. Jest on dedykowany pogłębionej analizie leksykalnej. Wizualizacja ta realizuje funkcję drążenia danych, umożliwiając zespołowi analitycznemu przejście od ogólnych wskaźników sentymentu do szczegółowych przyczyn stojących za ocenami graczy. Celem utworzenia wizualizacji zestawiono ze sobą siłę oddziaływania czynnika na osi rzędu z ilością występowania tokenu powiązanego z czynnikiem na osi odcięcia. Czytelność pulpitu została amplifikowana zmianą skalowania osi odcięcia z liniowej na logarytmiczną przez co słowa rozkładają się bardziej równomiernie na przestrzeni analitycznej. Kolejnym krokiem było dodanie nazwy tokenu, częstości wystąpień pod przypisanymi im punktom i agregacji punktów kolorem definiowanym przed wydźwięk. Ostatnim elementem pulpitu został filtr czynników, który pozwala zmieniać widok pomiędzy kategoriami tematycznymi tokenów przedstawianych na wizualizacji. Ostateczny wygląd pulpitu przedstawiono na Rysunku 82.



Rysunek 82 Matryca korelacji siły oddziaływania i liczebności

Źródło: opracowanie własne

Zrealizowany etap budowy pulpitów menedżerskich w środowisku Tableau Desktop stanowi zwieńczenie prac implementacyjnych, integrując wyniki analizy danych z warstwą prezentacyjną, niezbędną dla zespołu decyzyjnego. Zaprojektowany system wizualizacji, składający się z pulpitu głównego oraz narzędzia do pogłębionej analizy leksykalnej, skutecznie transformuje sformalizowane wyniki modelu regresji logistycznej w użyteczną wiedzę biznesową.

5. Ewaluacja

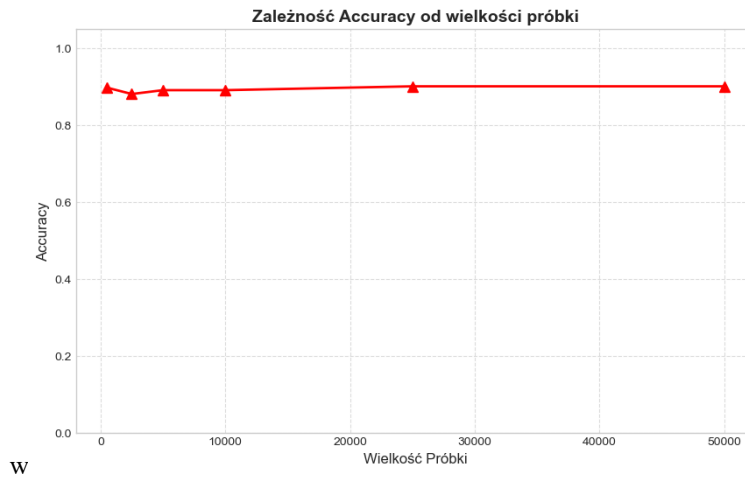
5.1 Analiza wyników modelu

Kluczowym aspektem weryfikacji przydatności zaprojektowanego narzędzia jest ocena jego stabilności i skuteczności w zależności od ilości dostarczonych danych. W projektach z zakresu przetwarzania języka naturalnego panuje powszechne przekonanie, że jakość modelu jest wprost proporcjonalna do wielkości zbioru uczącego (Kaplan, et al., 2020). Aby zweryfikować tę hipotezę w kontekście specyficznego żargonu graczy gier soulslike, przeprowadzono eksperyment polegający na iteracyjnym trenowaniu modelu regresji logistycznej na podzbiorach danych o rosnącej liczebności: 500, 2 500, 5000, 10 000, 25 000 oraz 50 000 recenzji. Dla każdej iteracji zachowano stałe parametry, aby wyniki były porównywalne. Wyniki eksperymentu, obejmujące miary Accuracy, Kappa Cohena oraz AUC zestawiono w Tabeli 10

Tabela 10 Mierniki jakości w zależności od wielkości próbki danych
Źródło: opracowanie własne

Próbka	Accuracy	Kappa	AUC
500	0,89	-0,04	0,7
2500	0,88	0,18	0,76
5000	0,89	0,3	0,84
10000	0,89	0,41	0,87
25000	0,9	0,5	0,9
50000	0,9	0,52	0,9

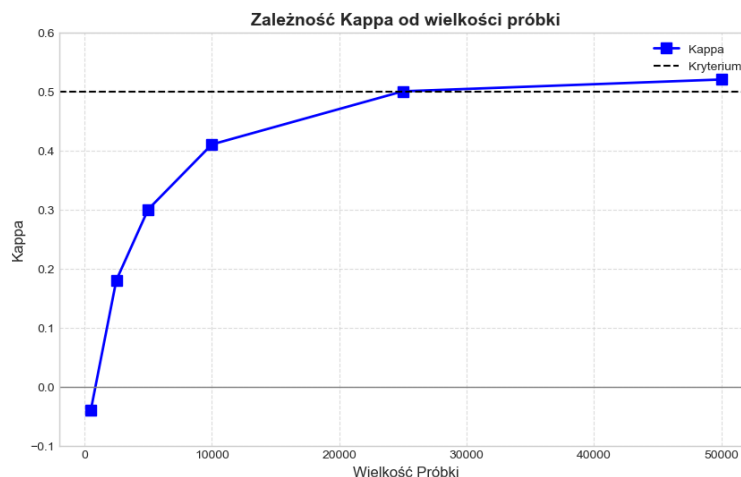
Poniżej przedstawiono szczegółową interpretację kształtowania się poszczególnych metryk z pomocą przejrzystych wizualizacji. Wykres dokładności w relacji do wielkości próby wykazuje zjawisko, które w literaturze przedmiotu określane jest mianem paradoksu dokładności (Kuhn & Johnson, 2013). Zależność tę przedstawia Rysunek 83. Już przy najmniejszej próbie (N=500) model osiąga wynik bliski 89% co teoretycznie mogłoby sugerować wysoką skuteczność. Jest to jednak wynik mylący, wynikający bezpośrednio z silnego niezbalansowania zbioru danych, w którym opinie pozytywne stanowią około 90% wszystkich rekordów. Model przy małej ilości danych "uczy się" jedynie statystyki występowania klas, klasyfikując niemal wszystko jako "pozytywne", co daje mu wysoką trafność ogólną, ale zerową użyteczność analityczną.



Rysunek 83 Wykres zależności Accuracy od wielkości próbki
Źródło: opracowanie własne

Zauważalny spadek dokładności przy próbie $N=2500$ (do poziomu 0,88) jest w rzeczywistości sygnałem pozytywnym. Świadczy o tym, że model zaczyna "oduczać się" ślepego faworyzowania klasy większościowej i próbuje identyfikować rzeczywiste wzorce, nawet jeśli początkowo popełnia przy tym więcej błędów. Dopiero przy dużych wolumenach ($N>25000$) dokładność ponownie rośnie, tym razem podparta rzeczywistym zrozumieniem sentymentu.

Analiza statystyki Kappa Cohena, będącej bardziej rzetelną miarą dla niebalansowanych zbiorów, weryfikuje użyteczność modelu przy małych próbkach. Przedstawiony wykres na Rysunku 84 pokazuje, że dla małej próbki ($N=500$) wartość Kappa wynosi -0.04, co oznacza poziom klasyfikacji gorszy od losowego rzutu monetą. Model w takim stanie jest bezużyteczny. Wraz ze wzrostem wolumenu danych obserwujemy liniowy wzrost tej metryki, jednak kluczowe bariery są przekraczane powoli.

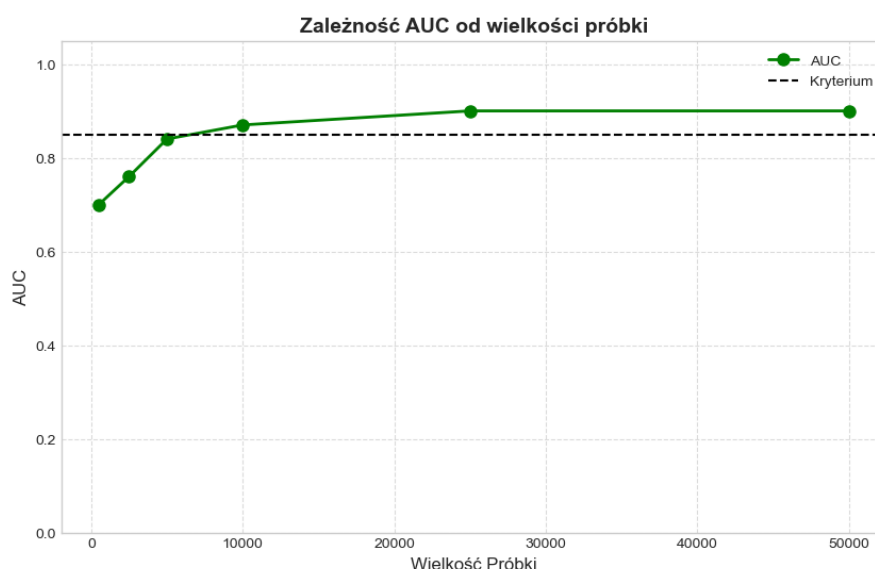


Rysunek 84 Zależność współczynnika Kappa od wielkości próbki
Źródło: opracowanie własne

Krytyczny punkt zwrotny następuje przy próbie 25 000, gdzie Kappa osiąga wartość 0,5. Jest to próg, który w badaniach nad sentymentem (szczególnie w trudnej domenie slangu graczy) uznawany jest za granicę akceptowalności modelu do zastosowań

biznesowych. Dalszy wzrost do 0,52 przy $N=50\ 000$ potwierdza, że model wymaga dziesiątek tysięcy próbek do prawidłowego nauczenia się wzorców zaszytych w opiniach.

Krzywa AUC na Rysunku 85 wykazuje wzrost w początkowej fazie eksperymentu, startując od poziomu 0,7 przy $N=500$. Wartość ta stosunkowo szybko rośnie, osiągając 0,84 już przy 5 000 próbek, co sugeruje, że model nabywa umiejętność rankingu próbek. Mimo wysokich wskazań AUC, należy je interpretować w tandemie z współczynnikiem Kappy. Wysokie AUC przy niskiej Kappie (jak w przypadku $N<5000$) sugeruje, że model dobrze separuje klasy, ale ma źle dobrany próg odcięcia.



Rysunek 85 Wykres zależności AUC od wielkości próbki

Źródło: opracowanie własne

Pełna użyteczność analityczna, gdzie wysokie AUC (0,87) idzie w parze z akceptowalną precyzją klasy mniejszościowej, zaczyna się dopiero od progu 10 000 recenzji. Finalne spłaszczenie wykresu na poziomie 0,9 dla prób 25k i 50k wskazuje, że dalsze dokładanie danych przynosi już malejące przychody w zakresie zdolności dyskryminacyjnej modelu.

Podsumowując przeprowadzoną analizę, można sformułować jednoznaczną rekomendację dotyczącą wymagań danych. Minimalną wielkością próbki, gwarantującą podstawową stabilność i wiarygodność biznesową modelu, jest zbiór liczący 25000 recenzji. Przy tej wartości model osiąga akceptowalne wskaźniki (Kappa 0,50, AUC 0,90), co pozwala na precyzyjną identyfikację niuansów językowych. Poniżej tej wartości model, mimo myląco wysokiej miary Accuracy, wykazuje tendencję do nadmiernego upraszczania rzeczywistości, co dyskwalifikuje go jako narzędzie wspomagania decyzji.

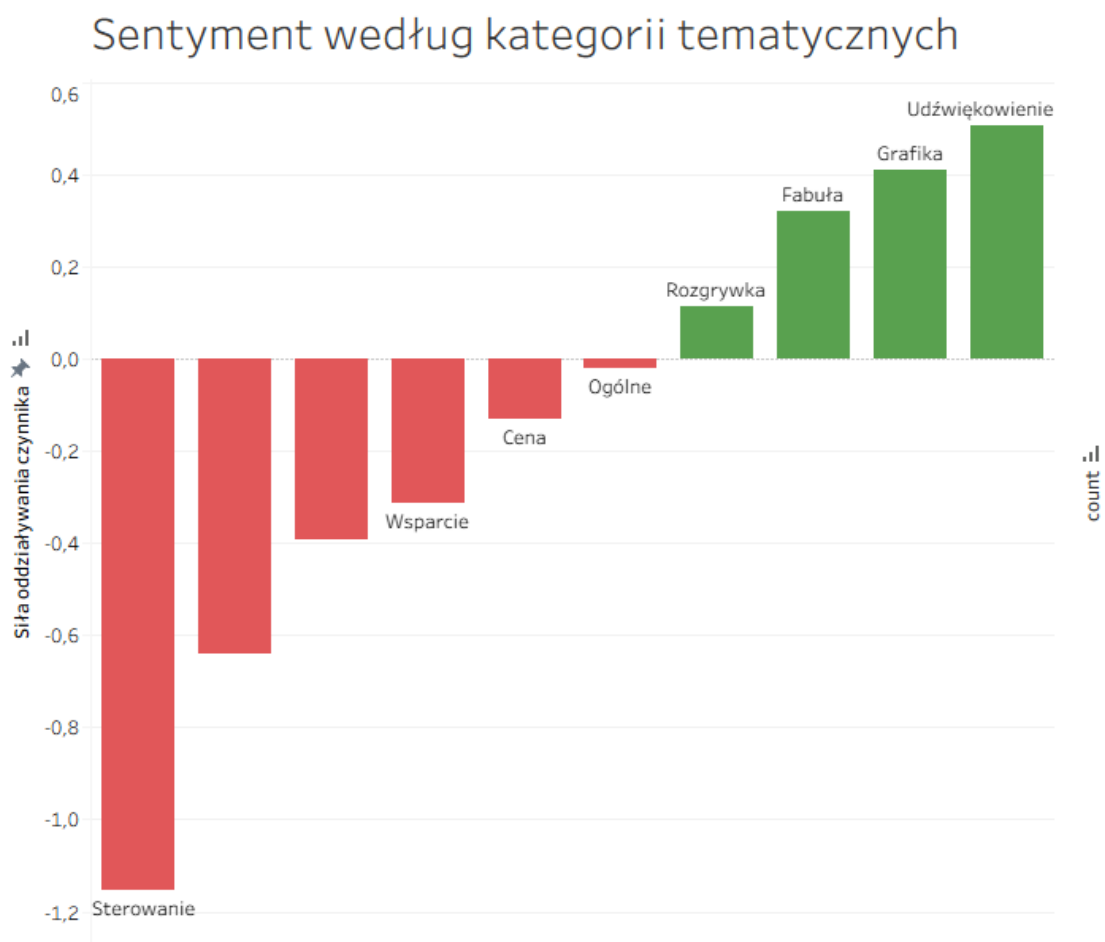
Eksperyment dowiódł, że przy zapewnieniu odpowiedniego wolumenu danych, algorytm regresji logistycznej skutecznie przezwycięża problem niezbalansowania klas i potrafi wyekstrahować sentyment ukryty w specyficznym żargonie graczy, przechodząc drogę od statystycznego zgadywania do rzeczywistego "rozumienia" kontekstu.

5.2 Wyniki analizy danych przykładowych

W niniejszej pracy analiza została przeprowadzona dla danych, które pobrane zostały z platformy Steam 24 listopada 2025 roku. Podstawą do wyciągnięcia kluczowych wniosków biznesowych jest zestaw pulpitów menedżerskich będących wynikiem działania narzędzia. Poniższy proces wyciągania wniosków z wizualizacji jest jednocześnie realizowany zgodnie z zakładanym podejściem do analizy kluczowych czynników sukcesu,

który przewiduje zebranie informacji ogólnych wykorzystując pierwszy pulpit, pogłębienie analizy o najważniejsze składowe czynniki dzięki drugiemu pulpitowi, a następnie ogólna analiza wszystkich składowych danego czynnika.

Proces rozpoczęto od strategicznego pulpitu menedżerskiego przedstawionego na Rysunku 86, który zestawia ze sobą siłę oddziaływania sentymentu z częstotliwością występowania danej kategorii w opiniach graczy. Takie podejście pozwoliło na uniknięcie błędów poznawczych wynikających z analizy samych wag sentymentu.

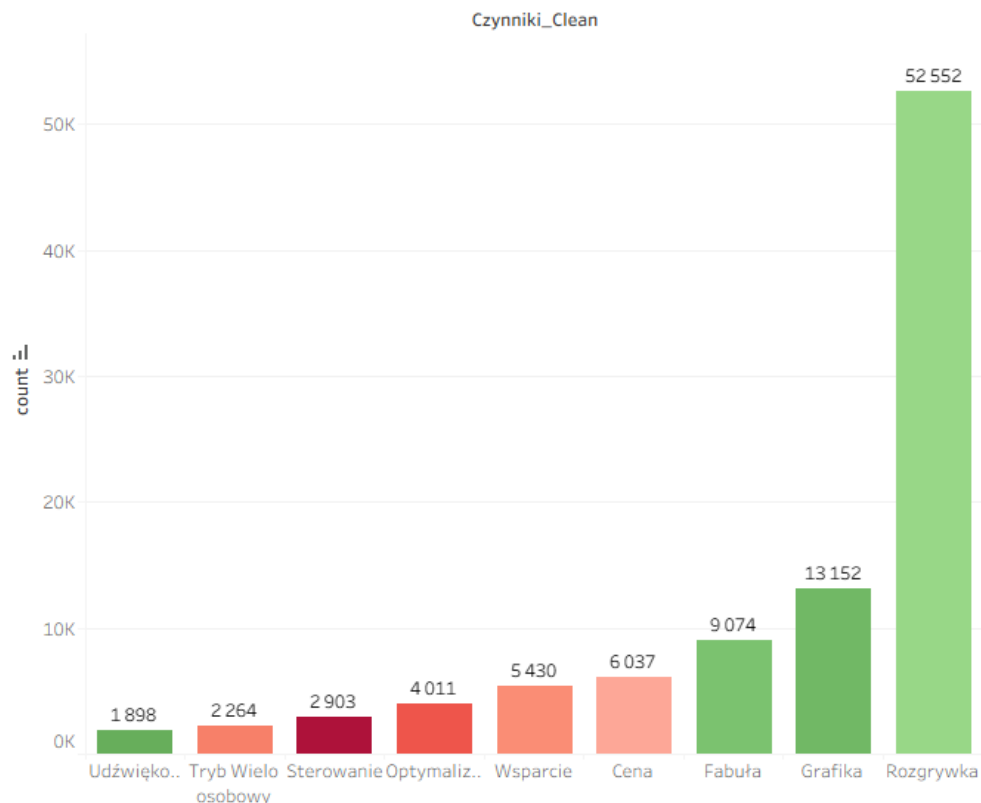


Rysunek 86 Analiza sentymentu dla danych przykładowych

Źródło: opracowanie własne

Wstępna analiza lewego wykresu, obrazującego wyłącznie siłę oddziaływania, mogłaby sugerować, że najważniejszym atutem gier studia FromSoftware jest kategoria „Udźwiękowanie”, która osiągnęła najwyższy pozytywny wskaźnik siły oddziaływania. Analogicznie, za najbardziej krytyczny problem można by uznać „Sterowanie”, które charakteryzuje się skrajnie negatywnym wynikiem. Jednakże, zestawienie tych danych z prawym wykresem liczebności przedstawionym na Rysunku 87 diametralnie zmienia interpretację wyników.

Kategorie tematyczne w opiniach



Rysunek 87 Analiza liczebności dla danych przykładowych

Źródło: opracowanie własne

Mimo że „Udźwiękowanie” jest oceniane niezwykle pozytywnie, pojawia się ono w zaledwie 1 898 opiniach. Dla porównania, kategoria „Rozgrywka”, choć posiada niższy jednostkowy wskaźnik sentymentu, została poruszona w aż 52 552 wzmiankach. Wskazuje to jednoznacznie, że to właśnie rozgrywka, a nie muzyka czy efekty dźwiękowe, jest absolutnym fundamentem doświadczenia gracza i głównym motorem napędowym sukcesu produkcji. Podobna relacja zachodzi w przypadku „Sterowania”. Choć jest ono skrajnie krytykowane, liczba skarg jest relatywnie niewielka w skali całego zbioru danych, co sugeruje, że jest to problem dotkliwy, ale niszowy, a nie systemowy błąd dyskwalifikujący grę w oczach większości odbiorców.

Analiza obu wykresów na danych podstawowych pozwala zdefiniować rzeczywiste filary sukcesu gier soulslike. Są nimi trzy kategorie, które łączą pozytywny sentyment z bardzo wysokim wolumenem dyskusji:

- **Rozgrywka** - dominujący temat dyskusji (52552 wzmianek), definiujący gatunek. Siła oddziaływania czynnika wyniosła 0,114 co określa małą, dodatnią korelację w stosunku do oceny recenzenta. Taki wynik implikuje jak polaryzującym tematem jest specyficzna rozgrywka w gatunku soulslike. Ostateczny wynik powyżej 0 jest jednak sygnałem, że większa część graczy docenia podejście prezentowane przez FromSoftware.
- **Grafika** - 17 965 wzmianek o silnym pozytywnym wydźwięku jednoznacznie pokazuje docenienie warstwy wizualnej produkcji przez graczy

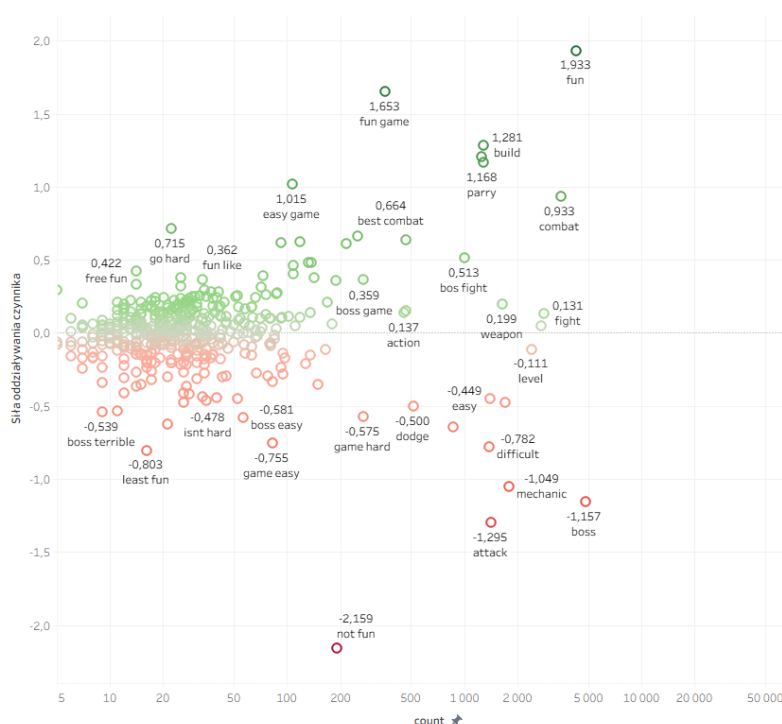
- **Fabula** - 16 006 wzmianek, potwierdzających wagę unikalnej narracji środowiskowej studia.

Weryfikacja obszarów problematycznych również wymagała uwzględnienia skali. Choć sterowanie ma najgorszy wynik punktowy, to gracze znacznie częściej narzekają na aspekty użytkowe i ekonomiczne. Rzeczywistymi barierami w odbiorze gier, zidentyfikowanymi na podstawie połączenia ujemnego sentymentu i istotnej liczby wzmianek, są:

- **Cena** - 6 037 wzmianek przy wydzźwięku na poziomie (-0,132), co sugeruje niejednoznaczne podejście do ceny produktów wskazujące lekko negatywny sentyment.
- **Wsparcie** - 5 360 wzmianek o znacznie negatywnym wydzźwięku na poziomie (-0,312), odnoszących się do błędów i jakości obsługi technicznej.

W celu głębszego zrozumienia zidentyfikowanych trendów, przeprowadzono analizę szczegółową z wykorzystaniem macierzy korelacji. Pozwoliła ona na dekompozycję ogólnych kategorii na pojedyncze tokeny i zbadanie ich indywidualnego wpływu na sentyment.

W kategorii „Rozgrywka”, będącej najczęściej dyskutowanym aspektem, zauważalna jest na Rysunku 88 wyraźna polaryzacja, która doskonale oddaje specyfikę gatunku soulslike.



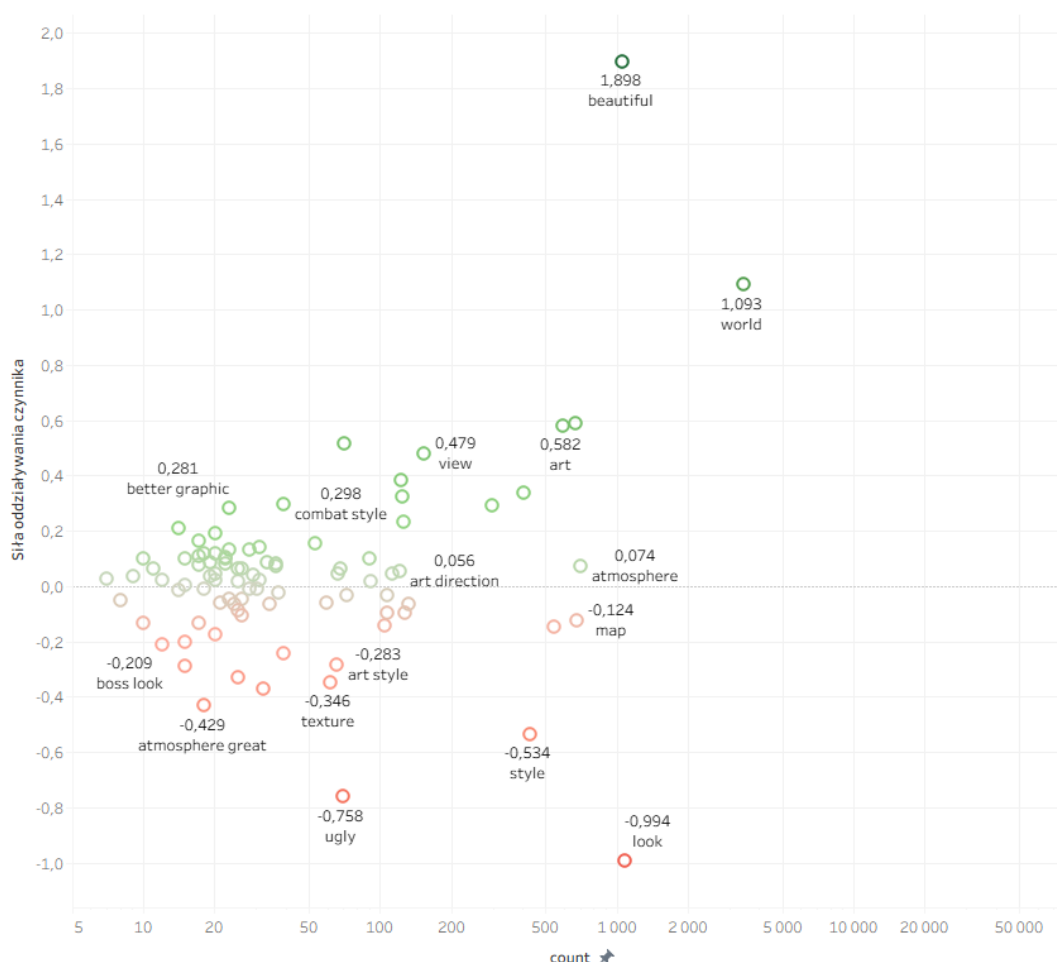
Rysunek 88 Szczegółowa analiza tokenów dla kategorii Rozgrywka

Źródło: opracowanie własne

Najsilniejszymi pozytywnymi tokenami odnoszącymi się do satysfakcji z rozgrywki były „fun”, „build”, „combat” czy „parry”. Potwierdza to, że w ogóle rozgrywki system walki i możliwość tworzenia własnej postaci są filarami sukcesu gier FromSoftware. Z drugiej jednak strony można zauważyć duży negatywny impakt tokenów „boss”, „mechanic”, „attack” czy „difficult”, które mogą sugerować wysoki odsetek graczy

frustrujących się grą w tak trudne tytuły. Mimo że wysoki poziom trudności jest przewidywaną cechą gatunku to nadal większość graczy nie jest w stanie tego zaakceptować i wpływa to na negatywne opinie co może sugerować potrzebę dodania do gier FromSoftware dodatkowego trybu upraszczającego rozgrywkę. Byłoby to jednak niespójne z filozofią studia. Najbardziej negatywnym sygnałem jest fraza „not fun”, wskazująca na momenty kiedy frustracja graczy przekracza granicę satysfakcji. Porównując obraz studia i jego dotychczasowy dobytek do uzyskanych wyników możemy wnioskować o celowaniu do niszy, która jak żadna inna docenia unikalne podejście projektowe przy tworzeniu systemów rozgrywki. Wiele graczy nie jest w stanie pokonać tej bariery wejścia co skutkuje negatywnymi recenzjami.

W kategorii „Grafika” ukazanej na Rysunku 89, zauważamy dominację pozytywnego odzewu na warstwę wizualną gier.

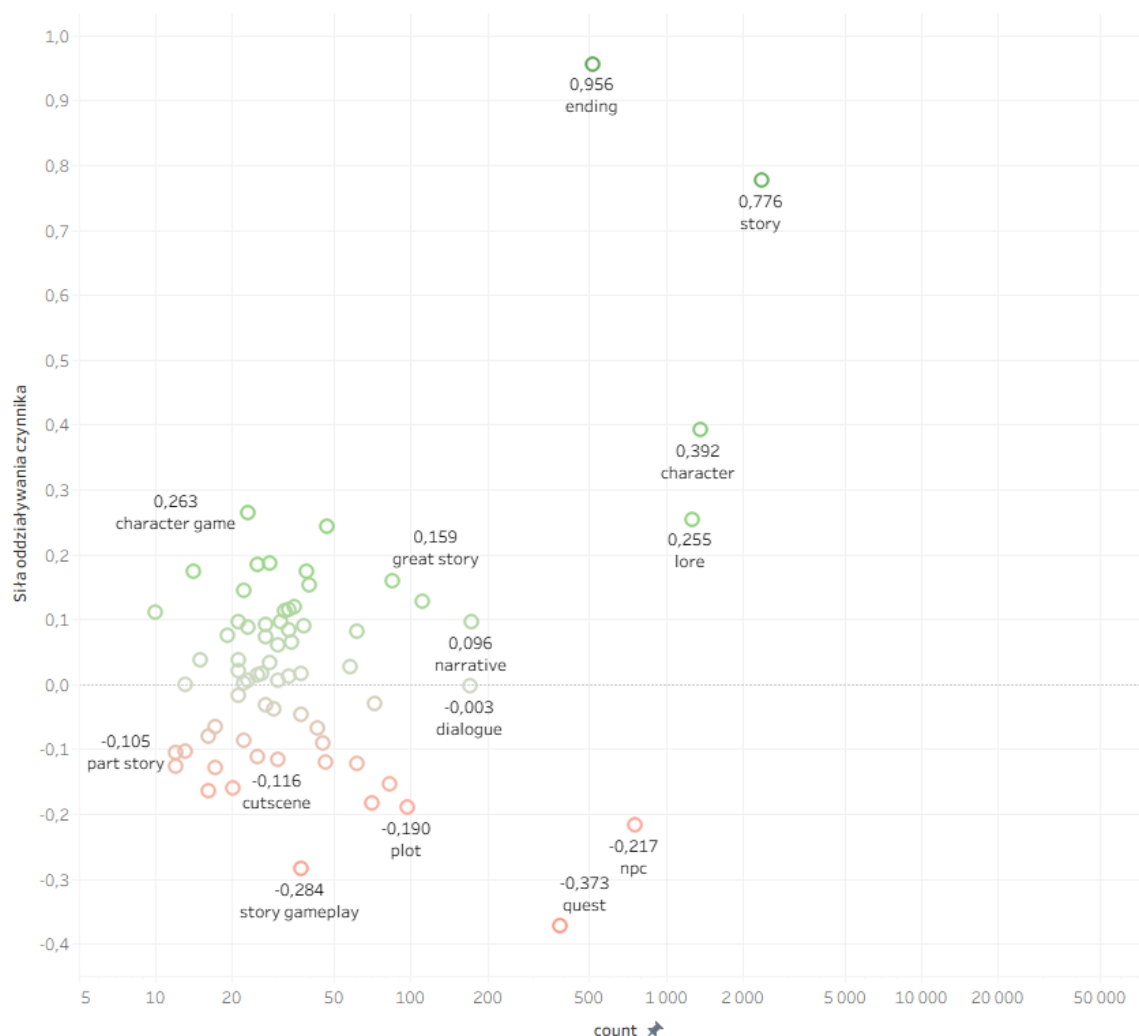


Rysunek 89 Szczegółowa analiza tokenów dla kategorii Grafika

Źródło: opracowanie własne

Token „Beautiful” jest liderem tej kategorii przy wydźwięku równym 1,898. Z poszczególnych elementów gracze wspominają o „world”, „view” czy „art.” doceniając budowę świata przedstawionego, krajobrazy w nim zawarte i odczucie jakby gra była sztuką samą w sobie. Jeśli chodzi o negatywne składowe to nie przekraczają one wyniku (-1) i wskazują takie aspekty jak „ugly”, „look” czy „style”, co pokazuje grupę graczy, która nie została oczarowana wizualiami produktów. Analizując ogólny wydźwięk gracze niezadowoleni są zdecydowaną mniejszością.

Kategoria Fabuła przedstawiona na Rysunku 90, wykazuje spójny i pozytywny charakter.

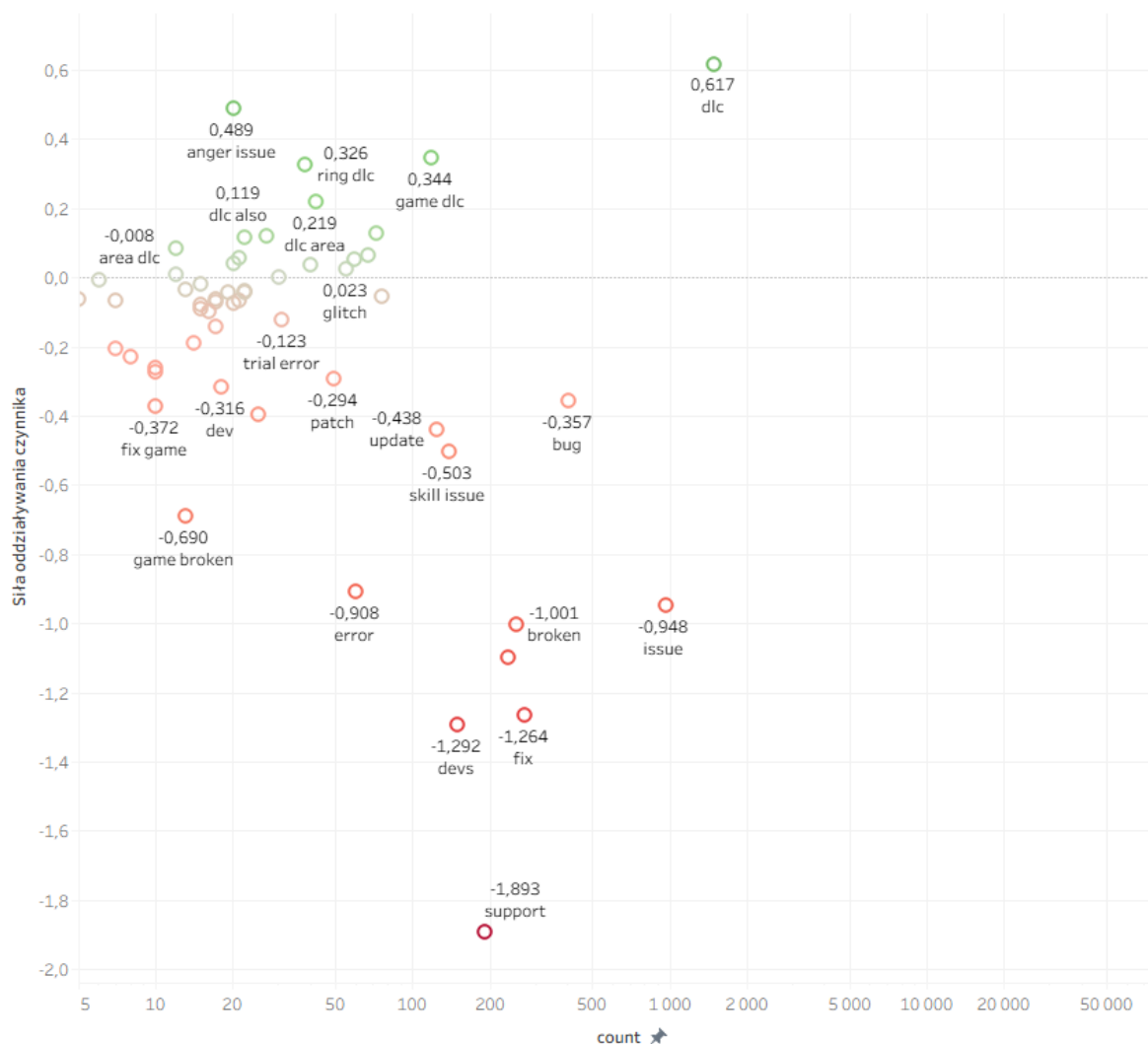


Rysunek 90 Szczegółowa analiza tokenów dla kategorii Fabuła

Źródło: opracowanie własne

Tokenem o najwyższej pozytywnej wadze jest „ending”, co świadczy o tym, że finał historii jest bardzo wysoko oceniany przez graczy. Kolejnymi zachwalanymi elementami gry są „story”, „character” i „lore” co wskazuje na zadowolenie całokształtem przedstawionej historii i postaci w niej zawartych. Jedynymi zauważalnymi problemami z historią są „quest” i „npc” co w połączeniu z wiedzą o grach FromSoftware, może świadczyć o niejasności zadań wykonywanych w grze i postaci niezależnych z nimi powiązanych.

Następnie analizowane były aspekty oceniane w grach studia negatywnie zaczynając od Wsparcia przedstawionym na Rysunku 91.

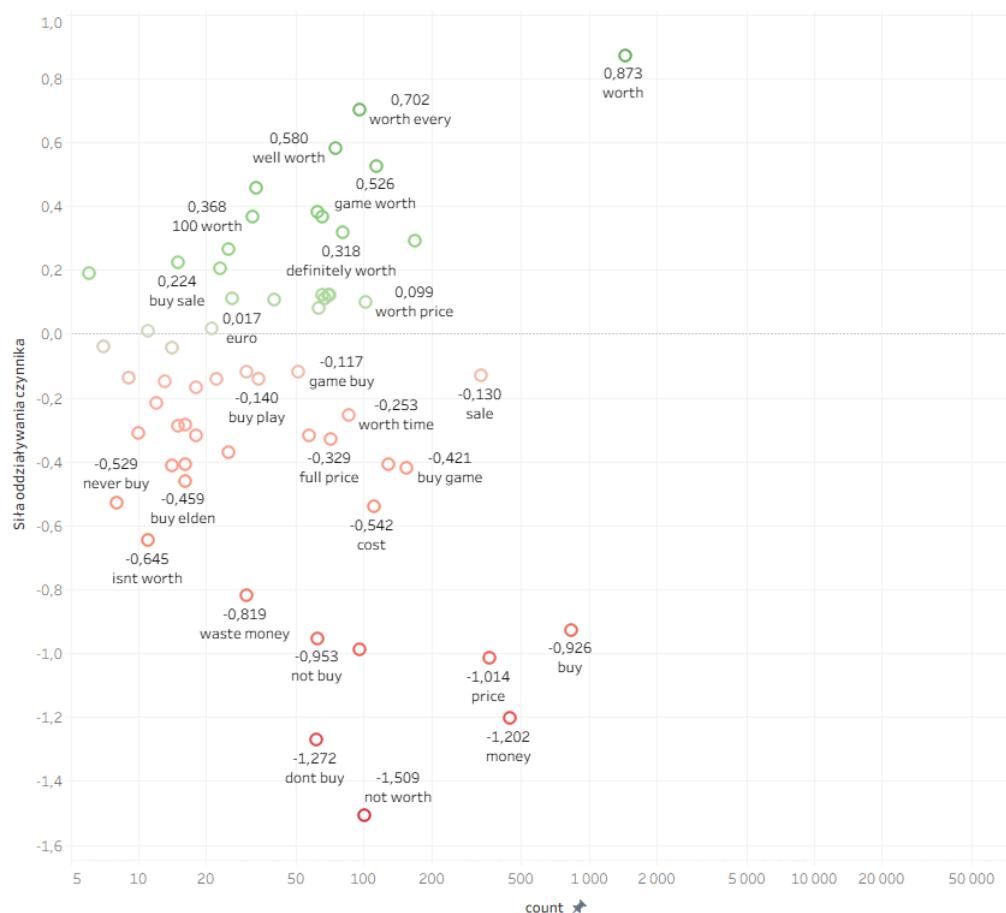


Rysunek 91 Szczegółowa analiza tokenów dla kategorii Wsparcie

Źródło: opracowanie własne

Zdecydowanie większość terminów z negatywnym wydźwiękiem są powiązane z kwestiami technicznymi. Tokeny „error”, „fix”, „devs” czy „broken” mogą sugerować małe zaangażowanie zespołu deweloperskiego w naprawianiu problemów dotyczących graczy. Najsilniej negatywnym słowem jest „support” co może sugerować niski poziom wsparcia, ale wymagałoby dodatkowej analizy kontekstu, przez wieloznaczeniowość słowa „support”. Ciekawym wnioskiem płynącym z analizy tego aspektu jest generalne zadowolenie z zawartości dodatkowej („dlc”) przez społeczność graczy. Jest to token o najwyższym wydźwięku w tej kategorii, a także znajduje się w większości bigramów takich jak „game dlc”, „dlc area” czy „ring dlc” co może wskazywać na wysoki poziom dodatkowej treści tworzonej na potrzeby już wydanych gier.

Ostatnim elementem analizowanym z pomocą matrycy korelacji siły oddziaływania czynników jest Cena ukazanej na Rysunku 92.



Rysunek 92 Szczegółowa analiza tokenów dla kategorii Cena

Źródło: opracowanie własne

Przez subiektywny obraz wartości pieniądza w społeczeństwie trudno jest wyciągnąć jednoznaczne wnioski z analizy tekstu. Tokeny „worth” i „not worth” są dwoma najbardziej znaczącymi pod względem wydźwięku w obrazie tego czynnika. Negatywny wydźwięk może wynikać z wysokiej ceny produktów, która na dzień 12 grudnia 2025 waha się od 149 złotych do 249 złotych. Zauważalny jest także token „buy elden”, który może sugerować supremację najnowszego tytułu FromSoftware „Elden Ring” ponad starszymi tytułami, stąd sugestia recenzujących do polecania nowszego produktu.

Podsumowując, analiza szczegółowa potwierdziła, że sukces gier FromSoftware opiera się na aspektach projektowych takich jak wizja artystyczna, satysfakcja z walki, głębia świata, podczas gdy główne zagrożenia dla odbioru produktów leżą w warstwie technicznej oraz barierze wejścia przez trudność przeciwników czy niejasne zadania generujące frustrację interpretowaną przez model jako sentyment negatywny.

5.3 Ocena w kontekście rozwiązania problemu biznesowego

Głównym problemem biznesowym, który stał się podstawą niniejszej pracy, był brak zautomatyzowanych narzędzi analitycznych zdolnych do poprawnej interpretacji opinii w niszowych i polaryzujących gatunkach gier, takich jak „soulslike”. Cel pracy, zdefiniowany jako zaprojektowanie narzędzia pozwalającego na identyfikację kluczowych

czynników wpływających na odbiór gier studia FromSoftware, został osiągnięty. Przeprowadzona ewaluacja potwierdza skuteczność rozwiązania w fundamentalnych dla problemu biznesowego obszarach:

1. Przełamanie bariery interpretacyjnej specyficznego języka graczy

Istotą problemu biznesowego była niezdolność standardowych narzędzi do zrozumienia specyfiki gatunku, gdzie trudność jest zaletą, a nie wadą. Zaprojektowane narzędzie skutecznie rozwiązało ten problem poprzez zastosowanie dedykowanego modelu. Model poprawnie interpretuje specyficzny żargon, przypisując dodatnie wagi tokenom takim jak „challenge”. „dying” czy „hard”.

2. Precyzyjna identyfikacja i kwantyfikacja czynników sukcesu

Zgodnie z tematem pracy, narzędzie miało służyć identyfikacji kluczowych czynników sukcesu. Analiza wyników potwierdza, że system potrafi precyzyjnie wyodrębnić aspekty decydujące o pozytywnym odbiorze produkcji studia. Zastosowana metryka siły oddziaływania czynnika pozwoliła na wyłonienie hierarchii atutów gier. Dla próbki danych (N=50000) narzędzie wskazało, że kluczowymi czynnikami sukcesu budującymi najwyższy sentyment są aspekty immersyjne i artystyczne - „Udźwiękowanie”, „Grafika” oraz „Fabuła”, natomiast najczęściej dyskutowanym fundamentem sukcesu jest „Rozgrywka”. Dodatkowo wprowadzone drażnienie danych pogłębia możliwość analizy danych wyjściowych modelu o identyfikację najważniejszych składowych czynników.

Podsumowując, narzędzie w pełni realizuje cel pracy, dostarczając rozwiązania dedykowanego dla specyfiki gier FromSoftware. Skutecznie eliminuje lukę analityczną, pozwalając na precyzyjne zidentyfikowanie tego, co w opinii graczy stanowi o sukcesie analizowanych tytułów.

5.4 Możliwość wdrożenia

Analiza techniczna zaprojektowanego narzędzia wskazuje na wysoką wykonalność wdrożenia oraz niską barierę wejścia dla studia deweloperskiego. System charakteryzuje się architekturą modułową, integrującą warstwę przetwarzania danych opartą na języku Python z warstwą wizualizacji w środowisku Tableau Desktop. Taka konstrukcja sprawia, że implementacja w środowisku profesjonalnym nie wymaga skomplikowanej integracji z wewnętrznymi systemami firmy, gdyż narzędzie operuje na danych zewnętrznych pozyskiwanych z platformy Steam. Z perspektywy infrastrukturalnej, rozwiązanie nie generuje wysokiego zapotrzebowania na moc obliczeniową.

Istotną przewagą ekonomiczną projektu nad gotowymi rozwiązaniami rynkowymi jest całkowity brak kosztów pozyskania danych. W przeciwieństwie do platform social listeningowych, które wymagają kosztownych subskrypcji, zaprojektowane narzędzie korzysta z publicznie dostępnych recenzji na platformie Steam, nie wymagając zakupu zewnętrznych baz ani dostępu do płatnych API. Utrzymanie systemu ogranicza się do cyklicznego uruchamiania skryptów aktualizujących oraz odświeżania źródła danych, co jest procesem wysoce zautomatyzowanym i niewymagającym znacznych nakładów pracy ludzkiej. W rezultacie studio otrzymuje rozwiązanie o wysokim potencjale zwrotu z inwestycji, które przy minimalnych kosztach operacyjnych rozwiązuje specyficzny problem biznesowy, dostarczając analiz dostosowanych precyzyjnie do unikalnej specyfiki gier typu soulslike.

5.5 Perspektywa rozszerzenia narzędzia

Analiza obecnej architektury narzędzia wskazuje na istotny obszar potencjalnego rozwoju, jakim jest transformacja modelu przetwarzania danych z trybu statycznego na dynamiczną analizę przyrostową. W obecnym kształcie narzędzie funkcjonuje jako system

statyczny. Proces akwizycji danych, opisany w rozdziale 4, opiera się na jednorazowym pobraniu określonej liczby recenzji i zapisaniu ich do pliku wynikowego. Każde kolejne uruchomienie analizy wymaga ponownego pobrania całego zbioru danych lub tworzy odseparowaną „migawkę” stanu opinii na dany dzień. Takie podejście, choć wystarczające do analizy historycznej, nie pozwala na ciągły monitoring zmian nastrojów graczy. Rozszerzenie narzędzia polegałoby na implementacji mechanizmu przyrostowego. W tym modelu system pobierałby pełną historię opinii tylko podczas pierwszego uruchomienia. Następnie przy każdym kolejnym cyklu, przykładowo codziennie o godzinie 24:00 skrypt sprawdzałby datę ostatniego wpisu w bazie danych, a zautomatyzowany scraper dodawałby do listy wyłącznie te opinie, które pojawiły się na platformie Steam po tej dacie. Warunkiem koniecznym dla wdrożenia pobierania przyrostowego jest zmiana sposobu magazynowania danych. Obecnie wykorzystywane pliki CSV i Excel musiałyby zostać zastąpione systemem zarządzania bazą danych SQL.

Równoległym i równie istotnym kierunkiem rozwoju jest zwiększenie wolumenu gromadzonych danych, co bezpośrednio przełożyłoby się na precyzję analityczną. Jak wykazała przeprowadzona ewaluacja, stabilność modelu regresji logistycznej osiągnięta jest przy próbie rzędu 25 000 recenzji. Obecne ograniczenia w liczbie pobranych opinii wymusiły agregację danych ze wszystkich gier do jednego zbioru treningowego, aby spełnić te wymogi statystyczne. Implementacja mechanizmu ciągłego pobierania danych pozwoliłaby z czasem na przekroczenie progu minimalnej liczebności próby dla każdej gry z osobna. To z kolei umożliwiłoby odejście od generalizacji na rzecz wytrenowania dedykowanych modeli dla każdego tytułu indywidualnie. W rezultacie możliwe stałoby się wyodrębnienie unikalnych czynników sukcesu dla poszczególnych produkcji, takich jak Sekiro czy Elden Ring, zamiast jednego zestawu cech dla całego studia. Takie podejście znacząco spotęgowałoby potencjał analityczny narzędzia, umożliwiając przeprowadzanie precyzyjnej obserwacji ewolucji oczekiwań graczy pomiędzy kolejnymi premierami.

Zakończenie

W ramach niniejszej pracy w pełni zrealizowano założony cel badawczy, opracowując autorskie narzędzie analityczne do identyfikacji kluczowych czynników sukcesu gier studia FromSoftware. Dzięki zastosowaniu technik eksploracji tekstu, udało się pokonać barierę interpretacyjną, jaką stanowi hermetyczny żargon społeczności graczy, co dotychczas było istotnym ograniczeniem standardowych systemów analizy sentymentu.

Osiągnięty zakres prac obejmuje stworzenie kompletnego i skalowalnego ekosystemu przetwarzania danych, od w pełni zautomatyzowanej akwizycji recenzji z platformy Steam, przez zaawansowany preprocessing, aż po implementację modelu regresji logistycznej o wysokim poziomie interpretowalności. Przeprowadzone testy walidacyjne wykazały, że model nie tylko charakteryzuje się wysoką dokładnością klasyfikacji, ale przede wszystkim skutecznie wychwytuje niuanse znaczeniowe specyficzne dla gatunku soulslike. Wdrożenie wyników do interaktywnego środowiska Tableau Desktop pozwoliło na przekształcenie surowych danych w strategiczną wiedzę biznesową.

Opracowane pulpity menedżerskie umożliwiają kadrze zarządzającej precyzyjną, ilościową ocenę elementów tworzonych produktów, które mają realny wpływ na pozytywny odbiór produktu. Tym samym praca stanowi gotową podstawę do optymalizacji procesów decyzyjnych w studiach deweloperskich. Otwiera ona również drogę do dalszego rozwoju systemu, w szczególności w obszarze analizy trendów w czasie rzeczywistym.

Bibliografia

1. Biswas, S., Mohammad, W. i Rajan, H., 2022. *The Art and Practice of Data Science Pipelines*, Ames: Iowa State University.
2. Cantone, G. G., Tomaselli, V. i Mazzeo, V., 2024. Review bombing: ideology-driven polarisation in online ratings: The case study of The Last of Us (part II). *Springer*.
3. Chapman, P. i inni, 2000. *CRISP-DM 1.0: Step-by-step data mining guide*. Clevedon: SPSS.
4. Feldman, R. i Sanger, J., 2009. *The Text Mining Handbook*. Cambridge: Cambridge University Press.
5. Grootendorst, M., 2022. *BERTopic: Neural topic modeling with a class-based TF-IDF procedure*. [Online]
Available at: <https://arxiv.org/abs/2203.05794>
6. Han, J., Kamber, M. i Jian, P., 2011. *Data mining. Concepts and Techniques*. 3rd red. brak miejsca: Morgan Kaufmann Publishers In.
7. Heimerl, F., Lohmann, S., Lange, S. i Ertl, T., 2024. Word Cloud Explorer: Text Analytics Based on Word Clouds. *Hawaii International Conference on System Sciences*, 6 January.
8. Janik, J., 2022. *Gry jako obiekt oporny*. Kraków: Wydawnictwo Uniwersytetu Jagiellońskiego.
9. Jurafsky, D. i Martin, H. J., 2025. *Speech and Language Processing*. 3rd red. Stanford: Stanford University.
10. Kaplan, J. i inni, 2020. *Scaling Laws for Neural Language Models*, brak miejsca: OpenAI.
11. Kuhn, M. i Johnson, K., 2013. *Applied Predictive Modeling*. Michigan: Springer Science+Business Media New York.
12. Kusleika, D., 2021. *Data visualization with Excel® dashboards and reports*. Indianapolis: Wiley & Sons, Incorporated.
13. Lefever, E., Van Hee, C. i Hoste, V., 2018. *SemEval-2018 Task 3: Irony Detection in English Tweets*, New Orleans: Association for Computational Linguistics.
14. Liu, B., 2015. *Sentiment Analysis: Mining Opinions, Sentiments, and Emotions*. Cambridge: Cambridge University Press.
15. McKinney, W., 2022. *Python for Data Analysis*. 3rd red. brak miejsca: O'Reilly Media.
16. Mitchell, R., 2018. *Web Scraping with Python. Collecting More Data from the Modern Web*. 2nd red. brak miejsca: O'Reilly Media.
17. Miyazaki, H., 2023. *Elden Ring Creator Is Looking to Multiplayer Games Like Escape From Tarkov for Inspiration* [Wywiad] (27 February 2023).
18. Miyazaki, H., 2024. *'I have always felt the world was a harsh place': Elden Ring's Hidetaka Miyazaki on why he may never stop making games* [Wywiad] (21 June 2024).
19. Miyazaki, H. i Yamamura, M., 2022. *Exclusive: The First Armored Core 6 Details With Hidetaka Miyazaki and Masaru Yamamura* [Wywiad] (17 December 2022).
20. Obinnaya, N., 2023. How Artificial Intelligence Influences Project Management. *ResearchGate*.

21. Pang, B., Lee, L. i Vaithyanathan, S., 2002. Thumbs up? Sentiment Classification using Machine Learning Techniques. W: *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing ({EMNLP} 2002)*. brak miejsca: Association for Computational Linguistics, pp. 79-86.
22. Schuller, B., Cambria, E., Xia, Y. i Havasi, C., 2013. New Avenues in Opinion Mining and Sentiment Analysis. *IEEE Intelligent Systems*, 21 February, pp. 15-21.
23. Teh, W. Y., Jordan, I. M., Beal, J. M. i Blei, M. D., 2005. *Hierarchical Dirichlet Processes*. 1st red. Berkeley: Berkeley University.

Spis rysunków

Rysunek 1 Przedstawienie istoty NLP	6
Rysunek 2 Schemat działania CRISP-DM.....	14
Rysunek 3 Środowisko PyCharm jako przykładowe IDE języka Python.....	15
Rysunek 4 Wygląd interfejsu środowiska PyCharm jako przykładowego IDE dla języka Python.....	16
Rysunek 5 Środowisko RStudio jako przykładowe IDE języka R	17
Rysunek 6 Interfejs Altair AI Studio.....	19
Rysunek 7 Wygląd interfejsu MS Power BI	20
Rysunek 8 Wygląd środowiska Tableau Desktop.....	21
Rysunek 9 Strona internetowa przejęta przez WebDriver	23
Rysunek 10 Surowe pobrane dane ze Steam.....	24
Rysunek 11 Załadowana próbka danych do środowiska Python	25
Rysunek 12 Oczyszczone dane z niedoskonałości.....	25
Rysunek 13 Wynik przygotowania danych.....	26
Rysunek 14 Przykładowy proces przygotowania danych tekstowych.....	26
Rysunek 15 Treść przykładowej opinii.....	26
Rysunek 16 Efekt normalizacji i tokenizacji.....	27
Rysunek 17 Zbiór tokenów po usunięciu stopwords	27
Rysunek 18: Przykładowa analiza sentymentu na przykładowych danych	29
Rysunek 19 Pulpit podstawowych informacji.....	34
Rysunek 20 Matryca korelacji siły oddziaływania czynników	35
Rysunek 21 Import bibliotek.....	36
Rysunek 22 Mapowanie słownika identyfikowanych gier	36
Rysunek 23 Zdefiniowanie funkcji get_reviews()	37
Rysunek 24 Mechanizm przewijania strony i lokalizacja elementów	37
Rysunek 25 Logika limitowania przetwarzanych danych.....	37
Rysunek 26 Pętla ekstrahująca dane z obsługą błędów	38
Rysunek 27 Funkcja główna i konfiguracja słownika	38
Rysunek 28 Pętla wykonawcza i zapis danych	39
Rysunek 29 Wygląd danych po imporcie	40
Rysunek 30 Import bibliotek.....	40
Rysunek 31 Łączenie zbiorów danych z różnych plików w jedną ramkę danych.....	41
Rysunek 32 Usuwanie artefaktów technicznych z tekstu recenzji i daty.....	41
Rysunek 33 Binarizacja zmiennej rekomendacji	42
Rysunek 34 Funkcja normalizująca zróżnicowane formaty daty.....	42
Rysunek 35 Zastosowanie konwersji daty w ramce danych	43
Rysunek 36 Dane po transformacjach.....	43
Rysunek 37 Funkcja filtrująca spam	44
Rysunek 38 Konfiguracja niestandardowej listy stopwords	44
Rysunek 39 Implementacja czyszczenia i tokenizacji tekstu.....	45
Rysunek 40 Kolumna cleaned_review po usunięciu stopwords	45
Rysunek 41 Przeprowadzenie lematyzacji tekstu	46
Rysunek 42 Wynik funkcji .info().....	46
Rysunek 43: Lista słów w opiniach.....	47
Rysunek 44 Lista bigramów w opiniach.....	47
Rysunek 45 Rozkład charakteru opinii	48
Rysunek 46 Rozkład ze względu na charakter opinii w czasie.....	49
Rysunek 47 Histogram ilości znaków	49
Rysunek 48 Histogram ilości słów.....	50

Rysunek 49 Przygotowanie zmiennych wejściowych i decyzyjnych do modelowania.....	51
Rysunek 50 Podział danych na zbiór treningowy i testowy	51
Rysunek 51 Separacja klas decyzyjnych w zbiorze treningowym	51
Rysunek 52 Równoważenie klas metodą downsamplingu.....	52
Rysunek 53 Finalne przygotowanie zbalansowanego zbioru treningowego	52
Rysunek 54 Konfiguracja wektoryzatora TF-IDF	52
Rysunek 55 Transformacja tekstu na macierze numeryczne.....	53
Rysunek 56 Trenowanie modelu regresji logistycznych	53
Rysunek 57 Generowanie predykcji dla zbioru testowego	53
Rysunek 58 Obliczanie i wyświetlanie metryk skuteczności modelu	53
Rysunek 59 Generowanie i wyświetlanie szczegółowego raportu klasyfikacyjnego	54
Rysunek 60 Wygląd przykładowych statystyk modelu.....	54
Rysunek 61 Ekstrakcja wag wydźwięku z modelu	54
Rysunek 62 Przykładowe wagi wydźwięku	55
Rysunek 63 Zliczanie częstości występowania pojedynczych słów	56
Rysunek 64 Generowanie i zliczanie częstości występowania bigramów	56
Rysunek 65 Agregacja liczników słów i bigramów w jedną ramkę danych	57
Rysunek 66 Integracja wag sentymentu z danymi o liczebności	57
Rysunek 67 Przykładowy wynik ramki merged.....	57
Rysunek 68 Optymalizowanie słownika tematów (spłaszczanie struktury)	58
Rysunek 69 Inicjalizacja listy na wyniki kategoryzacji	59
Rysunek 70 Algorytm pętli przypisującej kategorii tematyczne do tokenów	59
Rysunek 71 Finalne przypisanie zidentyfikowanych tematów do ramki danych	59
Rysunek 72 Ostateczna struktura danych	60
Rysunek 73 Eksport przetworzonych danych analitycznych do pliku Excel.....	60
Rysunek 74 Pole obliczeniowe dla wartości binarnej	61
Rysunek 75 Obliczanie procentowego wyniku Steam	61
Rysunek 76 Panel wskaźników wydajności	61
Rysunek 77 Formuła siły oddziaływania czynnika	62
Rysunek 78 Obsługa wartości pustych w kolumnie czynników	62
Rysunek 79 Wykres sentymentu według kategorii tematycznych	62
Rysunek 80 Wykres liczebności kategorii tematycznych w opiniach.....	63
Rysunek 81 Strategiczny pulpit menedżerski.....	63
Rysunek 82 Matryca korelacji siły oddziaływania i liczebności.....	64
Rysunek 83 Wykres zależności Accuracy od wielkości próbki	66
Rysunek 84 Zależność współczynnika Kappa od wielkości próbki.....	66
Rysunek 85 Wykres zależności AUC od wielkości próbki.....	67
Rysunek 86 Analiza sentymentu dla danych przykładowych	68
Rysunek 87 Analiza liczebności dla danych przykładowych.....	69
Rysunek 88 Szczegółowa analiza tokenów dla kategorii Rozgrywka	70
Rysunek 89 Szczegółowa analiza tokenów dla kategorii Grafika.....	71
Rysunek 90 Szczegółowa analiza tokenów dla kategorii Fabuła	72
Rysunek 91 Szczegółowa analiza tokenów dla kategorii Wsparcie.....	73
Rysunek 92 Szczegółowa analiza tokenów dla kategorii Cena.....	74

Spis tabel

Tabela 1 Przykładowa lista 5 najczęściej występujących słów.....	27
Tabela 2 Przykładowa lista słów z zastosowanym stemmingiem.....	28
Tabela 3 Przykładowe n-gramy dla $n = 2$	28
Tabela 4 Powiązania czynników ze składnikami	31
Tabela 5: Przykładowe składowe czynników	32
Tabela 6 Przykład wiersza pliku clean_df.xlsx	32
Tabela 7 Przykładowe wiersze pliku wynik.xlsx	32
Tabela 8 Porównanie sentymentu tokenów zawierających słowo "recommend"	55
Tabela 9 Słownik tematów	58
Tabela 10 Mierniki jakości w zależności od wielkości próbki danych.....	65